



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka

w specjalności Inżynieria Oprogramowania

Rozwiązania techniczno-informatyczne wspierające inteligentny ogród

Technical and IT solutions supporting a smart garden

Michał Rutyna

99677

Mikołaj Sałek

99682

Promotor: dr inż. Tomasz Szymczyk

Streszczenie

Praca inżynierska prezentuje projekt i implementację systemu umożliwiającego zdalny monitoring parametrów środowiskowych w ogrodzie. Na rozwiązanie składają się aplikacja mobilna, stacja pomiarowa oraz serwer obsługujący żądania od stacji. Celem projektu jest stworzenie systemu łatwo dostępnego dla konsumenta.

Aplikacja mobilna została zaprogramowana w języku Kotlin, wykorzystując bibliotekę Android Jetpack i platformę Firebase. Urządzenie pomiarowe zostało oparte o mikrokontroler z rodziny ESP32 i język C++. Żądania obsługuje niewielka aplikacja serwerowa wykorzystująca Express.js.

W pracy zawarto projekt systemu, składający się z specyfikacji wymagań i najważniejszych rodzajów diagramów UML, np. diagram przypadków użycia czy diagram aktywności. Opisano także implementację zastosowanych rozwiązań, i przetestowano je z wynikiem pozytywnym. Projekt udało się zrealizować niskim kosztem części elektronicznych. Oferuje on także szeroką funkcjonalność i przyjazny interfejs użytkownika.

Słowa kluczowe: Inteligentny ogród, IoT, Android, Kotlin, Jetpack, Firebase, Firestore, C++, ESP32, Express.js

Abstract

This paper presents the design and implementation of a system allowing remote monitoring of environmental parameters in a garden. This solution consists of a mobile application, a measurement station and a server, which handles requests from the station. The objective of this project was to make a customer-friendly system.

The mobile application has been programmed in the Kotlin programming language, and utilises the Android Jetpack library and the Firebase platform. The measurement device has been based on the ESP32 microcontroller, and programmed using C++. Requests from this device are handled by a small application using Express.js.

This paper contains the design of the system, consisting of enumeration of requirements along with a few of the most important UML diagrams, for example a use case diagram or activity diagrams. The work also comprises the implementation of used solutions and the successful results of testing. The project was completed using cheap electronic parts and offers a wide range of functionality along with user-friendly interface.

Keywords: Smart garden, IoT, Android, Kotlin, Jetpack, Firebase, Firestore, C++, ESP32, Express.js

Spis treści

1. Wstęp.....	9
2. Cel i zakres pracy.....	10
2.1 Cel pracy.....	10
2.2 Zakres pracy.....	10
2.3 Podział pracy.....	10
3. Analiza rynku.....	12
3.1 Ambient Weather WS-2902.....	12
3.2 ECOWITT Gateway GW1000.....	13
3.3 Spacetronek SP-ST01.....	14
4. Projekt systemu.....	15
4.1 Wymagania funkcjonalne.....	15
4.2 Wymagania niefunkcjonalne.....	16
4.3 Diagram przypadków użycia.....	17
4.4 Diagramy sekwencji.....	18
4.5 Diagramy aktywności.....	20
4.6 Diagram klas.....	24
4.7 Schemat bazy danych.....	25
4.8 Interfejs.....	29
5. Wykorzystanie technologie.....	34
5.1 Języki programowania i frameworki.....	34
5.2 Narzędzia programistyczne.....	41
6. Implementacja.....	42
6.1 Aplikacja serwerowa.....	42
6.2 Stacja pomiarowa.....	49
6.3 Aplikacja mobilna.....	64
7. Testowanie.....	102
7.1 Testy serwera.....	102
7.2 Testy aplikacji mobilnej.....	104
8. Wnioski.....	111
Literatura.....	113

1. Wstęp

Rozwiązania Internetu Rzeczy IoT (ang. internet of things) zdają się rosnać na popularności z każdym rokiem. Jedną z gałęzi, w której nowoczesne rozwiązania techniczno – informatyczne znajdują zastosowanie jest szeroko pojęte zarządzanie ogrodami, uprawami i innymi terenami zielonymi. Właściciele roślin coraz częściej sięgają po zautomatyzowane rozwiązania, określane mianem inteligentnego ogrodu.

Inteligentny ogród to system składający się z czujników, elementów wykonawczych i programów, który ma na celu monitorowanie, a niekiedy także automatyczne sterowanie warunkami środowiskowymi. Analizie często podlegają takie dane jak wilgotność gleby, temperatura czy nasłonecznienie. Po agregacji, pomiary tych wartości stanowią wygodne medium do obserwacji warunków, w jakich żyją rośliny.

Celem niniejszej pracy jest zaprojektowanie i implementacja takiego systemu w odpowiedzi na zapotrzebowania użytkowników. Projekt skupia się na monitorowaniu i analizie parametrów środowiskowych. W pracy przedstawiono zarówno aspekty teoretyczne związane z zastosowanymi technologiami, jak i praktyczne zagadnienia dotyczące realizacji sprzętowej oraz programowej systemu.

W aspekcie biznesowym, system ten skupia się na wypełnieniu braku na polskim rynku rozwiązań ogrodowych. Mianowicie, wszystkie systemy inteligentnego ogrodu są drogie, oferują małe możliwości pomiarowe, lub są bardzo nieprzyjazne dla użytkownika. Ta praca stara się odwrócić tą sytuację. Zastosowane czujniki są bardzo proste i łatwo dostępne, co pozwoli zapewnić niski koszt przy wielu mierzonych właściwościach. Ergonomiczne doświadczenia użytkownika zapewni aplikacja mobilna zaprogramowana zgodnie z najnowszymi zaleceniami dotyczącymi interfejsu [23]. Ważnym aspektem tej aplikacji jest przyjazność nowemu użytkownikowi. System ten powinien być dostępny dla każdej grupy konsumentów

2. Cel i zakres pracy

2.1 Cel pracy

Celem pracy jest stworzenie systemu obsługi ogrodu opartego na mikrokontrolerze z serii ESP32. Ponadto, należy zaprojektować i oprogramować aplikację mobilną umożliwiającą ergonomiczne korzystanie z systemu. Aplikacja mobilna zostanie oprogramowana z wykorzystaniem technologii Kotlin. Celem systemu informatycznego jest wspomaganie monitorowania parametrów atmosferycznych i glebowych w ogrodzie. Pomiarowi podlegają: wilgotność gleby i powietrza, temperatura powietrza, ciśnienie atmosferyczne i poziom nasłonecznienia. Użytkownik po zainstalowaniu stacji w ziemi i skonfigurowaniu jej, może w telefonie sprawdzać parametry otoczenia. Ma również możliwość ustawienia alarmu, powiadamiającego o przekroczeniu konkretnego parametru poza ustawiony zakres.

2.2 Zakres pracy

Zakres pracy obejmuje:

- analizę istniejących na rynku konkurencyjnych rozwiązań;
- sporządzenie diagramów przypadków użycia, sekwencji, aktywności i klas;
- określenie wymagań funkcjonalnych i niefunkcjonalnych systemu;
- opracowanie diagramu ERD i doboru optymalnej bazy danych;
- zaimplementowanie bazy danych, aplikacji serwerowej i mobilnej;
- zaprojektowanie struktury i wzajemnych powiązań komponentów systemu;
- zaprojektowanie i wykonanie systemu wykonującego pomiary, opartego o mikrokontroler;
- przeprowadzenie testów zarówno dla aplikacji, jak i oprogramowania mikrokontrolera.

2.3 Podział pracy

Pracę stworzono w zespole dwuosobowym. Mikołaj Sałek odpowiada za ogólny schemat działania systemu oraz za część elektroniczną i oprogramowanie mikrokontrolera. Michał Rutyna odpowiada za: zaimplementowanie aplikacji mobilnej, opracowanie schematu ERD i implementację bazy danych. Prace projektowe, obejmujące analizę rynku, opracowanie wymagań funkcjonalnych i niefunkcjonalnych.

Przygotowanie aplikacji serwerowej oraz wykonanie diagramów, zostanie zrealizowane wspólnie przez zespół. Szczegółowy podział prac znajduje się w tabeli 2.1

Tabela 2.1 Dokładny podział pracy

Mikołaj Sałek	• Ogólny projekt systemu • Zaprojektowanie i implementacja systemu mikrokontrolera • Testy jednostkowe i manualne systemu pomiarowego
Michał Rutyna	• Implementacja aplikacji mobilnej • Implementacja i projekt bazy danych • Testy jednostkowe i integracyjne aplikacji mobilnej i serwera
Część wspólna	• Implementacja serwera • Analiza rynkowa • Diagramy przypadków użycia, sekwencji, aktywności i klas • Przegląd konkurencyjnych rozwiązań

3. Analiza rynku

Celem przeprowadzonej analizy rynkowej jest rozpoznanie aktualnie dostępnych produktów oferujących funkcje zbliżone do planowanego systemu.

3.1 Ambient Weather WS-2902

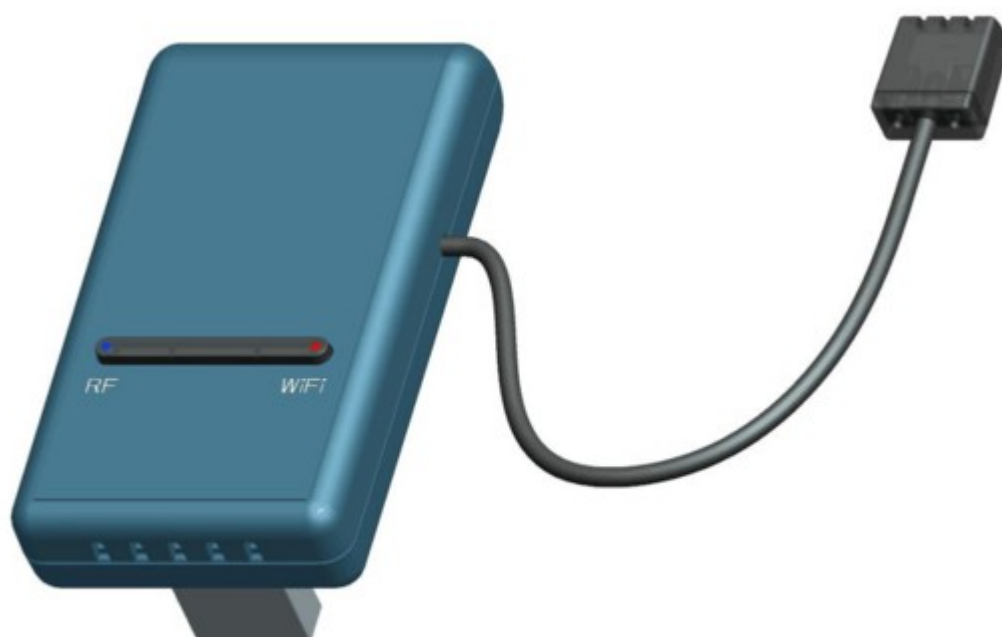
Stacja pogodowa Ambient Weather WS-2902 [30] to zaawansowany system, służący do monitorowania warunków atmosferycznych. Składa się z zewnętrznego zestawu czujników, które mierzą temperaturę i wilgotność powietrza, prędkość oraz kierunek wiatru, opady deszczu, promieniowanie słoneczne i promieniowanie UV. Urządzenie posiada także moduł Wi-Fi, umożliwiający bezprzewodowe przesyłanie danych do chmury producenta (AmbientWeather.net) lub innych serwisów meteorologicznych, takich jak Weather Underground. Dane mogą być również odczytywane lokalnie za pomocą konsoli wyposażonej w kolorowy wyświetlacz LCD, prezentującej dane w czasie rzeczywistym. Dodatkowo stacja umożliwia pomiar temperatury i wilgotności wewnątrz pomieszczenia, co pozwala na kompleksową ocenę warunków środowiskowych zarówno na zewnątrz, jak i domu. Na rysunku 3.1 przedstawiono model stacji



Rysunek 3.1 Główny moduł stacji pogodowej WS-2902 [30]

3.2 ECOWITT Gateway GW1000

Jest to centralny moduł-bramka WiFi dla ekologicznej stacji pogodowej/ogrodowej w ramach systemu ECOWITT [29]. Urządzenie (rys 3.2) wyposażone jest we wbudowany czujnik 3-w-1 (temperatura, wilgotność, ciśnienie barometryczne) oraz odbiornik radiowy, który może zbierać dane z wielu bezprzewodowych czujników (np. wilgotność gleby, nasłonecznienie, opad, wiatr) zgodnych z systemem ECOWITT. Bramka przesyła dane przez WiFi do chmury ECOWITT lub innych serwisów pogodowych (np. Weather Underground) i umożliwia podgląd odczytów w aplikacji mobilnej lub własnej integracji.



Rysunek 3.2 Bramka WiFi ECOWITT Gateway GW1000 [29]

3.3 Spacetronek SP-ST01

Spacetronek SP-ST01 [28] to bezprzewodowy czujnik wilgotności i temperatury gleby przeznaczony do precyzyjnego monitorowania środowiska. Urządzenie łączy się za pośrednictwem aplikacji Tuya lub Smart Life i działa poprzez bramkę internetową SL-G02. Obie aplikacje zapewniają odczyt parametrów w czasie rzeczywistym, definiowane przez użytkownika alerty dotyczące poziomów progowych, dostęp do danych historycznych oraz integrację z systemami automatyki, takimi jak zawory, pompy i zraszacze. Wilgotność gleby jest mierzona na podstawie jej rezystancji elektrycznej. Urządzenie jest zasilane dwiema bateriami AAA, a producent deklaruje żywotność baterii do sześciu miesięcy w typowych warunkach pracy.



Rysunek 3.3 Czujnik Spacetronek SP-ST01 [28]

4. Projekt systemu

Analizując szczegółowo dostępne obecnie na rynku czujniki można stwierdzić, że brakuje rozwiązania jednocześnie taniego i oferującego szerokie możliwości pomiarowe. Celem opisywanej pracy jest zaprojektowanie takiego systemu, który udostępni wiele możliwości pomiarowych i niski koszt.

4.1 Wymagania funkcjonalne

Celem zapewnienia użyteczności systemu, konieczne będzie zapewnienie określonych funkcjonalności. Wymagania funkcjonalne dla systemu obejmują wszystkie operacje, które są istotne dla jego działania. Poniżej zostały przedstawione te wymagania:

- Logowanie do aplikacji odbywa się przy użyciu numeru telefonu użytkownika.
- Aplikacja mobilna umożliwia połączenie ze stacjami pomiarowymi za pomocą interfejsu Bluetooth.
- Aplikacja mobilna może łączyć się z serwerem za pośrednictwem internetu w celu wymiany danych pomiarowych oraz synchronizacji ustawień.
- System umożliwia za pomocą aplikacji mobilnej konfigurację parametrów stacji pomiarowych połączonych przez Bluetooth, takich jak połączenie z siecią Wi-Fi i częstotliwość wykonywania pomiarów.
- Użytkownik ma możliwość przypisywania stacji pomiarowych do swojego konta w aplikacji mobilnej.
- Aplikacja mobilna wyświetla pomiary ze wszystkich przypisanych stacji pomiarowych.
- Aplikacja mobilna umożliwia wizualizację danych w formie wykresów, pokazujących wartości w czasie dla poszczególnych stacji pomiarowych.
- Użytkownik może w aplikacji konfigurować własne alarmy dla wybranych parametrów, reagujące na wykroczenie poza ustalony zakres wartości.
- Aplikacja mobilna odbiera i pokazuje powiadomienia nawet, gdy nie jest aktywnie uruchomiona.
- Serwer dla każdego odebranego pomiaru analizuje dane pomiarowe w kontekście alarmów użytkownika. W przypadku przekroczenia pożądanego zakresu przesyła powiadomienie na aplikację mobilną.

- Stacja pomiarowa rejestruje dane dotyczące wilgotności gleby i powietrza, pH ziemi, temperatury powietrza, ciśnienia atmosferycznego i poziomu nasłonecznienia.
- Stacja pomiarowa wysyła pomiary na serwer, częstotliwość czego da się ustawić w aplikacji mobilnej.

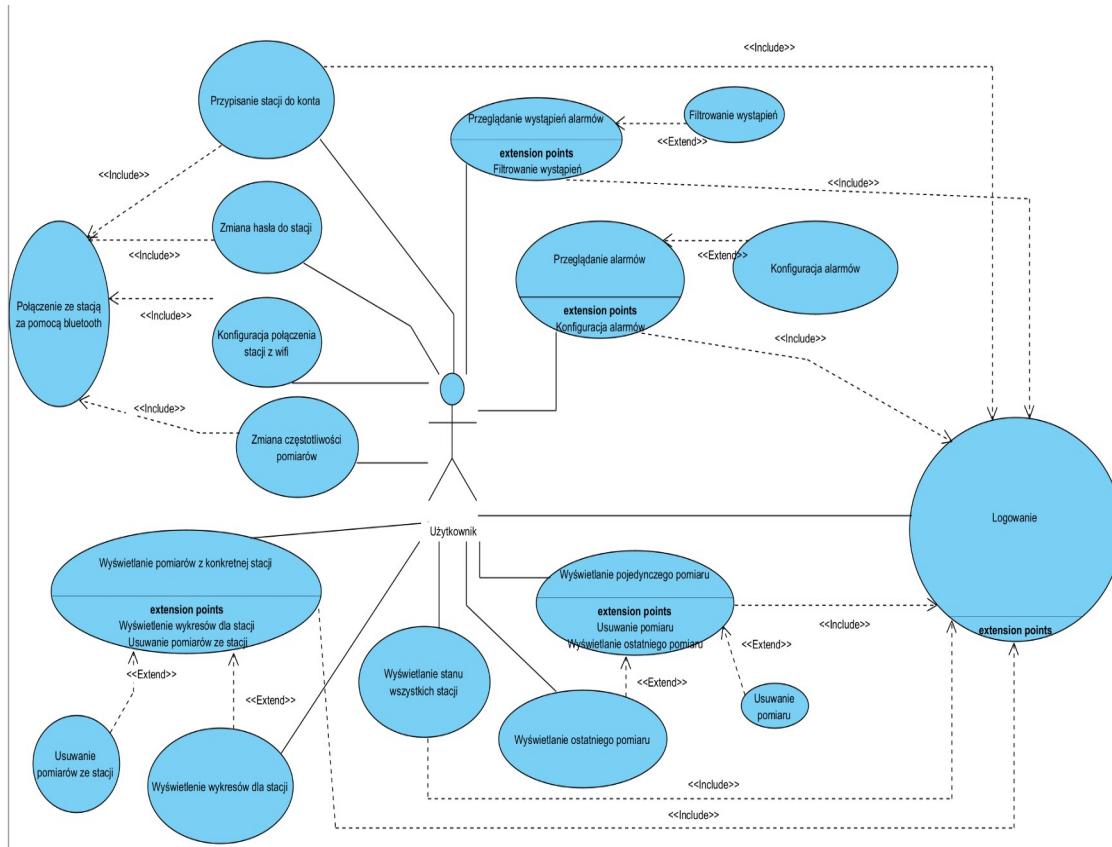
4.2 Wymagania niefunkcjonalne

Aby system informatyczny mógł działać efektywnie i zgodnie z oczekiwaniami użytkowników, określone zostały także następujące wymagania niefunkcjonalne:

- Aplikacja mobilna jest przeznaczona na urządzenia z systemem Android w wersji 11.0 lub wyższej oraz o rozdzielczości ekranu nie mniejszej niż 1080 x 2400 px i nie większej niż 1320 x 2868 px.
- Urządzenie mobilne użytkownika musi być wyposażone w funkcjonalności Bluetooth oraz odbierania SMS.
- Do odbierania powiadomień Push, urządzenie musi posiadać zainstalowane usługi Google Play w wersji wpieranej przez posiadaną wersję systemu Android.
- Stacja pomiarowa wymaga oprogramowania do zbierania danych z sensorów i wysyłania ich na serwer.
- Stacja potrzebuje do poprawnego działania połączenia Wi-Fi o minimalnej prędkości 5 Mb na sekundę.
- Stacja zaprojektowana jest na urządzenie typu ESP32
- Zasilanie stacji przez kabel USB typu C.
- Wymagana jest niewielka aplikacja serwerowa, obsługująca żądania od pozostałych części systemu, przetwarzająca dane i operująca na bazie danych.
- Serwer powinien działać w środowisku chmurowym, posiadającym minimalne specyfikacje 2 GB RAM i 10 GB miejsca na dysku twardym.

4.3 Diagram przypadków użycia

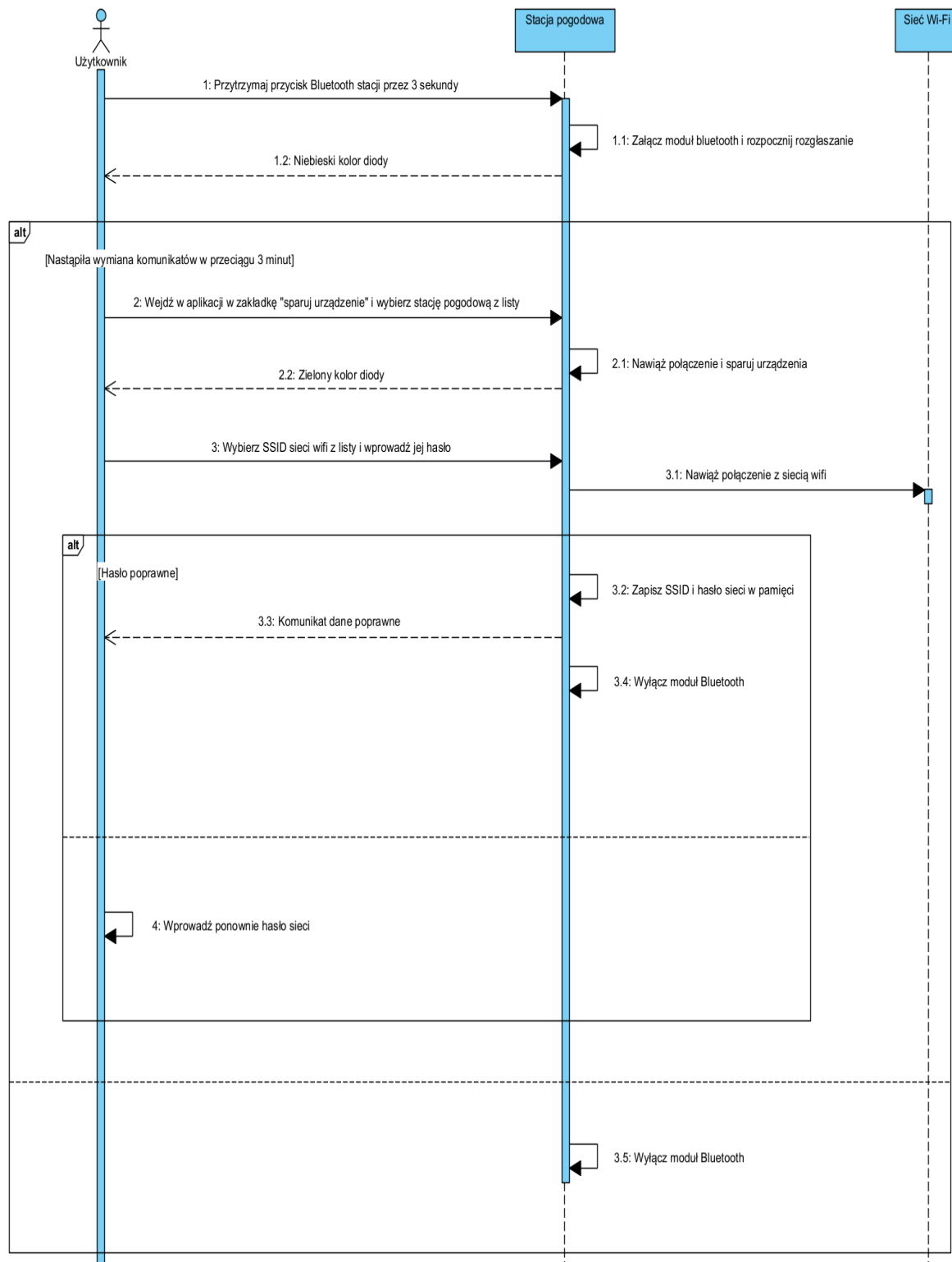
Przedstawiony diagram przypadków użycia pokazuje w zwięzły sposób wszystkie dostępne dla użytkowników funkcjonalności. Analizując go, dobrze widać ogólny zarys systemu [1]. Do większości przypadków użycia wymagane jest logowanie, stąd jego centralne położenie na diagramie. Na górnej części diagramu widoczne są funkcjonalności opierające się na konfiguracji stacji i alarmów. Poniżej znajdują się przypadki użycia polegające na wyświetlaniu danych i manipulacji nimi.



Rysunek 4.1 Diagram przypadków użycia całego systemu

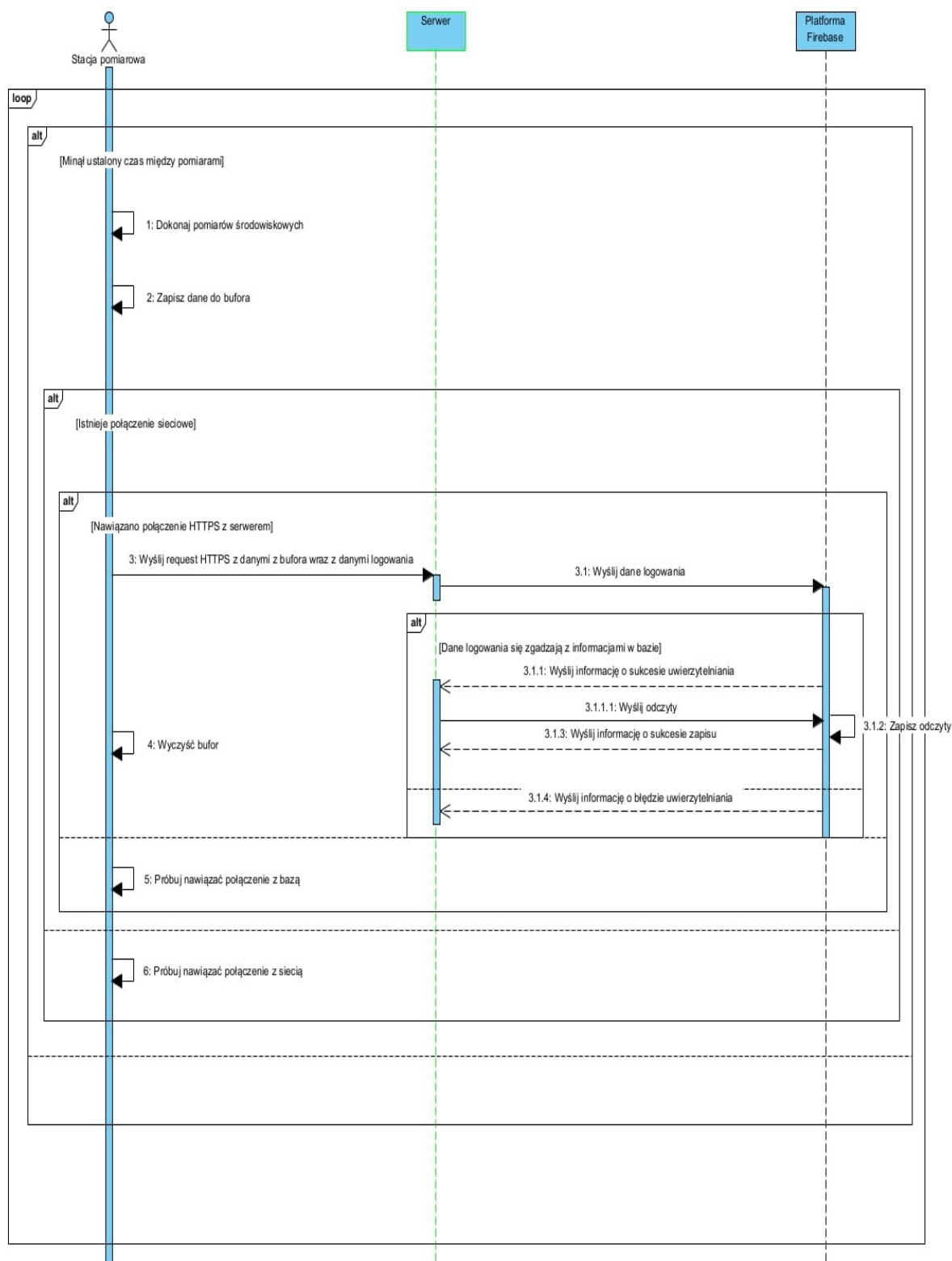
4.4 Diagramy sekwencji

Widoczny na rysunku 4.2 diagram przedstawia proces konfiguracji połączenia Wi-Fi na stacji. Ułatwia on przede wszystkim implementację procesu parowania, ale także samego zachowania stacji [1].



Rysunek 4.2 Diagram sekwencji przedstawiający połączenie stacji z siecią Wi-Fi

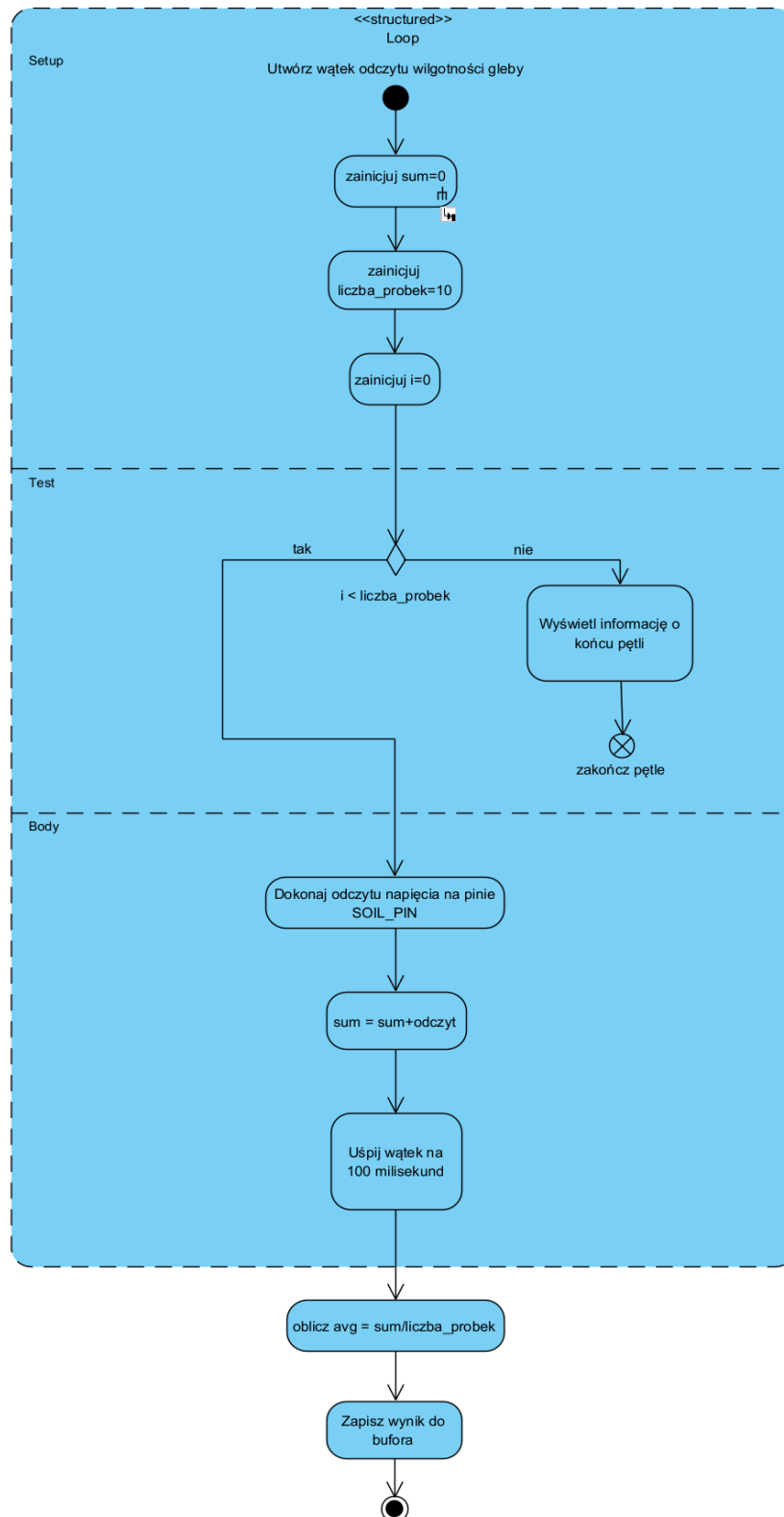
Na rysunku 4.3 przedstawiono proces wysłania pomiaru na stację, który przebiega za pośrednictwem serwera.



Rysunek 4.3 Diagram sekwencji pokazujący wysłanie pomiaru przez stację

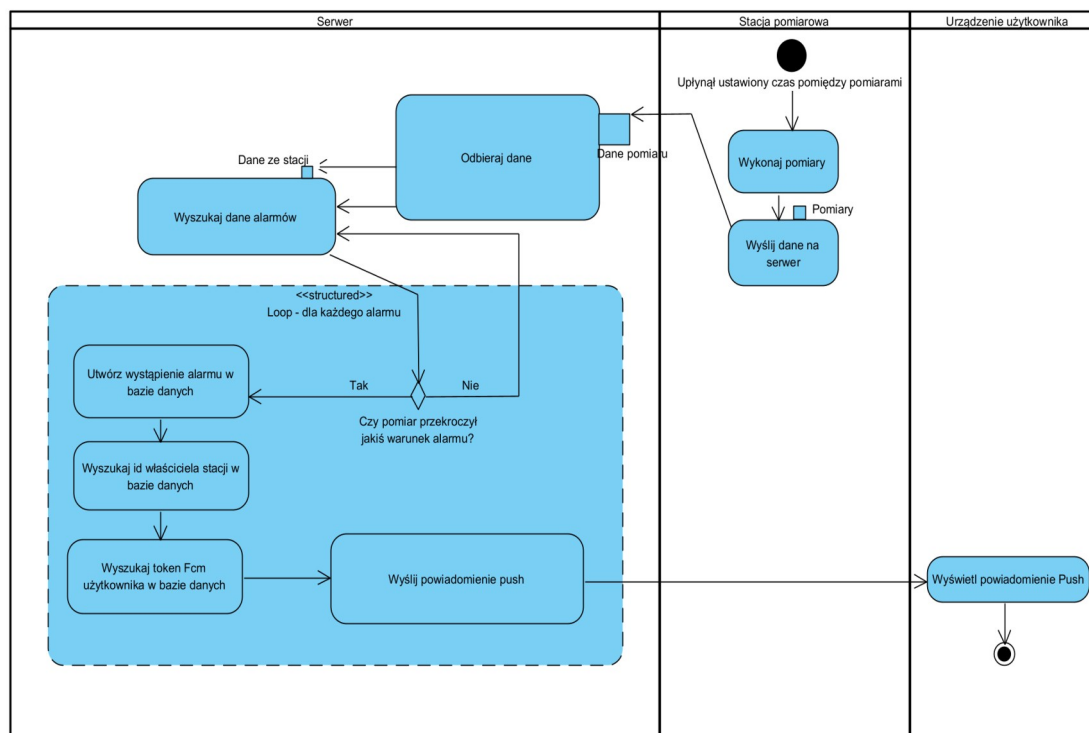
4.5 Diagramy aktywności

Diagramy na rysunku 4.4 prezentuje przebieg procesu wykonywania pomiaru. Został użyty przy implementacji tych procesów, kierując przebieg aktywności [1].



Rysunek 4.4: Diagram aktywności wykonywania pomiaru

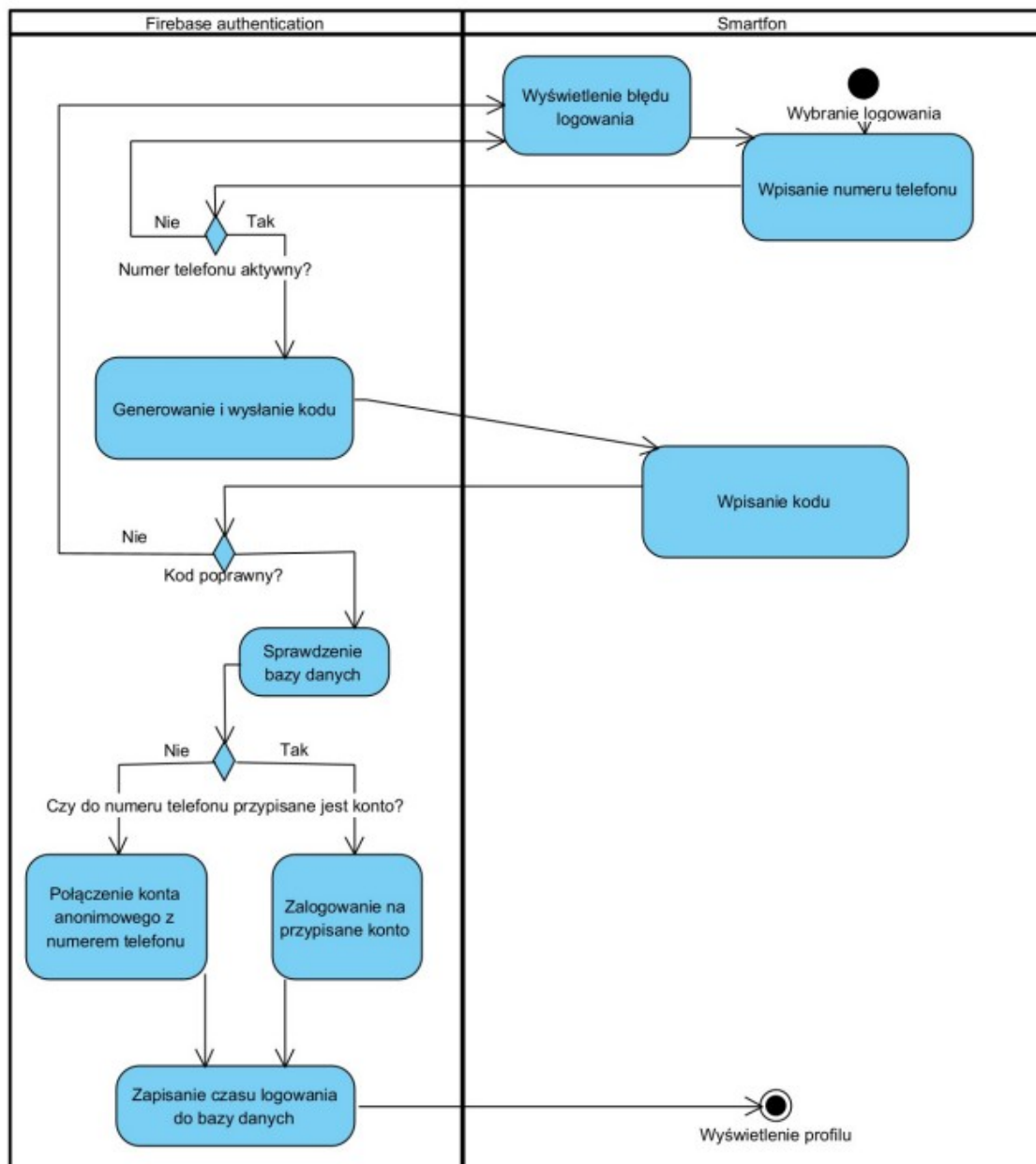
Diagram na rysunku 4.5 przedstawia proces odebrania pomiaru na serwerze i sprawdzenia alarmów. Serwer dla każdego odebranego danych sprawdza bazę danych, szukając warunków alarmu które zostały przekroczone. W przypadku napotkania takiego warunku, zapisuje on dane o tym zdarzeniu do bazy danych i przesyła powiadomienie systemem Push na odpowiednie urządzenie mobilne. Przesyłanie powiadomień odbywa się poprzez wcześniej zarejestrowane przez aplikację mobilną tokeny.



Rysunek 4.5: Diagram aktywności sprawdzania alarmów

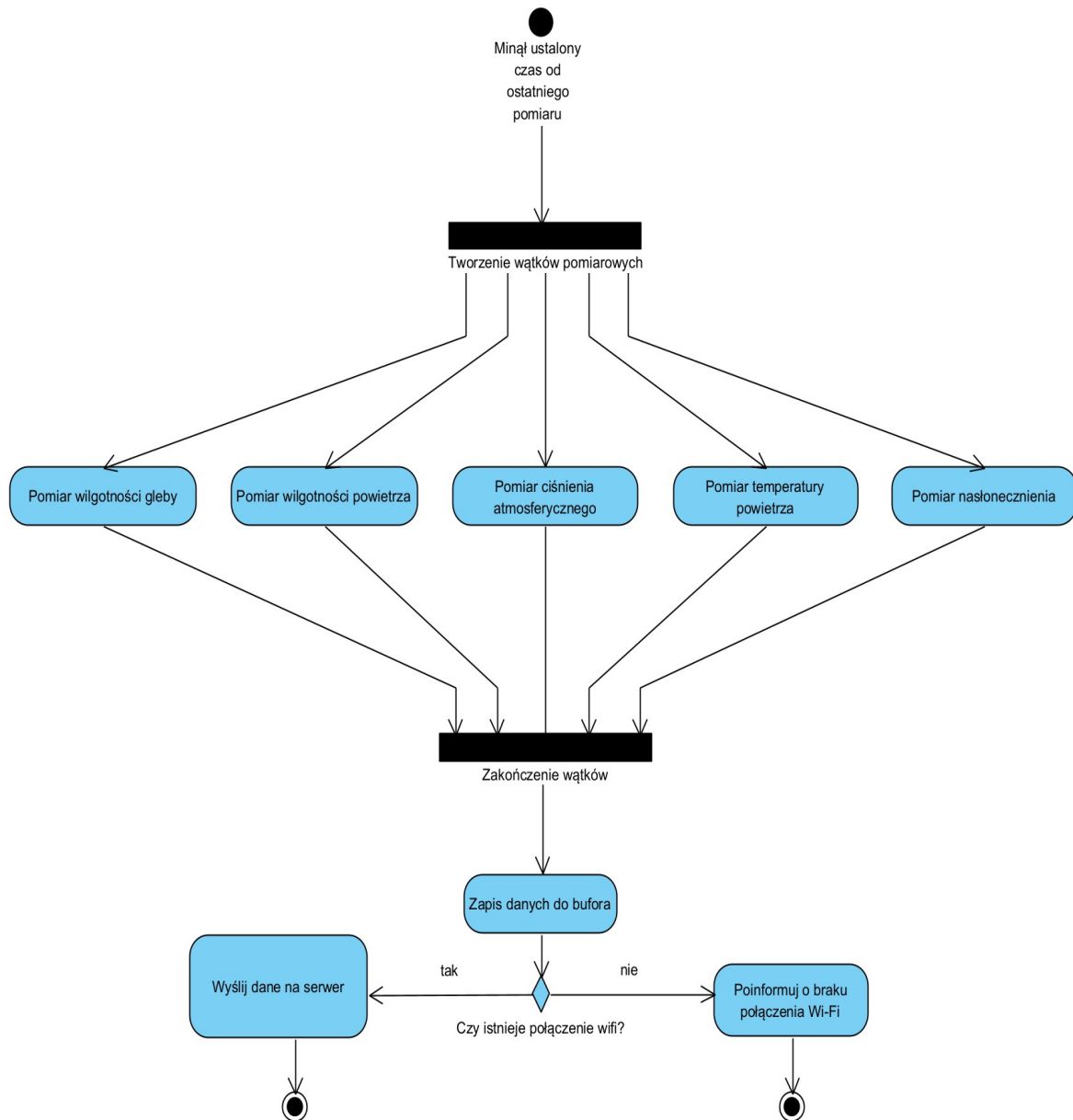
Rolą urządzenia mobilnego użytkownika w tej aktywności jest wyświetlenie powiadomienia Push. Dzieje się to poprzez usługi Google Play [27].

Rysunek 4.6 pokazuje, jak przebiega proces logowania. Aplikacja mobilna komunikuje się z platformą Firebase, która odpowiada za wszystkie kroki procesu uwierzytelniania. Odpowiedni serwis wysyła wiadomość SMS, generuje kod potwierdzający i zarządza kontami użytkowników. W przypadku pierwszego uruchomienia aplikacji tworzone jest konto anonimowe, przypisywane do numeru telefonu po logowaniu.



Rysunek 4.6: Diagram aktywności logowania

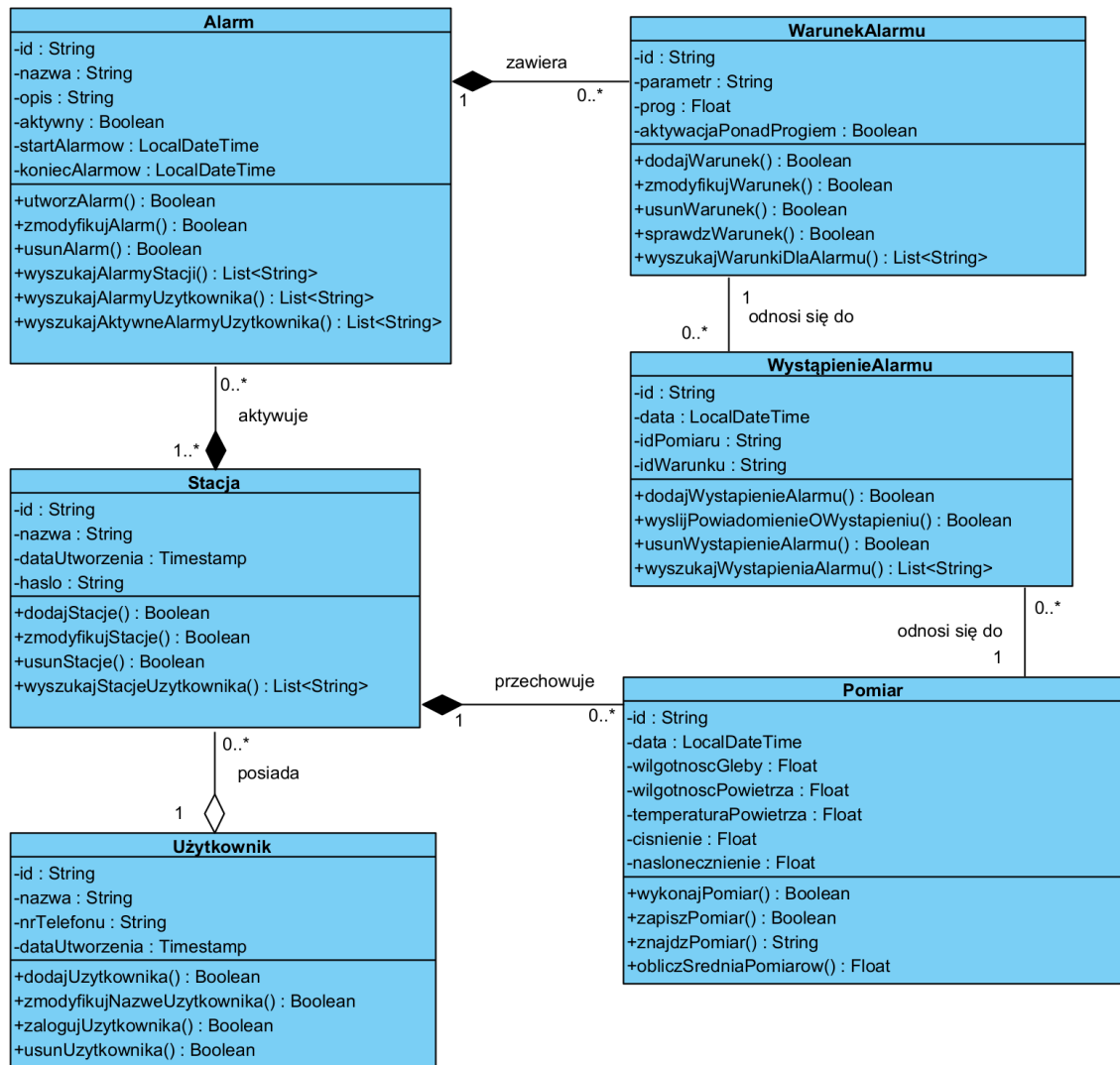
Na rysunku 4.7 widać proces okresowo przebiegający na stacji. Za jego pomocą zbierane są dane o warunkach środowiskowych. Istotą tej aktywności jest równoległe wykonywanie pomiarów. Pozwala to na optymalizację zużycia energii.



Rysunek 4.7 Diagram aktywności wykonywania pomiaru na stacji

4.6 Diagram klas

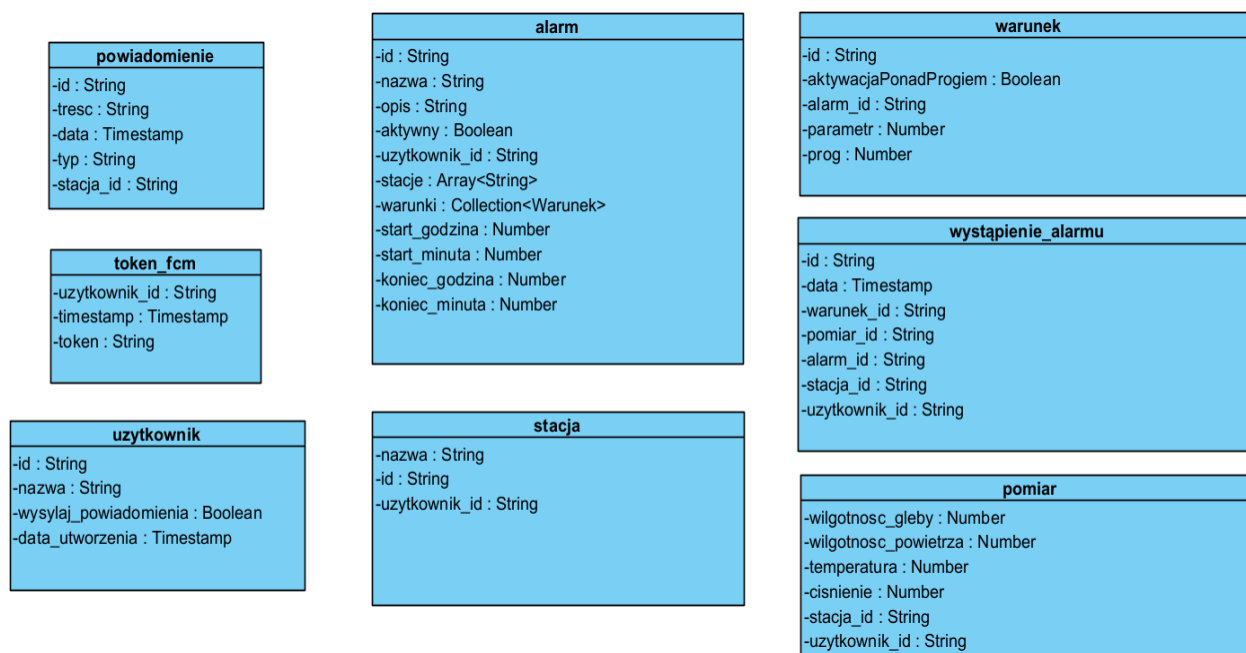
Diagram klas służy zaprezentowaniu ogólnych relacji między danymi w systemie. W tej formie, stanowi bardzo przydatne odniesienie w trakcie pracy nad niemalże każdym elementem systemu [1], gdyż pokazuje funkcje wszystkich jego części. Niektóre z zaprezentowanych na rysunku 4.8 klas są używane przez zarówno aplikację mobilną, serwer jak i stację pomiarową.



Rysunek 4.8 Diagram klas

4.7 Schemat bazy danych

Na poniższym diagramie zaprezentowano schemat nierelacyjnej bazy danych. Korzysta on z dialektu bazy danych czasu rzeczywistego Firestore. W związku z brakiem więzów integralności, nie ma na nim połączeń między encjami. Jedyne połączenie występuje między kolekcjami „alarm” i „warunek”. Oznacza ono, że „warunek” jest podkolekcją, zawartą w dokumencie kolekcji „alarm”.



Rysunek 4.9 Schematu nierelacyjnej bazy danych

Poniżej zaprezentowano tabele, opisujące kolekcje zawarte w bazie danych. System bazodanowy ogranicza tylko wielkość poszczególnych dokumentów do 1 MB, w związku z czym nie wyszczególniono ograniczeń wielkościowych konkretnych pól.

Aplikacje korzystające z bazy danych są przystosowane, aby każde pole w schemacie było opcjonalne. Najczęściej wiąże się to z pominięciem rekordu, uznając go za błędny.

Tabela 4.1 Opis kolekcji

Nazwa tabeli	Opis
uzytkownik	Kolekcja przechowująca dodatkowe informacje o użytkowniku, nieistotne z punktu widzenia logowania
stacja	Kolekcja przechowująca dane stacji pomiarowych
pomiar	Kolekcja przechowująca wszystkie pomiary jakie wykonały stacje
alarm	Kolekcja przechowująca dane o alarmie
warunek	Kolekcja przechowująca warunki dla każdego alarmu
wystapienie_alarmu	Kolekcja przechowująca wystąpienia alarmów
powiadomienie	Kolekcja przechowująca wszelkie powiadomienia dla użytkowników
token_fcm	Kolekcja przechowująca tokeny identyfikujące urządzenia do wysyłania powiadomień push

Tabela 4.2 Kolekcja "użytkownik"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
nazwa	String	Nazwa wyświetlana użytkownika
wysylaj_powiadomienia	Boolean	Określa, czy użytkownik chce otrzymywać powiadomienia push
data_utworzenia	Timestamp	Zaznacza, kiedy konto zostało utworzone

Tabela 4.3: Kolekcja "stacja"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
nazwa	String	Nazwa wyświetlana stacji
uzytkownik_id	String	Użytkownik, który jest właścicielem stacji i będzie otrzymywał z niej pomiary

Tabela 4.4 Kolekcja "pomiar"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
data	Timestamp	Data wykonania pomiaru
wilgotnosc_powietrza	Number	Wartość wilgotności powietrza
wilgotnosc_gleby	Number	Wartość wilgotności gleby
naslonecznienie	Number	Wartość nasłonecznienia
temperatura	Number	Wartość temperatury
cisnienie	Number	Wartość ciśnienia
stacja_id	String	Stacja która wykonała ten pomiar
uzytkownik_id	String	Właściciel stacji która wykonała pomiar

Tabela 4.5 Kolekcja "alarm"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
nazwa	String	Nazwa wyświetlana alarmu
opis	String	Dodatkowy opis alarmu dla użytkownika
aktywny	Boolean	Oznacza, czy ten alarm będzie tworzył wystąpienia
start_godzina	Timestamp	Oznacza godzinę od której alarm będzie tworzył wystąpienia
start_minuta	Timestamp	Oznacza minutę od której alarm będzie tworzył wystąpienia
koniec_godzina	Timestamp	Oznacza godzinę do której alarm będzie tworzył wystąpienia
koniec_minuta	Timestamp	Oznacza minutę do której alarm będzie tworzył wystąpienia
uzytkownik_id	String	Odnosi się do użytkownika który jest właścicielem alarmu
stacje	Array<String>	Przechowuje listę id stacji, które sprawdzają ten alarm
warunki	Collection<warunek>	Oznacza zgromadzoną kolekcję warunków

Tabela 4.6 Zagnieżdżona kolekcja "warunek"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
alarm_id	String	Alarm który sprawdza ten warunek
parametr	String	Parametr którego wartości sprawdza ten warunek
prog	Number	Wartość po której przekroczeniu zostanie utworzone wystąpienie alarmu
wyzej_niz_prog	Boolean	Oznacza, czy warunek sprawdza wartości mniejsze czy większe niż próg

Tabela 4.7 Kolekcja "wystapienie_alarmu"

Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
data	Timestamp	Data wystąpienia alarmu
warunek_id	String	Warunek alarmu który został przekroczony
alarm_id	String	Alarm który został wywołany
stacja_id	String	Stacja która wykonała wywołujący pomiar
pomiar_id	String	Pomiar który przekroczył wartość warunku
uzytkownik_id	String	Właściciel stacji która wykonała wywołujący pomiar

Tabela 4.8 Kolekcja "powiadomienie"

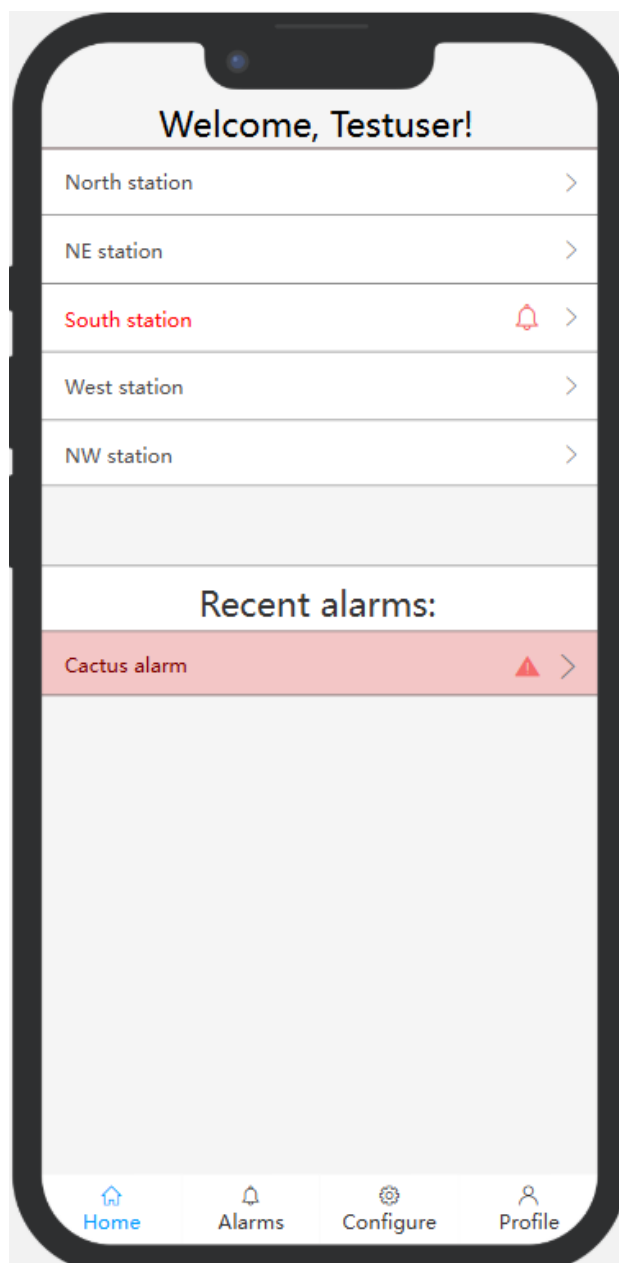
Nazwa pola	Typ pola	Opis
id	String	Identyfikator dokumentu
tresc	String	Krótką wiadomość do wyświetlenia użytkownikowi
data	Timestamp	Data utworzenia powiadomienia
typ	String	Typ do ustawienia ikony powiadomienia w aplikacji mobilnej
stacja_id	String	Stacja do której odnosi się powiadomienie

Tabela 4.9 Kolekcja "token_fcm"

Nazwa pola	Typ pola	Opis
uzytkownik_id	String	Identyfikator dokumentu, pokrywa się z identyfikatorem kolekcji „uzytkownik”
timestamp	Timestamp	Data ostatniej aktualizacji tokenu
token	String	Treść tokenu

4.8 Interfejs

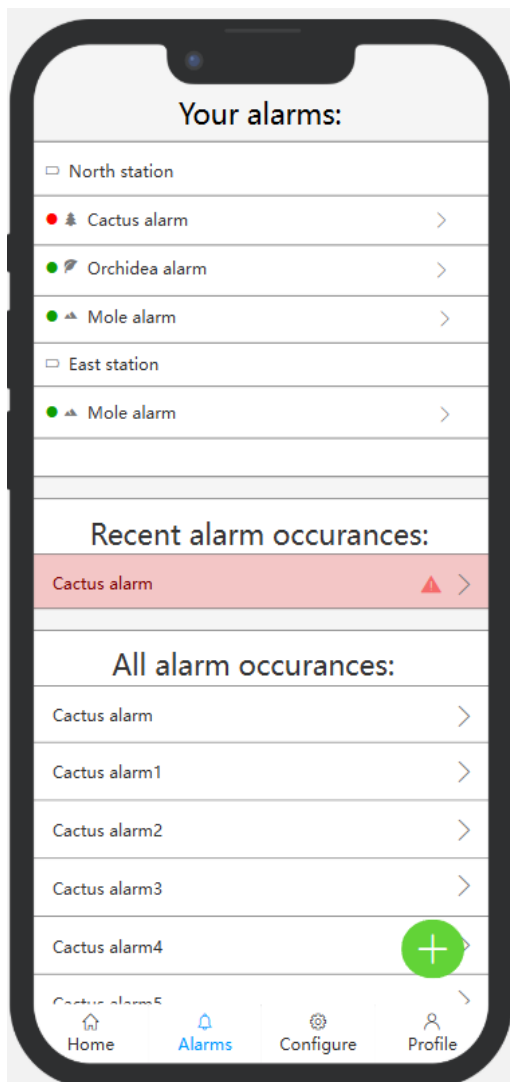
Poniżej zaprezentowane są makiety (ang. mockup) ekranów aplikacji mobilnej. Zostały one wykonane w aplikacji internetowej Figma [26]. Na podstawie tych makiet powstał ostateczny interfejs użytkownika. Drobne różnice w prezentowanej funkcjonalności wynikają ze zmian wymagań funkcjonalnych w późnej fazie implementacji. Przykładowo, logowanie hasłem zostało zastąpione logowaniem numerem telefonu.



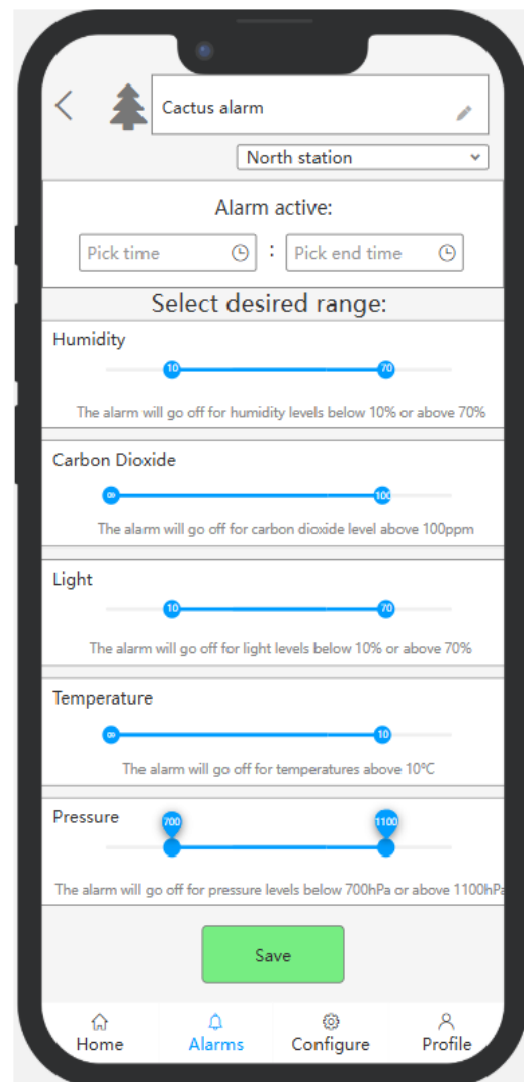
Rysunek 4.10 Mockup ekranu domowego

Aplikacja mobilna powinna skupiać się na udostępnieniu konfiguracji alarmów w sposób jak najbardziej przejrzysty. Dlatego do wyboru zakresu parametrów zaproponowanym rozwiązaniem jest suwak dwustronny.

a)



b)



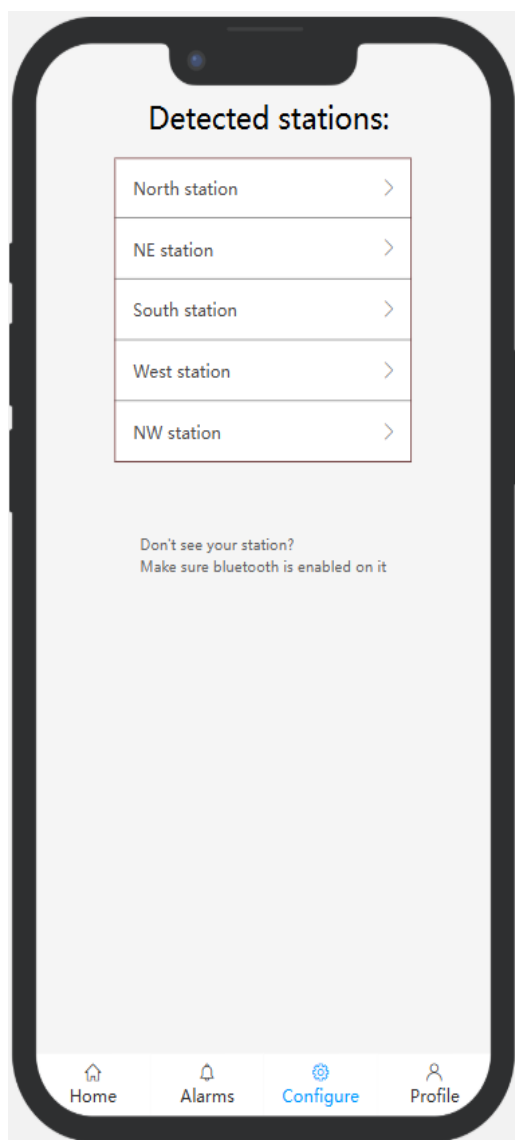
Rysunek 4.11 Mockupy ekranów alarmu

a) Ekran listy alarmów

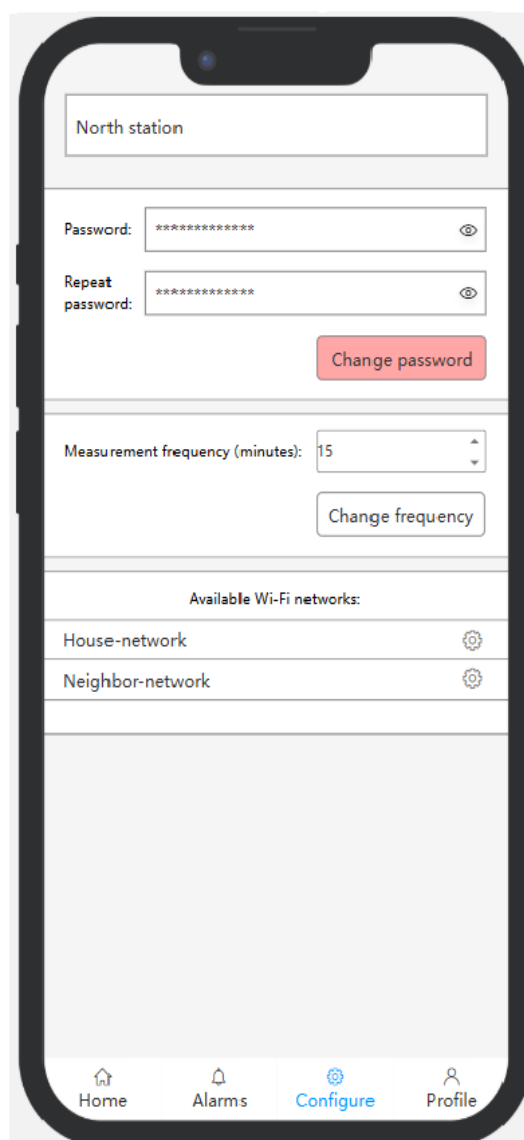
b) Ekran konfiguracji alarmu

Konfiguracja stacji powinna być procesem dostępnym dla nietechnicznego użytkownika. W związku z tym, ekrany konfiguracji zostały uproszczone do możliwie małej liczby komponentów.

a)



b)



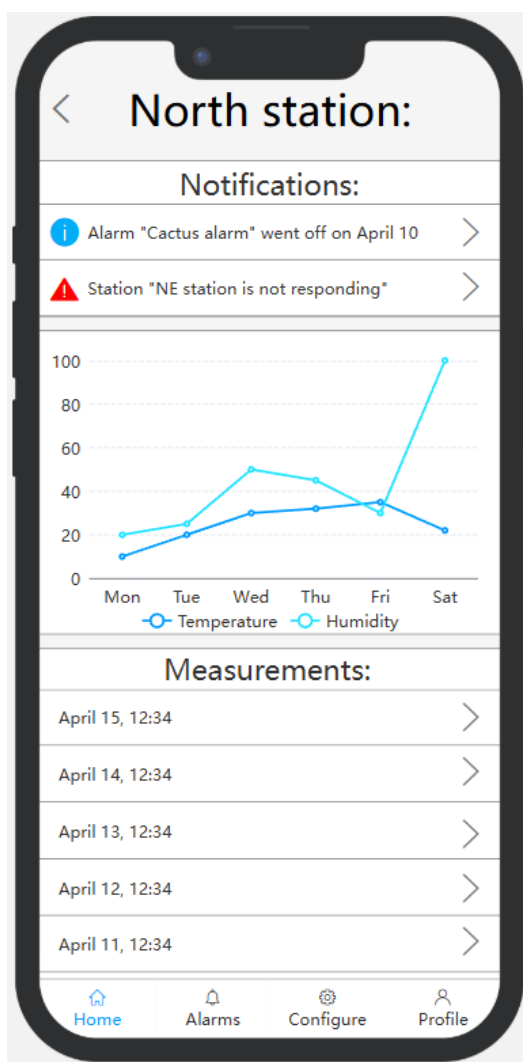
Rysunek 4.12 Mockupy ekranów konfiguracji

a) Ekran wyboru stacji

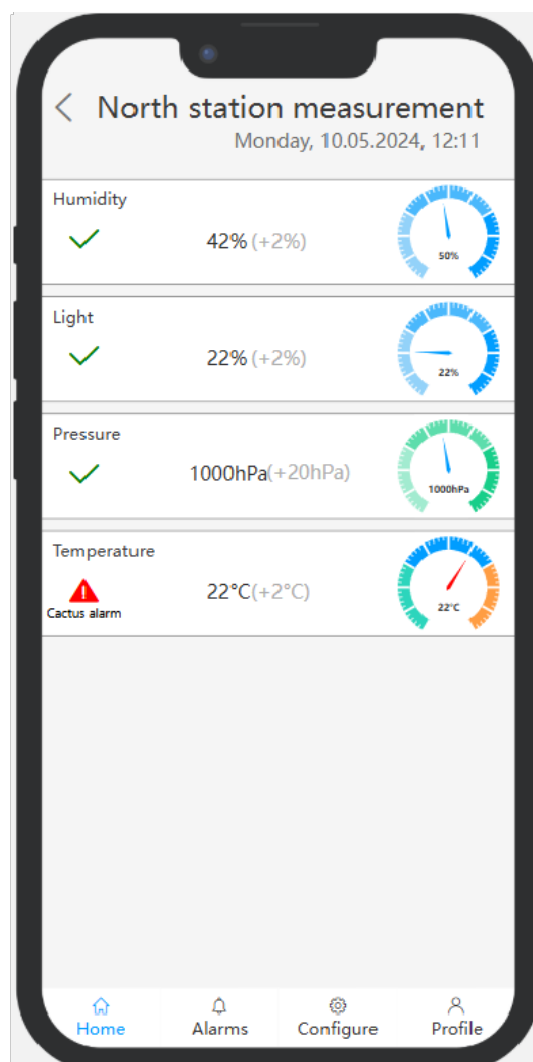
b) Ekran ustawień stacji

Ekran stacji i pomiaru są miejscami, gdzie użytkownik dowiaduje się o faktycznych danych zmierzonych przez swoje stacje. Aby ułatwić ten proces, powinny one być jak najbardziej treściwe i merytoryczne. Na ekranie pomiaru widocznym na rysunku 4.13, zawarto widoczne ostrzeżenie o uruchomionych alarmach, wskaźnik pokazujący gdzie pomiędzy wartością minimalną a maksymalną plasuje się dany pomiar oraz tekst w nawiasie oznaczający zmianę wartości od ostatniego pomiaru.

a)

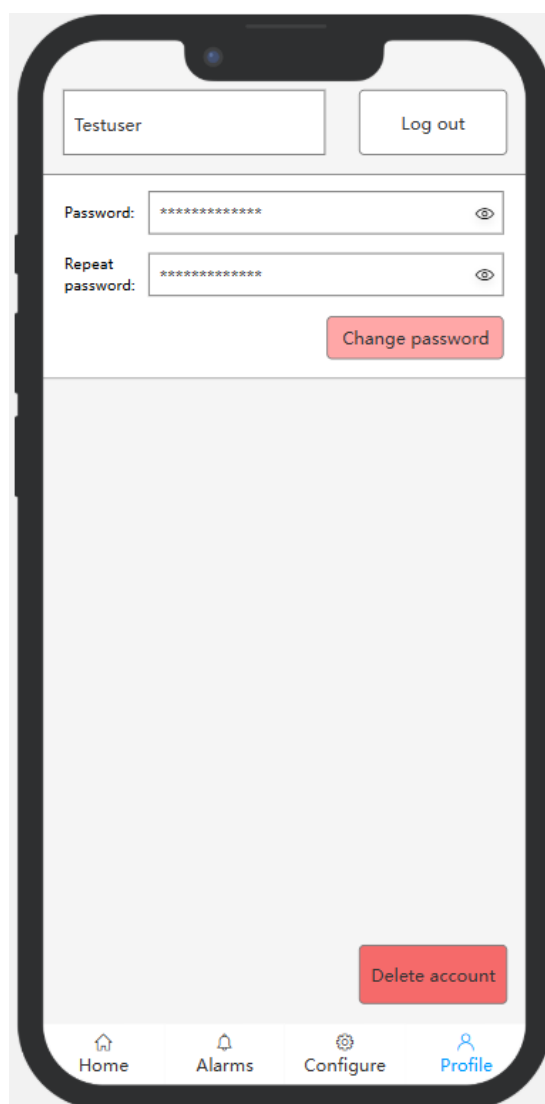


b)



Rysunek 4.13 Mockupy ekranów szczegółów
a) Ekran szczegółów stacji
b) Ekran szczegółów pomiaru

Profil użytkownika zawiera bardzo mało funkcjonalności, w związku z czym został zaprojektowany w postaci minimalistycznej. Użytkownik nie powinien czuć się przytłoczony koniecznością tworzenia konta w kolejnej aplikacji.



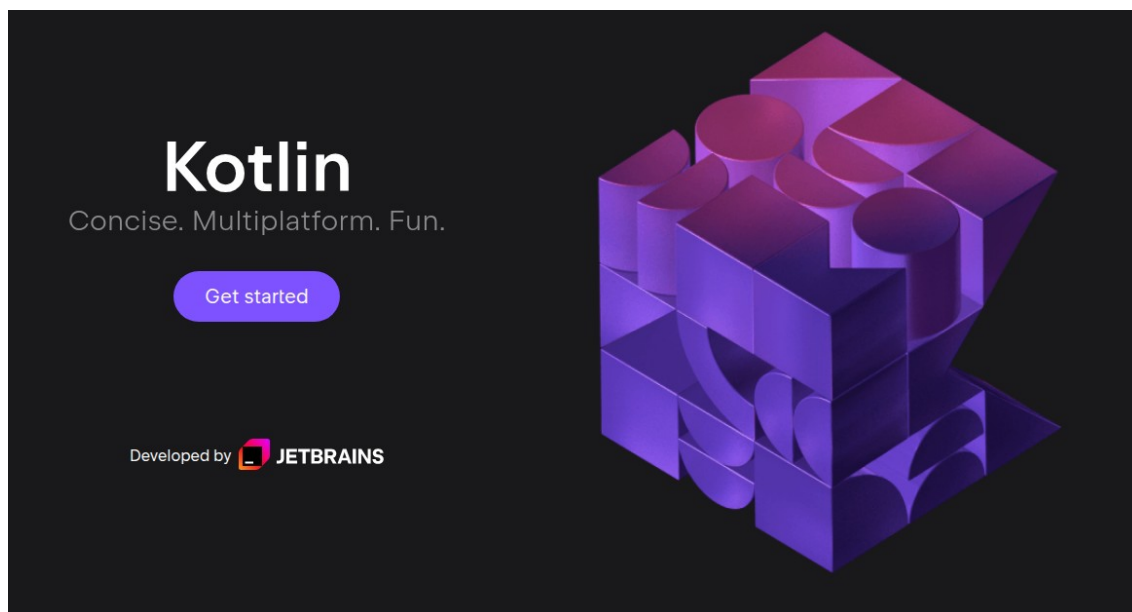
Rysunek 4.14 Mockup ekranu profilu

5. Wykorzystanie technologie

Po wnikliwej analizie wymagań funkcjonalnych i niefunkcjonalnych wybrano szereg powszechnie dostępnych technologii zapewniających (według wiedzy autorów) prawidłowe działanie zarówno części frontendowej jak i backendowej. Ponadto wybrano także bazę danych. Wszystkie wykorzystane technologie zostały opisane w następnych rozdziałach. Stacja pogodowa oparta o mikrokontroler z rodziny ESP32 została oprogramowana w darmowym środowisku programistycznym Arduino IDE w języku C++. Aplikacja mobilna stworzona została w Android Studio za pomocą języka Kotlin. Przetwarzanie i analizę danych zapewnia serwer stworzony w lekkim frameworku webowym dla środowiska uruchomieniowego Node.js – Express. Wszystkie części systemu komunikują się z chmurową bazą danych Firebase za pomocą API zapewnianego przed odpowiednie dla każdego języka biblioteki.

5.1 Języki programowania i frameworki

5.1.1 Kotlin



Rysunek 5.1 Strona tytułowa projektu Kotlin [24]

Kotlin to język programowania używający maszynę wirtualną Javy do działania. Jest on rozwijany na licencji Apache 2.0, czyli w systemie open-source [25].

Od 7 maja 2019r. Jest to preferowany język programowania aplikacji na platformę Android, działając jako następca Javy. Jest to w dużej części związane z tym, że w kodzie Kotlin, można wymiennie używać kodu Java.

Kotlin jest bardzo nowoczesnym językiem, posiadającym wiele funkcjonalności pożądaných przez programistów. Do najważniejszych należą [24]:

- Prostota. Kotlin posiada wiele skrótów obecnych w nowoczesnych językach programowania. Są to między innymi zmienne typu `var`, zakresy liczb w formie `0..10`, iteracja bezpośrednio po elementach kolekcji i wiele innych.



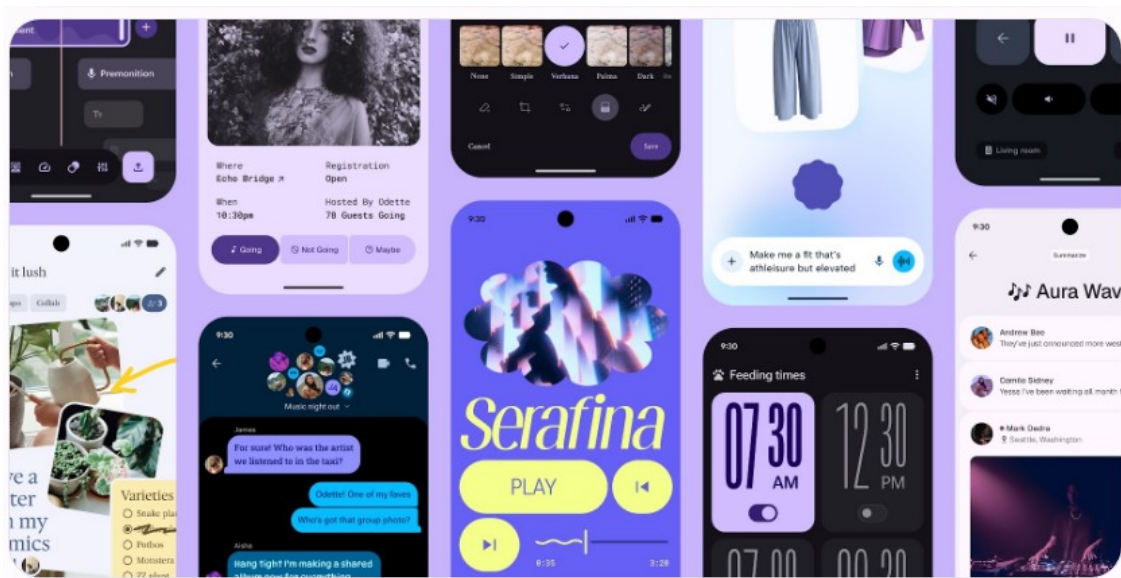
```
fun main() {  
    val name = "stranger"  
    println("Hi, $name!")  
    print("Current count:")  
    for (i in 0..10) {  
        print(" $i")  
    }  
}
```

Rysunek 5.2 Przykład kodu w języku Kotlin [24]

- Wsparcie operacji asynchronicznych. Język ten jest szeroko uznawany za jeden z najlepszych do wykonywania operacji asynchronicznych. Posiada bardzo rozbudowany system wątków, sterowany bardzo prostą składnią.
- Testowalność. Podobnie jak język Java, Kotlin kładzie duży nacisk na prostotę pisania testów do kodu.
- Dużo zasobów społeczności. Jako projekt open-source, Kotlin ma rozległą dokumentację i bardzo dużo zasobów do nauki programowania w tym języku.

5.1.2 Material Design

Material Design to system open-source od firmy Google, pozwalający na wytwarzanie estetycznych i ergonomicznych aplikacji mobilnych, ale także sieciowych i komputerowych. Wykorzystują go wszystkie aplikacje systemowe na nowych urządzeniach z Androidem. Firma Google stara się o ujednolicenie całego ekosystemu wytwarzania aplikacji na ich platformę mobilną [23].



Rysunek 5.3 Banner Material Design 3 [23]

Najnowszą podwersją Material Design jest M3 Expressive. Wprowadza on nowe reguły tworzenia UI, skupiające się na wywoływaniu konkretnych emocji w użytkowniku [23]. Jest to po części eksperymentalna wersja, przez co wymaga wyraźnych zgód na użycie jej w programie.

5.1.3 Jetpack

Android Jetpack to zbiór bibliotek skierowanych na pomoc programistom w plementowaniu aplikacji zgodnie z zaleceniami firmy Google. Wydawany jest w ramach przestrzeni nazw androidx i zastępuje oryginalne biblioteki z przestrzeni nazw android i biblioteki pomocy. Jetpack pomaga ograniczyć powielanie kodu, tworzyć bardziej czytelne programy oraz obejść stare rozwiązania, jednocześnie pozwalając na kompatybilność wsteczną. Częścią tego zbioru jest kilkadziesiąt bibliotek [22], skierowanych do szerokiej gamy użytkowników. Spora część z nich jest prawie niezbędna w każdej nowoczesnej aplikacji mobilnej.

activity *	Access composable APIs built on top of Activity.
appcompat *	Allows access to new APIs on older API versions of the platform (many using Material Design).
camera *	Build mobile camera apps.
compose *	Define your UI programmatically with composable functions that describe its shape and data dependencies.
databinding *	Bind UI components in your layouts to data sources in your app using a declarative format.
fragment *	Segment your app into multiple, independent screens that are hosted within an Activity.
hilt *	Extend the functionality of Dagger Hilt to enable dependency injection of certain classes from the androidx libraries.
lifecycle *	Build lifecycle-aware components that can adjust behavior based on the current lifecycle state of an activity or fragment.
Material Design Components *	Modular and customizable Material Design UI components for Android.
navigation *	Build and structure your in-app UI, handle deep links, and navigate between screens.
paging *	Load data in pages, and present it in a RecyclerView.
room *	Create, store, and manage persistent data backed by a SQLite database.
test *	Testing in Android.
work *	Schedule and execute deferrable, constraint-based background tasks.

Rysunek 5.4 Kilkanaście z najpopularniejszych bibliotek Jetpack [22]

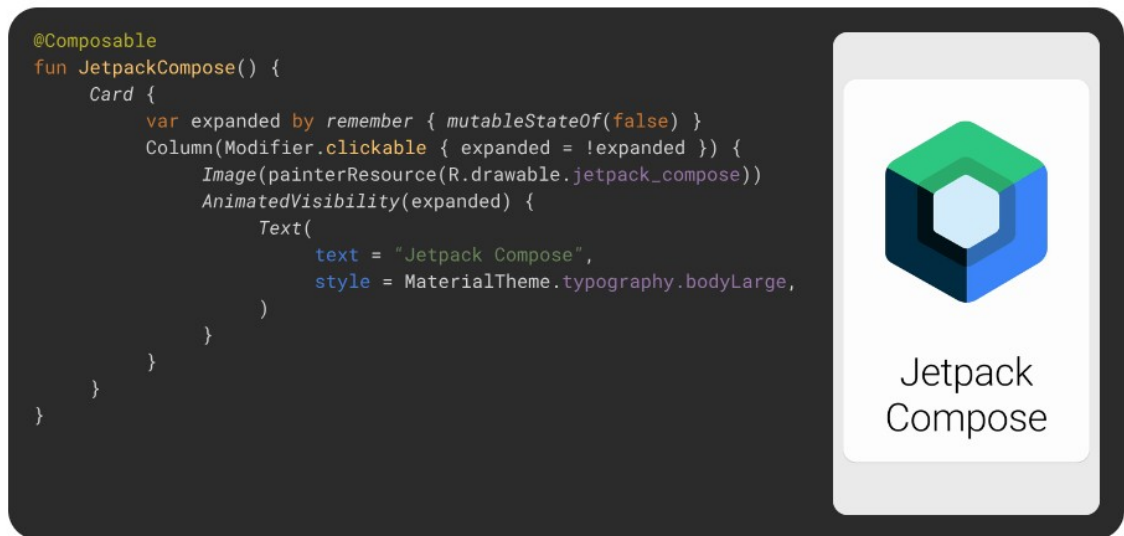
Poniżej znajdują się najważniejsze z bibliotek będących częścią Jetpack, które zostały zastosowane w projekcie.

Compose

Jetpack Compose to biblioteka redefiniująca tworzenie interfejsu użytkownika na Androidzie. Zamiast nieczytelnego kodu XML lub powolnego klikania atrybutów w edytorze graficznym, elementy interfejsu tworzone są w natywnym kodzie. Używa się do tego funkcji oznaczonych adnotacją `@Composable`. Jest to dużo szybsze, bardziej intuitywne i bardziej wszechstronne [21].

Dużą zaletą Compose jest też prostota zarządzania stanem. Umieszczenie go tak blisko warstwy interfejsu użytkownika powoduje znaczne przyspieszenie początkowych etapów tworzenia aplikacji. Do bardziej skomplikowanych zastosowań stosunkowo

łatwo zrefaktoryzować później kod, aby zachować odpowiednią separację warstw aplikacji. Biblioteka Compose pozwala na też na automatyczne zapamiętywanie stanu na potrzeby ponownego tworzenia aktywności. Jest to szczególnie przydatne na wypadek obrotu ekranu, lub chwilowego przełączenia aplikacji.



Rysunek 5.5 Przykładowy kod w Jetpack Compose [21]

Zalecaną przez Google techniką tworzenia aplikacji z biblioteką Compose jest tworzenie aplikacji z jedną aktywnością. Znaczaco upraszcza to niepotrzebne zarządzanie różnymi aktywnościami, fragmentami i przekazywaniem danych między nimi.

Navigation

Navigation 3 to stosunkowo nowa biblioteka wchodząca w skład pakietu Jetpack. Została specjalnie zaprojektowana do integracji z aplikacjami używającymi Compose. Znaczaco różni się od poprzednich wersji, przez co została wydzielona jako osobny artefakt. W nowej wersji, biblioteka zapewnia między innymi pełną kontrolę nad stosem nawigacji, znacznie uproszczony kod i większą wszechstronność. Istnieje jednak bardzo niewiele dokumentacji do tej wersji biblioteki [22].

Hilt

Hilt to biblioteka służąca do wstrzykiwania zależności na Androidzie [20]. Została stworzona na podstawie popularnej biblioteki Dagger. Jej główną zaletą w porównaniu z innymi rozwiązaniami jest zapewnianie specjalnie przygotowanych kontenerów dla każdej klasy Androida, i automatyczne zarządzanie ich cyklami życia.

Używanie biblioteki do wstrzykiwania zależności ułatwia programowanie na kilka sposobów. Są to między innymi ograniczenie liczby powtarzalnych fragmentów kodu, ograniczenie błędów w zarządzaniu cyklem życia elementów, czy ułatwienie refaktoryzacji i testowania.

5.1.4 Firebase

Firebase to rozwiązanie od firmy Google oferujące szeroki zakres gotowych funkcjonalności dla aplikacji webowych i mobilnych. Według portalu 42matters, zbierającego dane o aplikacjach mobilnych, Firebase jest obecny w znacznej części aplikacji na rynku [19]. Wśród aplikacji zbierających dane analityczne, 99,52% z nich używa Firebase [18].

W tym projekcie zastosowanie znalazły 3 główne usługi Firebase: Firestore, oferujący prostą nierelacyjną bazę danych, usługa Firebase Messaging, zapewniająca powiadomienia Push na smartfonach, oraz Authentication, pozwalająca na banalne uwierzytelnianie użytkowników.

5.1.5 Express

Express jest frameworkiem do rozwiązań serwerowych środowiska uruchomieniowego Node.js. Jest minimalistycznym, elastycznym rozwiązaniem dla małych serwerów napisanych w języku JavaScript. Według ankiety dla programistów przeprowadzonej na portalu Stack Overflow w 2025 roku, framework Express zajmuje 5 miejsce pod względem używanych frameworków webowych z popularnością na poziomie 20.8% [17]. Jego popularność opiera się na możliwości szybkiego tworzenia warstwy obsługi żądań HTTP, bogatej dokumentacji i minimalizmie.

```
import express from "express";

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.listen(PORT, () => console.log(`Serwer działa na porcie ${PORT}`));
```

Rysunek 5.6 Minimalny kod pozwalający na uruchomienie serwera Express

5.1.6 C++

C++ to język programowania należący do rodziny języków C, szeroko stosowany w branży systemów wbudowanych. Według serwisu JetBrains w 2021 roku jego udział w tworzeniu oprogramowania na urządzenia wbudowane wynosił 19% [16]. Zapewnia on większą warstwę abstrakcji w porównaniu do jego poprzednika C, w postaci programowania obiektowego, które jest kluczowe z perspektywy wykorzystanego w projekcie Arduino IDE i jego bibliotek. Upraszcza także zarządzanie pamięcią dzięki klasom takim jak `String`, `std::vector`, `std::unique_ptr`, zapewniając wyższe bezpieczeństwo w pracy z pamięcią dynamicznie przydzielaną.

```
class MyCallbacks : public NimBLECharacteristicCallbacks {
    void onWrite(NimBLECharacteristic* pCharacteristic, NimBLEConnInfo & connInfo){
        std::string value = pCharacteristic->getValue();
        Serial.print("Odebrano przez BLE: ");
        Serial.println(value.c_str());
        if (value == "ON") {
            Serial.println("LED ON");
            digitalWrite(10, HIGH);
        } else if (value == "OFF") {
            Serial.println("LED OFF");
            digitalWrite(10, LOW);
        }
    }
};
```

Rysunek 5.7 Fragment kodu w C++, wykorzystujący elementy programowania obiektowego

5.2 Narzędzia programistyczne

5.2.1 Android studio

Android Studio to oprogramowanie dla programistów tworzących aplikacje na platformę Android. Jest bardzo dobrze zintegrowane z całym jej ekosystemem. Podpowiedzi dla programisty są przystosowane do jej funkcji i klas. Środowisko Android studio posiada także wiele funkcji graficznych, umożliwiających bardziej przystępną interakcję z kodem. Przykładowo, program oferuje opcję podglądu zmian w graficznym interfejsie użytkownika bez rekompilacji [15].

Ogromnym atutem Android Studio jest emulator urządzeń mobilnych. Jest on zintegrowany z narzędziem LogCat, umożliwiając podglądanie logów systemu. Powoduje to duży wzrost produktywności w porównaniu do manualnego uruchamiania Android Debug Bridge (ADB).

5.2.2 Arduino IDE

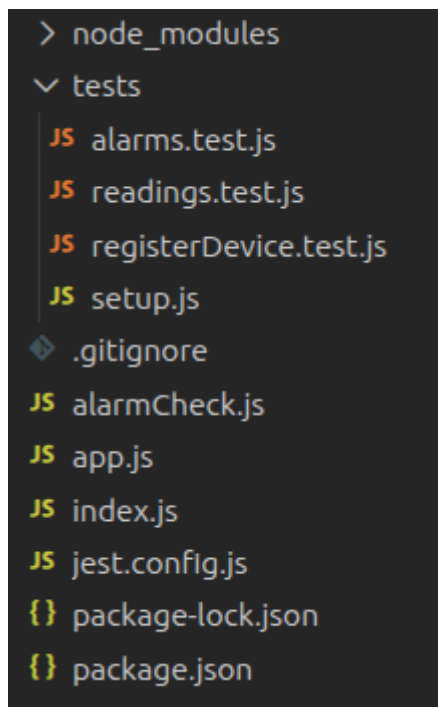
Arduino IDE jest zintegrowanym środowiskiem programistycznym, ułatwiającym tworzenie kodu na mikrokontrolery, takie jak płytki Arduino czy ESP32. Umożliwia on pisanie, kompilowanie oraz wgrywanie programu bezpośrednio na mikrokontroler za pomocą jednego narzędzia. Arduino IDE zapewnia kompilator GCC dla kodu napisanego w C/C++ i oferuje podstawowe funkcje wspomagające programowanie, takie jak kolorowanie składni, automatyczne formatowanie kodu czy menedżer bibliotek. Dzięki niemu można w prosty sposób dodać obsługę dodatkowych komponentów, np. czujników, modułów komunikacyjnych czy wyświetlaczy. Bardzo ważnym elementem tego środowiska jest monitor portu szeregowego (ang. Serial monitor) ułatwiającym debugowanie kodu poprzez wyświetlanie danych przetwarzanych przez mikrokontroler w czasie rzeczywistym [14].

6. Implementacja

Ten rozdział szczegółowo opisuje zastosowaną implementację całego systemu. Składa się z trzech podrozdziałów, każdy opisujący inną aplikację będącą częścią rozwiązania. Implementacja stacji prezentuje fragmenty kodu wykonującego pomiary i wysyłającego je na serwer. Część dotycząca serwera pokazuje ścieżki dostępne do wywołania na urządzeniu pomiarowym i konfigurację. Implementacja aplikacji mobilnej szczegółowo opisuje kod odpowiadający za logikę i interfejs aplikacji.

6.1 Aplikacja serwerowa

Część serwerowa składa się z prostej aplikacji opierającej się na Express.js. Strukturę tej aplikacji przedstawia rysunek 6.30. Funkcjonalność zawarta jest w plikach „alarmCheck.js”, gdzie znajdują się funkcje odpowiedzialne za sprawdzanie alarmów, oraz w „app.js”, który definiuje obiekt Express i endpoint-y.



Rysunek 6.1 Struktura plików aplikacji serwerowej

Komunikacja z serwerem jest realizowana poprzez protokół HTTPS. Aplikacja w ostatniej wersji składa się z trzech punktów dostępu (listing 6.1). Służą odpowiednio do zapisywania pomiarów przez stację, początkowej rejestracji stacji w systemie (ang. provisioning), oraz do odczytywania ostatniego zarejestrowanego pomiaru (w celach testowych).

```
app.post("/reading", async (req, res) => { ...
});

app.post("/registerDevice", async (req, res) => { ...
});

app.get('/reading', (req, res) => { ...
});
```

Listing 6.1 Punkty końcowe aplikacji serwerowej

Zapis pomiarów odbywa się w dwóch etapach. W pierwszym sprawdzana jest poprawność żądania (listing 6.2). Najpierw sprawdzana jest obecność wymaganych wartości. Następnie sprawdzany jest dokument stacji w bazie danych. Następnie sprawdzana jest poprawność hasła. W tym systemie, hasła hashowane są tylko po stronie serwera. Dodatkowe zabezpieczenia nie wpływają w tym przypadku w żaden znaczący sposób na bezpieczeństwo systemu. Faktyczne hasło to generowany fabrycznie losowy ciąg znaków.

```
app.post("/reading", async (req, res) => {
  try {
    const {
      station_id,
      password,
      humidityAir,
      humiditySoil,
      temperatureAir,
      pressureAir,
      sunlight
    } = req.body;

    if (!station_id || !password) {
      return res.status(400).json({ error: "Missing deviceId or password" });
    }

    const deviceDoc = await db.collection("stations").doc(station_id).get();
    if (!deviceDoc.exists) {
      return res.status(404).json({ error: "Device not found" });
    }

    const deviceData = deviceDoc.data();
    const passwordOK = await bcrypt.compare(password, deviceData.password_hash);

    if (!passwordOK) {
      return res.status(401).json({ error: "Invalid password" });
    }
  }
});
```

Listing 6.2 Pierwsza część punktu końcowego zapisywania pomiaru

Drugą częścią tego punktu dostępu jest zapis pomiaru do bazy danych. Aby był on jednak możliwy, konieczna jest inicjalizacja funkcjonalności Firebase, widoczna na listingu 6.3.

```
const serviceAccount = JSON.parse(process.env.FIREBASE_SERVICE_ACCOUNT);

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

const db = admin.firestore();
const messaging = admin.messaging();
```

Listing 6.3 Inicjalizacja Firebase

Sam proces zapisu prezentuje listing 6.4. Dzięki bazie danych Firestore, proces ten jest bardzo prosty.

```
let reading = {
  station_id,
  humidityAir,
  humiditySoil,
  temperatureAir,
  pressureAir,
  sunlight,
  date: admin.firestore.FieldValue.serverTimestamp()
};

reading = removeUndefined(reading);
lastReading = reading;

let documentRef;
let measurementId;

try {
  documentRef = db.collection("measurements").doc();
  measurementId = documentRef.id;
} catch (err) {
  console.error("Błąd przy kontakcie z bazą danych: " + err);
  return;
}

try {
  await documentRef.set(reading);
  console.log("Zapisano odczyt:", reading);
} catch (err) {
  console.error("Błąd przy zapisie pomiaru do bazy: " + err);
}

try {
  await handleMeasurement(db, messaging, admin, measurementId, station_id, reading);
} catch (err) {
  console.log("Błąd przy obsłudze alarmów: " + err);
}

return res.status(200).json({ ok: true, message: "Reading saved" });
```

Listing 6.4 Zapis pomiaru do bazy danych

Drugim ważnym endpoint-em jest rejestracja stacji. Jest on wywoływany przy pierwszym połączeniu stacji z aplikacją użytkownika. Głównym zadaniem kodu jest utworzenie dokumentu w bazie danych, zawierającego wybraną nazwę stacji, id użytkownika i hasło. Ten proces jest zaprezentowany na listingu 6.5.

```
try {
  console.log("Register device requested");
  const { station_id, password, name, ownerId } = req.body;

  if (!station_id || !password || !name || !ownerId) {
    console.log("Malformed request");
    return res.status(400).json({ error: "Malformed body" });
  }

  const docRef = db.collection("stations").doc(station_id);
  const doc = await docRef.get();

  if (doc.exists) {
    console.log("Device already exists");
    return res.status(400).json({ error: "Device already exists" });
  }

  const hash = await bcrypt.hash(password, 12);

  await docRef.set({
    name: name,
    owner_id: ownerId,
    password_hash: hash,
    created_at: admin.firestore.FieldValue.serverTimestamp()
  });
  console.log("Created a device: " + doc);
  res.json({ ok: true, message: "Device registered successfully" });
} catch (err) {
  console.error("Error registering device:", err);
  res.status(500).json({ error: "Server error" });
}
```

Listing 6.5 Rejestracja stacji

Najważniejszą funkcją serwera jest „handleMeasurement” (listing 6.6). Jest ona wywoływana dla każdego pomiaru który trafia na stację. Korzysta ona z kilku innych funkcji aby sprawdzić czy wystąpił jakiś alarm, utworzyć ewentualne wystąpienie w bazie danych oraz wysłać powiadomienie na telefon właściciela stacji.

```
async function handleMeasurement(db, messaging, admin, measurementId, stationId, reading) {
  const alarms = await checkAlarms(
    db,
    stationId,
    reading.temperatureAir,
    reading.humidityAir,
    reading.pressureAir,
    reading.sunlight,
    reading.humiditySoil
  );

  console.log("Triggered alarms:", alarms);

  await createAlarmOccurrences(db, admin, alarms, measurementId, stationId);
  await sendTriggeredAlarmNotifications(db, messaging, alarms, measurementId, stationId);
}
```

Listing 6.6 Główna funkcja pliku "alarmCheck.js"

Najważniejszą częścią funkcji „checkAlarms” jest pętla iterująca po każdym warunku alarmu (listing 6.7). Funkcja zwraca wszystkie uruchomione alarmy w postaci listy

```
conditionsSnapshot.forEach((condDoc) => {
  const cond = condDoc.data();

  const param = cond.parameter;
  const triggerLevel = cond.trigger_level;
  const triggerOnHigher = cond.trigger_on_higher;

  if (!(param in values)) return;

  const currentValue = values[param];

  if (triggerOnHigher && currentValue > triggerLevel) {
    triggeredConditions.push(condDoc.id);
  } else if (!triggerOnHigher && currentValue < triggerLevel) {
    triggeredConditions.push(condDoc.id);
  }
});

if (triggeredConditions.length > 0) {
  triggeredAlarms.push({
    alarmId: alarmId,
    conditionId: triggeredConditions[0]
  });
}
```

Listing 6.7 Pętla sprawdzająca warunki alarmu

Kolejnym krokiem jest utworzenie dokumentu w bazie danych dla każdego aktywowanego alarmu. Główną część tej funkcjonalności przedstawia Listing 6.8.

```
triggeredAlarms.forEach(({ alarmId, conditionId }) => {
  const docRef = db.collection("alarm_occurrences").doc();
  batch.set(docRef, {
    alarm_id: alarmId,
    condition_id: conditionId,
    measurement_id: measurementId,
    station_id: stationId,
    user_id: userId,
    date: admin.firestore.FieldValue.serverTimestamp()
  });
});
```

Listing 6.8 Utworzenie dokumentów wystąpień alarmu w bazie danych

Wysyłanie powiadomień na telefon użytkownika dzieje się poprzez usługę Firebase Cloud Messaging. Polega to na utworzeniu i wysłaniu specjalnego obiektu JSON, zawierającego odpowiednie dane. Najważniejszy fragment funkcji wysyłającej powiadomienia jest widoczny na Listingu 6.9.

```
const body = trigger_on_higher
  ? `${capitalized} exceeded ${formattedLevel}${unit}`
  : `${capitalized} dropped below ${formattedLevel}${unit}`;

const title = `Alarm Triggered: ${alarmName}`;

await messaging.send({
  token,
  notification: { title, body },
  data: {
    alarmId,
    conditionId,
    measurementId,
    stationId
  },
  android: {
    notification: { channel_id: "alarms" }
  }
});
```

Listing 6.9 Wysyłanie powiadomienia na urządzenie użytkownika

Widoczne na Listingu 6.9 pole „token” jest kluczowe do wysłania powiadomienia na odpowiednie urządzenie. Pobierany on jest z bazy danych (listing 6.10). Jego aktualizacją zajmuje się aplikacja mobilna.

```
const tokenDoc = await db.collection("fcmTokens").doc(userId).get();
if (!tokenDoc.exists) {
  console.error("FCM token not found for user:", userId);
  return;
}
const token = tokenDoc.data().token;
```

Listing 6.10 Pobieranie tokena z bazy danych

6.2 Stacja pomiarowa

Oprogramowanie obsługujące stację pogodową zostało stworzone w zintegrowanym środowisku programistycznym Arduino IDE za pomocą języka C++. Do ułatwienia komunikacji mikrokontrolera z czujnikami wykorzystano biblioteki Adafruit [13], zapewniającą warstwę abstrakcji pomijającą bezpośrednie działanie na rejestrach. Użyto także biblioteki NimBLE-Arduino [12] z tych samych powodów – ułatwienie oprogramowania komunikacji przez protokół Bluetooth Low Energy [11].

6.2.1 Kod aplikacji

Zgodnie z dobrą praktyką tworzenia oprogramowania na urządzenia wbudowane, pętla programu jest pusta, a główny plik służy jedynie do wysokopoziomowej konfiguracji pinów, inicjalizacji czujników, i stworzenia zadań.

Kod głównego pliku .ino - konfiguracja pinów w tryb wyjścia/wejścia, inicjalizacja czujników, połączenie z siecią wifi, stworzenie zadań kolejno: pomiarowego, wysyłającego dane i odpowiadającego za obsługę przycisku uruchamiającego moduł Bluetooth.

```
void setup() {
    Serial.begin(115200);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(LED_PIN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    initSensors();

    if (loadWiFiCredentials()) connectWiFi();
    xTaskCreate(
        measurementTask,
        "MeasurementTask",
        4096,
        nullptr,
        1,
        &measurementTaskHandle);

    xTaskCreate(
        senderTask,
        "SenderTask",
        4096,
        nullptr,
        1,
        nullptr
    );

    xTaskCreate(
        buttonBleTask,
        "ButtonBLE",
        2048,
        nullptr,
        1,
        &buttonTaskHandle
    );
}

void loop() {
}
```

Listing 6.11: kod w pliku głównym

Wszystkie dane konfiguracyjne zostały umieszczone w pliku nagłówkowym Config.h.

Pierwsza sekcja - PIN określa numer pinu odpowiadającego za połączenie z czujnikiem.

Druga sekcja - TIMER odpowiada za odstęp czasowy między uruchomieniem zadania pomiarowego (MEASUREMENT_DELAY_MS) oraz między dokonywanymi pomiarami (SAMPLE_DELAY_MS).

Trzecia sekcja - CALIBRATION określa przedział wartości odczytywanego napięcia na pinie czujnika wilgotności gleby. Czwarta sekcja - BLE UUID określa parametry charakterystyki Bluetooth.

Ostatnia sekcja - CREDENTIALS, ustala parametry stacji niezbędne w komunikacji sieciowej, a zmienna SERVER_URL wskazuje na adres na który będą przesyłane żądania REST z pomiarami.

```
#pragma once

/* ===== PIN ===== */
#define LED_PIN 10
#define BUZZER_PIN 2
#define BUTTON_PIN 5
#define SOIL_PIN 0
#define PHOTORESISTOR_PIN 1
#define DHTPIN 4
#define DHTTYPE DHT11
#define I2C_SDA 8
#define I2C_SCL 9

/* ===== TIMER ===== */
#define MEASUREMENT_DELAY_MS 300000 // 5 minutes
#define SAMPLE_DELAY_MS 1000

/* ===== CALIBRATION ===== */
#define DRY_VALUE 3000
#define WET_VALUE 1000

/* ===== BLE UUID ===== */
#define SERVICE_UUID "12345678-1234-1234-1234-1234567890ab"
#define CHARACTERISTIC_UUID "87654321-4321-4321-4321-ba0987654321"

/* ===== CREDENTIALS ===== */
static const char* STATION_ID = "station03";
static const char* STATION_PASSWORD = "Mocnathuja123";
static const char* SERVER_URL = "https://embedded-server-m7j9.onrender.com/reading";
```

Listing 6.12: Kod pliku konfiguracyjnego.

Operacje powiązane z czujnikami zostały wydzielone do pliku Sensors.cpp. Kod programu można podzielić na trzy sekcje:

Pierwsze trzy linijki odpowiadają za inicjalizację obiektów czujników za pomocą konstruktorów z biblioteki Adafruit.

Funkcja `initSensors()` odpowiada za uruchomienie czujników - uruchomienie magistrali I2C, ustalenie adresu czujnika ciśnienia na 0x76 oraz uruchomienie pozostałych.

Funkcje `read` odpowiadają za bezpośredni odczyt wartości przesyłanych przez czujnik. Na szczególną uwagę zasługuje odczyt wilgotności gleby – `readSoilPercent`. Odczytane napięcie jest skalowany na zakres określony w zmiennych w pliku `Config.h` tak, aby określić przedział sucho-mokro.

```
/* ===== SENSORS OBJECTS ===== */
Adafruit_BMP280 bmp;
Adafruit_TSL2591 tsl(2591);
DHT dht(DHTPIN, DHTTYPE);

void initSensors() {
    Wire.begin(I2C_SDA, I2C_SCL);

    bmp.begin(0x76);
    tsl.begin();
    dht.begin();
}

float readTemperature() {
    return bmp.readTemperature();
}

float readHumidity() {
    return dht.readHumidity();
}

float readPressure() {
    return bmp.readPressure() / 100.0;
}

float readLux() {
    sensors_event_t ev;
    tsl.getEvent(&ev);
    if (!isfinite(ev.light)) return NAN;
    return ev.light;
}

int readSoilPercent() {
    int raw = analogRead(SOIL_PIN);
    int pct = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
    return constrain(pct, 0, 100);
}
```

Listing 6.13: Funkcje realizujące konfigurację czujników i odczyty pomiarowe.

Konfiguracja połączenia sieciowego przez WIFI została wydzielona do pliku WifiManager.cpp. Funkcja connectWifi() odpowiada za połączenie z siecią WIFI. Początkowo sprawdzane jest, czy została zapisana jakakolwiek nazwa sieci. Jeżeli tak, następuje próba połączenia z siecią która trwa 15 sekund.

```
bool connectWifi() {
    if (wifiSSID.isEmpty()) {
        Serial.println("⚠ Brak SSID do połączenia");
        return false;
    }

    Serial.println("🔄 Próba połączenia z WiFi:");
    Serial.println("SSID: " + wifiSSID);

    WiFi.disconnect(true);
    delay(500);
    WiFi.begin(wifiSSID.c_str(), wifiPASS.c_str());

    unsigned long start = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
        delay(500);
        Serial.print(".");
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\n✅ Połączono z WiFi!");
        Serial.print("Adres IP: ");
        Serial.println(WiFi.localIP());
        saveWiFiCredentials();
        stopBLE();
        return true;
    } else {
        Serial.println("\n❌ Nie udało się połączyć z WiFi");
        return false;
    }
}
```

Listing 6.14: Kod odpowiadający za działania związane z ustanawianiem połączenia sieciowego

Jeżeli połączenie się uda, identyfikator i hasło sieci są zapisywane do pamięci nieulotnej mikrokontrolera i zatrzymywane jest rozgłaszanie serwera Bluetooth.

```
void saveWiFiCredentials() {
    prefs.begin("wifi", false);
    prefs.putString("ssid", wifiSSID);
    prefs.putString("pass", wifiPASS);
    prefs.end();
    Serial.println("💾 Zapisano WiFi w NVS");
}
```

Listing 6.15: Zapisywanie danych sieci w pamięci NVS

Przy uruchomieniu stacji te dane są odczytywane (listing 6.16)

```
bool loadWiFiCredentials() {
    prefs.begin("wifi", true);
    wifiSSID = prefs.getString("ssid", "");
    wifiPASS = prefs.getString("pass", "");
    prefs.end();

    if (wifiSSID.length() > 0) {
        Serial.println("📁 Wczytano dane WiFi z NVS:");
        Serial.println("SSID: " + wifiSSID);
        return true;
    } else {
        Serial.println("📁 Brak zapisanych danych WiFi w NVS");
        return false;
    }
}
```

Listing 6.16: Odczyt danych sieci z pamięci nieulotnej - przy każdym uruchomieniu stacji pomiarowej sprawdzane jest ich istnienie.

Każde z zadań uruchomionych podczas działania programu jest pojedynczą funkcją. Najważniejszą z nich jest zadanie pomiarowe. Ilość próbek z których zostanie obliczona średnia wynosi 10. Po zsumowaniu pomiarów, następuje obliczenie średniej i stworzenie obiektu Measurement reprezentującego pomiar, który naznaczany jest czasem w którym został dokonany i zostaje zapisany do bufora. Jeżeli istnieje połączenie sieciowe, następuje wysłanie pomiarów istniejących w buforze a następnie dokonywane jest jego wyczyszczenie.

```
void measurementTask(void* parameter) {
    const int SAMPLES = 10;

    while (true) {
        float tempSum = 0;
        float humSum = 0;
        float presSum = 0;
        float luxSum = 0;
        int soilSum = 0;

        for (int i = 0; i < SAMPLES; i++) {
            tempSum += readTemperature();
            humSum += readHumidity();
            presSum += readPressure();
            luxSum += readLux();
            soilSum += readSoilPercent();

            Serial.printf("\ Sample %d/10\n", i + 1);

            vTaskDelay(SAMPLE_DELAY_MS / portTICK_PERIOD_MS);
        }

        Measurement m;
        m.temperature = tempSum / SAMPLES;
        m.humidity = humSum / SAMPLES;
        m.pressure = presSum / SAMPLES;
        m.lux = luxSum / SAMPLES;
        m.soil = soilSum / SAMPLES;
        m.timestamp = millis();

        buffer.add(m);

        Serial.println("🟡 Średni pomiar dodany do bufora");
        if (WiFi.status() == WL_CONNECTED) {
            Measurement out;
            while (buffer.pop(out)) {
                if (!sendMeasurement(out)) {
                    buffer.add(out);
                    break;
                }
            }
        }
        vTaskDelay(MEASUREMENT_DELAY_MS / portTICK_PERIOD_MS);
    }
}
```

Listing 6.17: Kod zadania pomiarowego

Bufor w którym przechowywane są pomiary, został zaimplementowany w postaci bufora cyklicznego zabezpieczonego muteksem. Mieści 30 pomiarów, co przy częstotliwości pomiarowej wynoszącej 5 minut daje dwie i pół godziny pracy bez połączenia sieciowego.

```
bool Buffer::add(const Measurement& m) {
    if (xSemaphoreTake(mutex, portMAX_DELAY) != pdTRUE) {
        return false;
    }

    if (isFull()) {
        tail = (tail + 1) % BUFFER_SIZE;
        used--;
    }

    data[head] = m;
    head = (head + 1) % BUFFER_SIZE;
    used++;

    xSemaphoreGive(mutex);
    return true;
}
```

Listing 6.18: Dodawanie danych do bufora. Jeżeli muteks jest zajęty, dodawanie danej jest przerywane. Jeżeli bufor jest pełny, przesuwa wskaźniki a następnie nadpisuje najstarszy element i zwalnia muteks.

```
bool Buffer::pop(Measurement& m) {
    if (xSemaphoreTake(mutex, portMAX_DELAY) != pdTRUE) {
        return false;
    }

    if (isEmpty()) {
        xSemaphoreGive(mutex);
        return false;
    }

    m = data[tail];
    tail = (tail + 1) % BUFFER_SIZE;
    used--;

    xSemaphoreGive(mutex);
    return true;
}
```

Listing 6.19: Odczytanie danych z bufora. Jeżeli muteks jest zajęty, pobieranie danej jest przerywane. Jeżeli bufor jest pusty, muteks jest zwalniany. Pobranie danej jest realizowane przez kopiowanie zmiennej i przesunięcie ogona bufora.

Zadanie wysyłania danych jest uruchamiane cyklicznie co 5 minut. Jeżeli istnieje połączenie sieciowe, następuje odczyt danych z bufora i próba wysłania danych na serwer. Jeżeli się nie powiedzie, odczyt jest dodawany ponownie do bufora i operacja jest przerywana.

```
void senderTask(void* param) {
    while (true) {
        if (WiFi.status() == WL_CONNECTED) {
            Measurement out;
            while (buffer.pop(out)) {
                if (!sendMeasurement(out)) {
                    buffer.add(out);
                    break;
                }
            }
        }
        vTaskDelay(MEASUREMENT_DELAY_MS / portTICK_PERIOD_MS);
    }
}
```

Listing 6.20: Zadanie wysyłu danych.

Cała logika odpowiedzialna za transmisję Bluetooth została umieszczona w pliku BLEManager.cpp. Jeżeli dane rozpoczynają się od ciągu "WIFI:" następuje podział odebranego literału łańcuchowego na dwie części, gdyż pierwsza z nich zawiera identyfikator sieci, a druga hasło do niej.

```
enum BleState {
    BLE_OFF,           // Bluetooth off
    BLE_ADVERTISING,   // Broadcasting
    BLE_CONNECTED,     // Paired
    BLE_WIFI_CONFIG,   // Waiting for wifi credentials
    BLE_DONE           // Configuration done
};
```

Listing 6.21: Stany w jakich może się znaleźć moduł Bluetooth

Aby stacja była widoczna dla urządzenia mobilnego, musi ona rozgłaszać swój serwer BLE (listing 6.22). Podczas rozpoczęcia rozgłaszania Bluetooth tworzony jest identyfikator urządzenia o nazwie ESP32-Weather-Alarm, serwer obsługujący żądania, serwis ustalający charakterystykę połączenia i rozpoczynający rozgłaszanie. Na koniec zapalana jest niebieska dioda LED.

```
void startBLE() {
    NimBLEDevice::init("ESP32-Weather-Alarm");
    pServer = NimBLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

    NimBLEService* s = pServer->createService(SERVICE_UUID);
    pCharacteristic = s->createCharacteristic(
        CHARACTERISTIC_UUID,
        NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::WRITE
    );

    pCharacteristic->setCallbacks(new MyCallbacks());
    s->start();

    NimBLEDevice::getAdvertising()->addServiceUUID(SERVICE_UUID);
    NimBLEDevice::getAdvertising()->start();
    Serial.println("✅ BLE gotowe do parowania");
    neopixelWrite(LED_PIN, 0, 0, 100);
}
```

Listing 6.22: Rozpoczęcie rozgłaszania Bluetooth.

Po nawiązaniu połączenia obsługiwana jest konfiguracja Wi-Fi (listing 6.24)

```
class MyCallbacks : public NimBLECharacteristicCallbacks {
    void onWrite(NimBLECharacteristic* c, NimBLEConnInfo&) override {
        String v = c->getValue().c_str();
        Serial.println("📡 Otrzymano przez BLE: " + v);

        if (v.startsWith("WIFI:")) {
            bleState = BLE_WIFI_CONFIG;
            int sep = v.indexOf(';');
            if (sep > 5) {
                wifiSSID = v.substring(5, sep);
                wifiPASS = v.substring(sep + 1);
                Serial.println("📶 Konfiguracja WiFi przez BLE:");
                Serial.println("SSID: " + wifiSSID);
                Serial.println("PASS: " + wifiPASS);

                if(connectWiFi()) {
                    Serial.println("✅ WiFi ustawione i połączone przez BLE");
                } else {
                    Serial.println("❌ Nie udało się połączyć z WiFi przez BLE");
                }
            }
        }
        return;
    }
}
```

Listing 6.24: Odczyt danych przez Bluetooth.

Celem zmiany stanu logicznego i zapalenia zielonej dioda LED tworzony jest callback uruchamiany przy ustanowieniu sesji Bluetooth (listing 6.23)

```
void onConnect(NimBLEServer*, NimBLEConnInfo&) override {
    bleState = BLE_CONNECTED;
    neopixelWrite(LED_PIN, 100, 0, 0);
    Serial.println("🟢 BLE CONNECTED");
}
```

Listing 6.23: Obsługa ustanowienia sesji Bluetooth.

Przesłanie danych na serwer jest realizowane w pliku WifiSender.cpp i jest obsługiwane przez pojedynczą funkcję. Jeżeli nie istnieje połączenie sieciowe, operacja jest przerywana. W przeciwnym wypadku, konstruowany jest literał łańcuchowy reprezentujący dane pomiarowe w formacie JSON.

```
bool sendMeasurement(const Measurement& m) {
    if (WiFi.status() != WL_CONNECTED) return false;

    WiFiClientSecure client;

    http.begin(client, SERVER_URL);
    http.addHeader("Content-Type", "application/json");

    String json = "{";
    bool first = true;

    if (!isnan(m.temperature)) {
        json += "\"temperatureAir\":" + String(m.temperature, 2);
        first = false;
    }

    if (!isnan(m.humidity)) {
        if (!first) json += ",";
        json += "\"humidityAir\":" + String(m.humidity, 2);
        first = false;
    }

    if (!isnan(m.pressure)) {
        if (!first) json += ",";
        json += "\"pressureAir\":" + String(m.pressure, 2);
        first = false;
    }

    if (!isnan(m.lux)) {
        if (!first) json += ",";
        json += "\"lightLux\":" + String(m.lux, 2);
        first = false;
    }

    if (!first) json += ",";
    json += "\"humiditySoil\":" + String(m.soil);

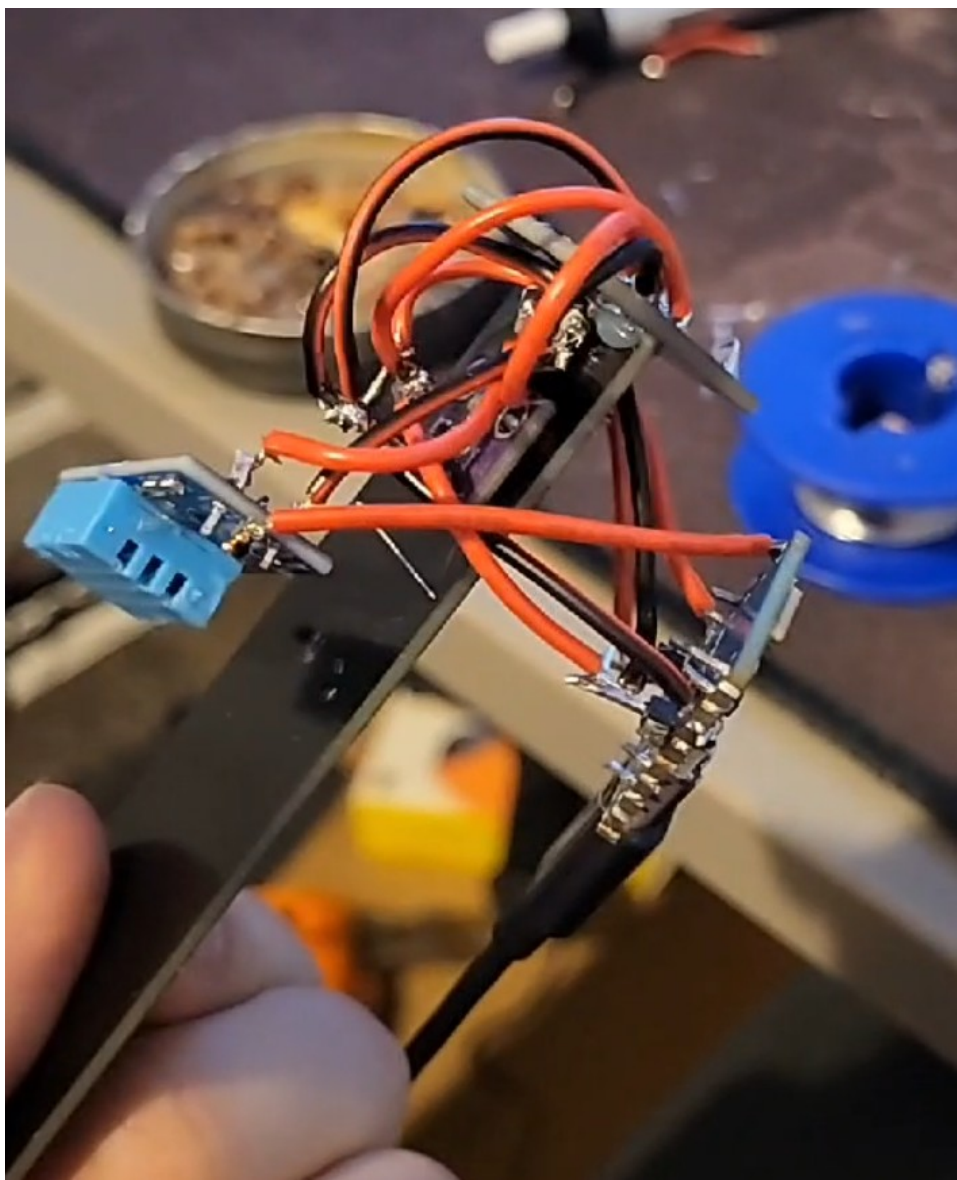
    json += ", \"station_id\": \"" + String(STATION_ID) + "\"";
    json += ", \"password\": \"" + String(STATION_PASSWORD) + "\"";
    json += "}";

    int code = http.POST(json);
    http.end();
    Serial.println("Sending: " + json);
    Serial.println(" HTTP code: " + String(code));
    return (code > 0 && code < 300);
}
```

Listing 6.25: Przesyłanie danych na serwer.

6.2.2 Elektronika stacji

Poszczególne czujniki są połączone zlutowanymi standardowymi kablami miedzianymi (rys. 6.2). Mikrokontroler ESP32 jest wpięty bezpośrednio do kabla zasilającego USB typu C. Do niego podłączone są czujniki. W urządzeniu zawarta jest też dioda sygnalizująca status połączenia Bluetooth, oraz przycisk odpowiadający za parowanie.

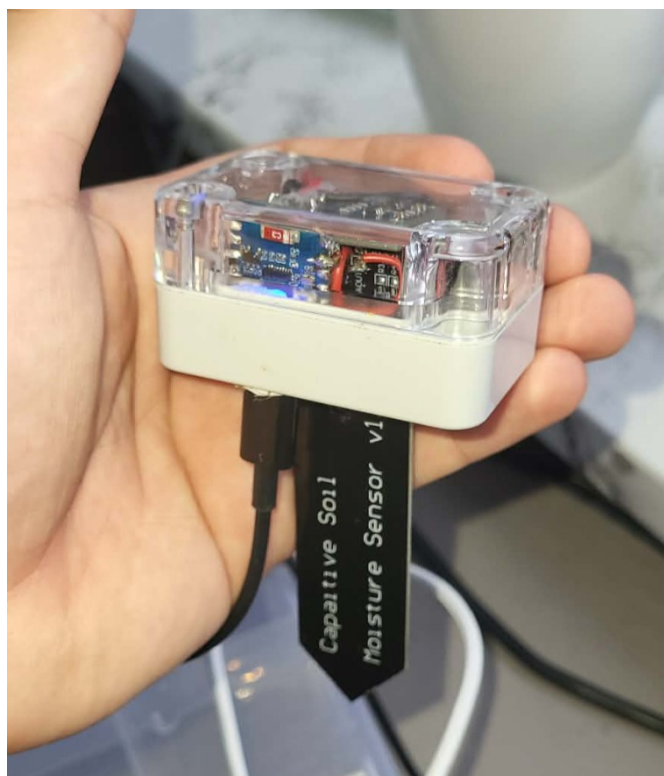


Rysunek 6.2 Wnętrze stacji

Komponenty elektroniczne wchodzące w skład urządzenia zostały umieszczone w kompaktowej obudowie z przezroczystą pokrywą, aby umożliwić działanie czujnika światła. Kolor obudowy został dobrany tak, aby nie przyciągała ona promieni słonecznych, co skutkowałoby przekłamaniem pomiarów temperatury.



Rysunek 6.4 Stacja pomiarowa z góry



Rysunek 6.3 Stacja pomiarowa z boku

Z dolnej strony obudowy zostały wyprowadzone części pomiarowe czujników. Podłużna czarna część czujnika wilgotności gleby zaprojektowana jest, aby wetknąć ją w glebę. Widoczny na rysunku 6.5 jest też przycisk odpowiadający za Bluetooth.



Rysunek 6.5 Dolna część stacji pomiarowej

Na rysunku 6.6 widoczna jest stacja podczas przykładowej pracy.



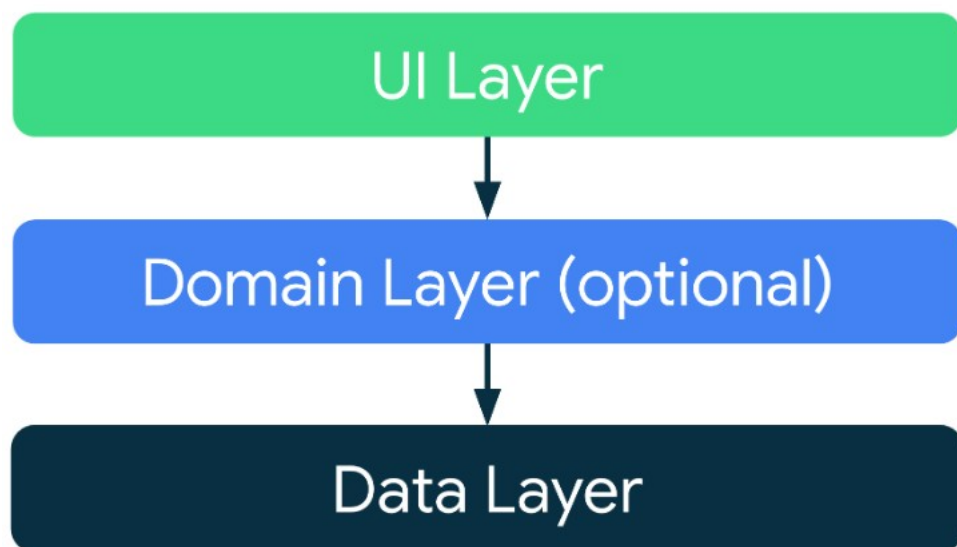
Rysunek 6.6 Stacja pomiarowa podczas pracy

6.3 Aplikacja mobilna

Aplikacja mobilna została zaprogramowana w środowisku Android Studio, wykorzystując język Kotlin. Do stworzenia interfejsu użytkownika posłużył Jetpack Compose. Aplikacja wykorzystuje też bibliotekę wstrzykiwania zależności Hilt [20]. Ważną częścią projektu było zastosowanie warstwowej architektury aplikacji, zgodnie z zaleceniami firmy Google [10]. Wszystkie zawarte w tym rozdziale zrzuty ekranu prezentujące interfejs aplikacji mobilnej wykonano na urządzeniu z Androidem w wersji 16.0 i wymiarach 2400x1080px.

6.3.1 Architektura

Aplikacja została podzielona na 3 warstwy. Pakiety projektu odnoszą się do nich jako UI, Domain i Data zgodnie z nazewnictwem firmy Google [10]. Można się też jednak spotkać z takim schematem jak Presentation, Domain, Data [2]. Taki podział jest bardzo naturalny dla aplikacji Android. Niesie on ze sobą wiele korzyści, między innymi ułatwia skalowalność, utrzymanie, testowanie czy debugowanie. Czyni on też zarówno strukturę jak i kod projektu dużo bardziej czytelnym.

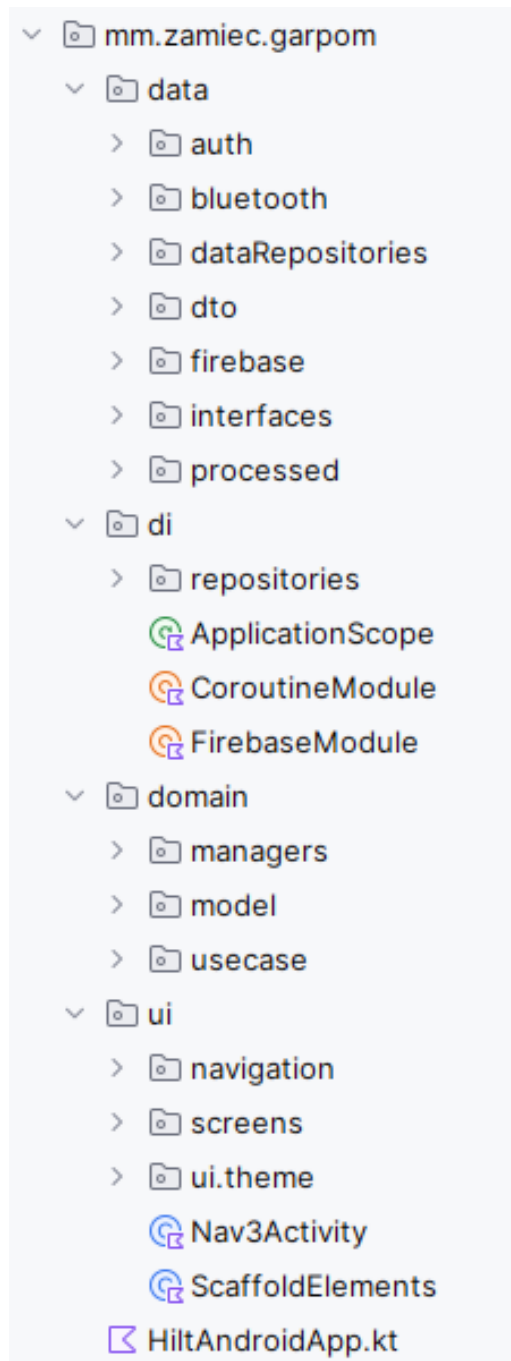


Rysunek 6.7 Rekomendowany podział na warstwy aplikacji Android [10]

W warstwie UI znajdują się komponenty Compose i ViewModel-e, warstwa domeny odpowiada za zastosowanie logiki biznesowej w Menager-ach i Use Case-ach, a warstwa danych zapewnia dane przez repozytoria.

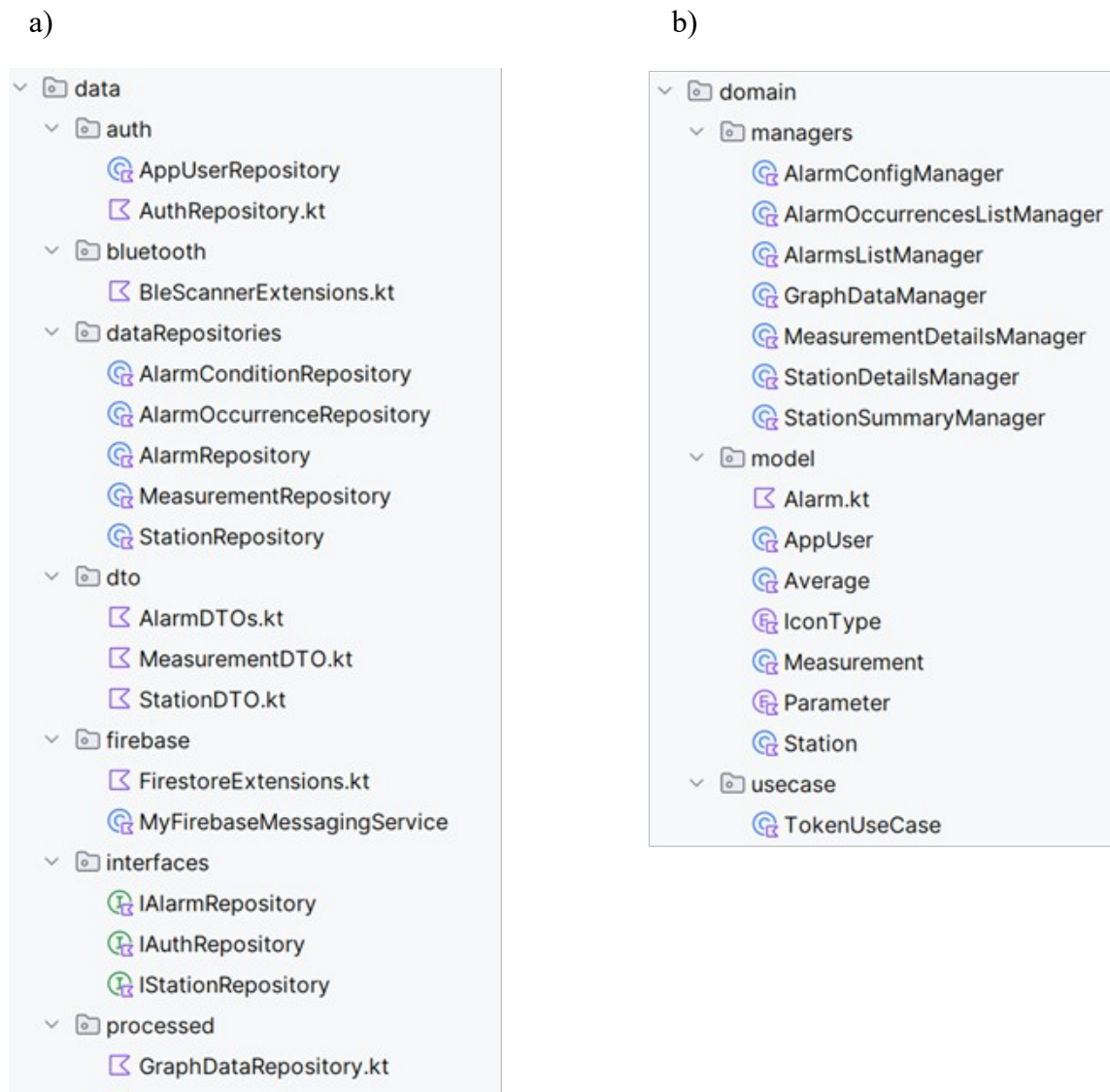
6.3.2 Struktura plików

Rysunek 6.8 przedstawia główną strukturę pakietów aplikacji. 3 główne katalogi odpowiadają warstwom systemu, a katalog „di” i plik „HiltAndroidApp” zawiera kod odpowiedzialny za wstrzykiwanie zależności.



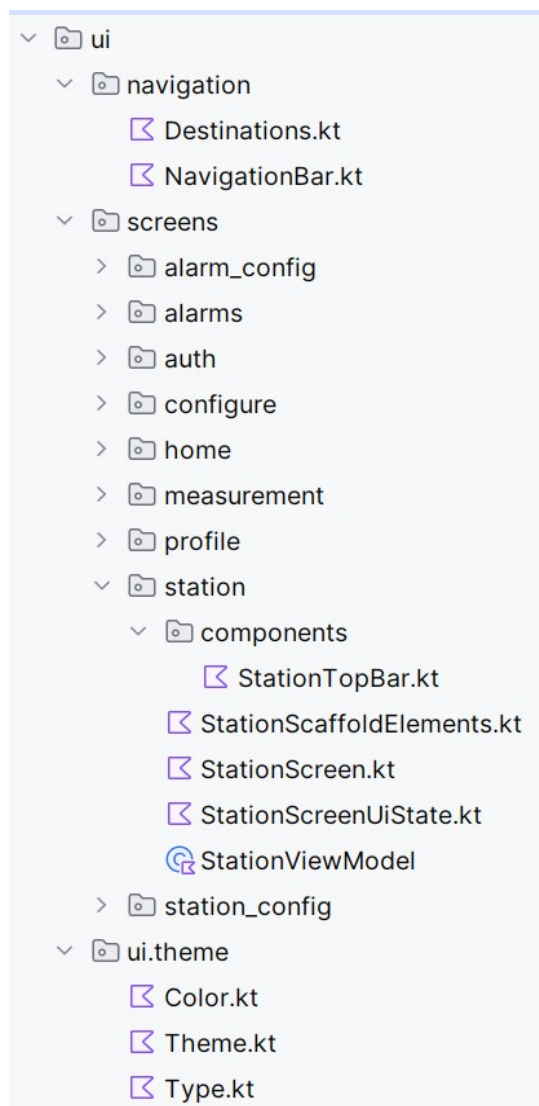
Rysunek 6.8: Podział projektu na pakiety

Rysunek 6.9 przedstawia niskopoziomową strukturę katalogów „data” i „domain”. W pakiecie warstwy danych znajdują się zarówno repozytoria zapewniające dane z bazy danych, jak i takie odpowiadające za uwierzytelnianie, niskopoziomowe funkcje Bluetooth, czy proste przetwarzanie danych (np. liczenie średnich wartości). W tym pakiecie znajdują się też interfejsy niektórych repozytoriów, oraz definicje obiektów transferu danych (ang. DTO) [9]. W pakiecie warstwy domeny znajdują się definicje modeli używanych na potrzeby logiki biznesowej, pojedynczy przypadek użycia (ang. use case), oraz zarządcy. Podział na 2 różne encje obsługujące logikę biznesową nastąpił, ponieważ realizowana funkcjonalność jest kosztowna, co spowodowało konieczność zastosowania pamięci podręcznej (ang. cache). W związku z tym, klasy te całkowicie naruszają główne założenia przypadku użycia, tzn. wielokrotną możliwość tworzenia obiektów i brak przetrzymywania danych.



Rysunek 6.9: Struktura poszczególnych pakietów
a) Pakiet warstwy danych
b) Pakiet warstwy domeny

Rysunek 6.10 przedstawia pakiet UI. Znajdują się w nim 2 proste pliki definiujące komponenty potrzebne do nawigacji, 3 pliki definiujące wygląd interfejsu oraz wiele plików definiujących interfejs użytkownika. Każdy folder w katalogu „screens” ma taką samą strukturę, z plikami nazwanymi tak samo jak w katalogu „stations”, z wyłączeniem przedrostka określającego dany ekran. Struktura folderu „stations” obrazuje więc strukturę wszystkich. Plik kończący się na „Screen” definiuje interfejs, korzystając z komponentów z folderu „components”. Interfejs korzysta ze stanu interfejsu zdefiniowanego w pliku z przyrostkiem „UiState”. Logikę interfejsu obsługują ViewModel-e w plikach zakończonych „ViewModel”. Pliki „ScaffoldElements” definiują pasek u góry ekranu i przyciski w rogu ekranu (ang. Floating Action Button, FAB).



Rysunek 6.10: Struktura katalogu warstwy prezentacji

6.3.3 Dostęp do danych

Firestore jest specyficzną chmurową bazą danych, pozwalającą na bezpośredni dostęp do bazy danych z aplikacji, gdzie serwer jest ukryty za interfejsem biblioteki. W związku z tym, aplikacja mobilna odnosi się do konkretnych kolekcji i dokumentów. Aplikacja korzysta też z aktualizacji danych w czasie rzeczywistym, zapewnianej przez Firestore. Na niskim poziomie, dzieje się to poprzez ustawienie funkcji zwrotnej (ang. callback), która zostanie aktywowana przy zmianie danego dokumentu lub kolekcji. Aby ułatwić to zadanie, utworzone zostały funkcje pomocnicze.

Listing 6.26 przedstawia funkcję ustawiającą funkcję zwrotną dla zmian konkretnego dokumentu. Pierwszym argumentem jest ścieżka do kolekcji, która w przypadku niezagnieżdżonych kolekcji jest po prostu nazwą kolekcji. Warto też zwrócić uwagę na dwa ostatnie argumenty, które przyjmują funkcje mapujące, tzn. takie, które przekonwertują surowe dane pobrane z bazy danych na obiekt transferu danych, a następnie na obiekt domenowy. Taki podział występuje ze względu na możliwość zastosowania różnych funkcji mapujących, w zależności od wymagań konkretnego przypadku użycia.

```
fun <T, D> FirebaseFirestore.documentAsFlow(
    collectionPath: String,
    documentId: String,
    dtoMapper: (doc: DocumentSnapshot) -> T?,
    domainMapper: (dto: T) -> D?,
): Flow<D?> = callbackFlow {
    val listener = collection(collectionPath)
        .document( documentPath = documentId )
        .addSnapshotListener { snapshot, error ->
            if (error != null) {
                close( cause = error )
                return@addSnapshotListener
            }

            val item = snapshot
                ?.let { dtoMapper(it) }
                ?.let { domainMapper(it) }
            trySend( element = item ).isSuccess
        }

    awaitClose { listener.remove() }
}
```

Listing 6.26 Ustawienie słuchania zmian dokumentu

Listing 6.27 przedstawia odpowiednik poprzedniej funkcji dla dowolnych zapytań. Funkcja ta zamiast pojedynczych obiektów zwraca ich listę.

```
fun <T, D> queryAsFlow(
    query: Query,
    dtoMapper: (doc: DocumentSnapshot) -> T?,
    domainMapper: (dto: T) -> D?,
): Flow<List<D>> = callbackFlow {
    val listener: ListenerRegistration =
        query
            .addSnapshotListener { snapshot, error ->
                if (error != null) {
                    close(cause = error)
                    return@addSnapshotListener
                }

                val items = snapshot?.documents
                    ?.mapNotNull { dtoMapper(it) }
                    ?.mapNotNull { domainMapper(it) }
                    .orEmpty()

                trySend(element = items).isSuccess
            }
        awaitClose { listener.remove() }
}
```

Listing 6.27 Ustawienie funkcji słuchającej dla dowolnego zapytania

Obie poprzednie funkcje polegają na mechanizmie Flow obecnym w języku Kotlin. Dzięki niemu, asynchroniczne funkcje mogą zwracać wiele wartości.

Dzięki zdefiniowaniu takich funkcji, tworzenie bardziej skomplikowanych zapytań staje się kwestią zdefiniowania samych wymagań, bez manualnego tworzenia Flow. Celem dalszego zapobiegnięcia powtarzaniu kodu, zostały zdefiniowane dodatkowe funkcje, co przedstawia Listing 6.28.

```

fun <T, D> FirebaseFirestore.filteredCollectionAsFlow(
    collectionPath: String,
    field: String,
    value: Any,
    dtoMapper: (doc: DocumentSnapshot) -> T?,
    domainMapper: (dto: T) -> D?,
): Flow<List<D>> =
    queryAsFlow(
        query = collection(collectionPath).whereEqualTo(field, value),
        dtoMapper,
        domainMapper
    )

fun <T, D> FirebaseFirestore.filteredArrayContainsCollectionAsFlow(
    collectionPath: String,
    arrayField: String,
    value: Any,
    dtoMapper: (doc: DocumentSnapshot) -> T?,
    domainMapper: (dto: T) -> D?,
): Flow<List<D>> =
    queryAsFlow(
        query = collection(collectionPath).whereArrayContains(arrayField, value),
        dtoMapper,
        domainMapper
    )

fun <T, D> FirebaseFirestore.collectionByIdsAsFlow(
    collectionPath: String,
    ids: List<String>,
    dtoMapper: (doc: DocumentSnapshot) -> T?,
    domainMapper: (dto: T) -> D?,
): Flow<List<D>> {
    if (ids.isEmpty()) return flowOf( value = emptyList())
    return queryAsFlow(
        query = collection(collectionPath).whereIn(FieldPath.documentId(), values = ids.take( n = 10)),
        dtoMapper,
        domainMapper
    )
}

```

Listing 6.28 Dodatkowe funkcje otaczające

Pierwsza funkcja służy do pobierania kolekcji filtrowanej po jednym polu. Druga pobiera dokumenty zawierające listę z daną wartością. Trzecia zwraca dokumenty o identyfikatorach zawartych w liście przekazanej jako argument. Schematy te są często używane, przez co zostały wyszczególnione jako funkcje parametryzowane. Istnieje też jednak możliwość używania funkcji „queryAsFlow” bezpośrednio w repozytorium.

Listing 6.29 pokazuje przykładowe repozytorium danych. Wykorzystuje ono poprzednio opisywane funkcje do pobierania danych z bazy na różne możliwe sposoby. Zapis „::dtoMapper” oznacza referencję do funkcji w języku Kotlin.

```
@Singleton
open class StationRepository @Inject constructor() : IStationRepository {

    private val db = Firebase.firestore

    private fun dtoMapper(doc: DocumentSnapshot): StationDto? {
        val ownerId: String = doc.getString( field = "owner_id") ?: return null
        val name: String = doc.getString( field = "name") ?: "Unnamed station"
        return StationDto(
            id = doc.id,
            name = name,
            ownerId = ownerId
        )
    }

    private fun domainMapper(dto: StationDto): Station =
        Station(
            id = dto.id,
            name = dto.name
        )

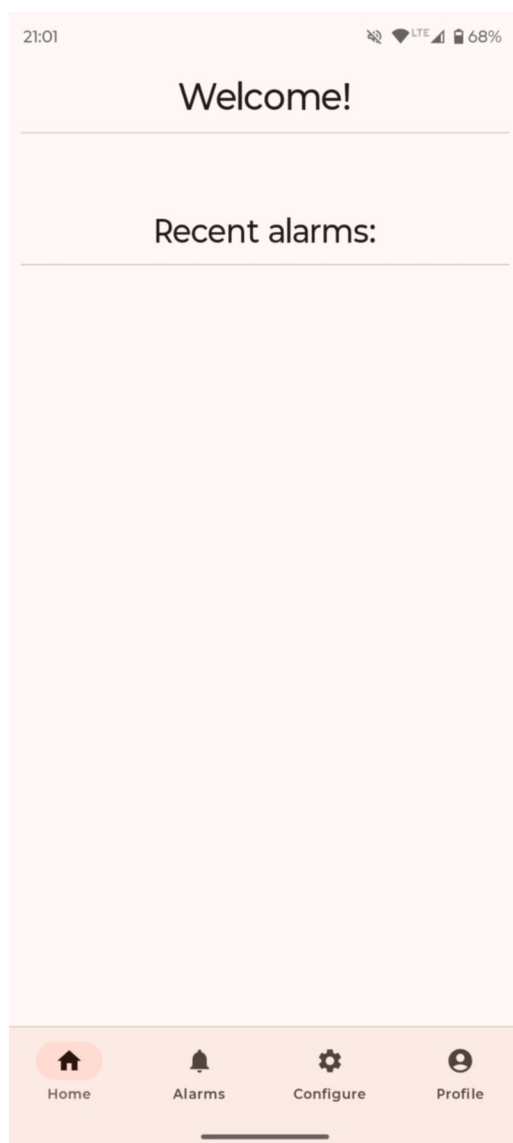
    override fun getStationById(id: String): Flow<Station?> =
        db.documentAsFlow(
            collectionPath = "stations",
            documentId = id,
            dtoMapper = ::dtoMapper,
            domainMapper = ::domainMapper
        )

    override fun getStationsByOwner(ownerId: String): Flow<List<Station>> =
        db.filteredCollectionAsFlow(
            collectionPath = "stations",
            field = "owner_id",
            value = ownerId,
            dtoMapper = ::dtoMapper,
            domainMapper = ::domainMapper
        )
}
```

Listing 6.29 Repozytorium danych o stacji

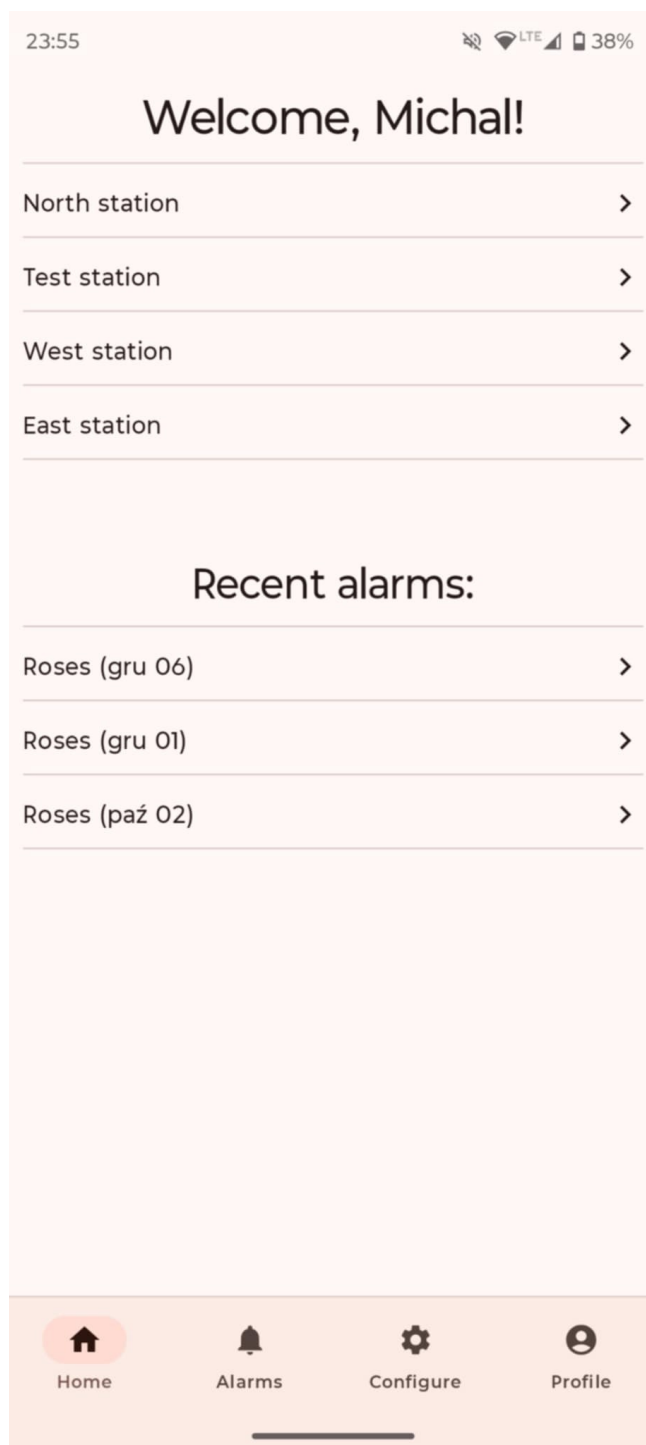
6.3.4 Ekran domowy

Pierwszy ekran jaki użytkownik zobaczy, to pusty ekran główny. Przedstawia go Rysunek 6.11. Jedyna znacząca akcja do jakiej użytkownik ma dostęp to zalogowanie się.



Rysunek 6.11: Pusty ekran główny

Po zalogowaniu, ekran domowy pokazuje wybrany pseudonim użytkownika, wszystkie jego stacje oraz 3 ostatnie wystąpienia alarmu. Nowy użytkownik zobaczy te pola puste. Zarówno stacje jak i wystąpienia są klikalne. Aplikacja przechodzi wtedy odpowiednio do ekranu podsumowania stacji i szczegółów pomiaru.



Rysunek 6.12 Ekran domowy aplikacji

Aby wyświetlić ekran, aplikacja używa klasy przechowującej dane interfejsu. Przedstawia ją Listing 6.30. Składa się ona z list przechowujących obiekty klas pomocniczych. Słowo kluczowe „data” powoduje, że kompilator Kotlin automatycznie generuje kod metod specyficznych dla klas przechowujących dane, takich jak „equals”, czy „copy”

```
data class StationSummaryItemUiState(  
    val stationId: String = "",  
    val name: String = "",  
    val hasNotification: Boolean = false,  
    val hasError: Boolean = false,  
)  
  
data class RecentAlarmOccurrenceItemUiState(  
    val alarmName: String = "",  
    val measurementId: String = "",  
    val date: LocalDateTime = LocalDateTime.now()  
)  
  
data class HomeUiState(  
    val isAnonymous: Boolean = true,  
    val username: String = "",  
    val stations: List<StationSummaryItemUiState> = emptyList(),  
    val recentAlarmOccurrences: List<RecentAlarmOccurrenceItemUiState> = emptyList(),  
)
```

Listing 6.30: Klasa przechowująca stan interfejsu ekranu domowego

Dzięki takiej klasie, aplikacja może łatwo przekazywać dane z ViewModel-u do ekranu Compose. Dzieje się to poprzez udostępnianie pola obiektu podczas tworzenia interfejsu. Pokazuje to Listing 6.31. Widać na nim też wstrzykiwanie zależności ViewModel-u.

```
@Composable  
fun HomeScreen(  
    homeViewModel: HomeViewModel = hiltViewModel(),  
    onStationSummaryClicked: (String) -> Unit,  
    onRecentAlarmOccurrenceClicked: (String) -> Unit,  
) {  
    val uiState: HomeUiState by homeViewModel.uiState.collectAsState( initial = HomeUiState())  
  
    HomeScreenContent(uiState, onStationSummaryClicked, onRecentAlarmOccurrenceClicked)  
}
```

Listing 6.31 Tworzenie ekranu domowego Compose

Wywołanie metody „collectAsState”, widoczne na poprzednim listingu, wynika z faktu, że ViewModel udostępnia pole typu Flow. Metoda ta tworzy specjalny obiekt State, który dla każdej wartości Flow spowoduje rekompozycję ekranu. Rekompozycja w Compose to proces ponownego utworzenia komponentów, które obserwowały zmieniony stan [8]. Ten schemat udostępniania Flow, a następnie „zbierania” go jako State jest powszechnie wykorzystywany przy każdym ekranie.

```
@HiltViewModel
class HomeViewModel @Inject constructor(
    authRepository: AuthRepository,
    stationSummaryManager: StationSummaryManager,
    alarmOccurrencesListManager: AlarmOccurrencesListManager
) : ViewModel() {

    val uiState: Flow<HomeUiState> =
        combine(
            flow = authRepository.currentUser,
            flow2 = stationSummaryManager.stationsSummaryForUser(),
            flow3 = alarmOccurrencesListManager.recentAlarmOccurrences(),
        ) { user, stations, alarms ->
            HomeUiState(
                isAnonymous = user.isAnonymous,
                username = user.username,
                stations = stations,
                recentAlarmOccurrences = alarms,
            )
        }

    companion object {
        private const val TAG = "HomeViewModel"
    }
}
```

Listing 6.32 ViewModel ekranu domowego

ViewModel ekranu domowego jest prosty, co pokazuje Listing 6.32. Adnotacje „@Inject” w konstruktorze wraz z „@HiltViewModel” służą wstrzykiwaniu zależności w sposób świadomy cyklu życia elementów Android-a. Głównym zadaniem kodu jest zbieranie Flow z menadżerów wstrzykiwanych w konstruktorze i udostępnianie ich dla interfejsu. Zapis „companion object” jest odpowiednikiem słowa kluczowego „static” w języku Kotlin. Pole „TAG” służy oznaczaniu wiadomości dla loggera.

Listing 6.33 zawiera kod przykładowego menadżera obserwującego dane w czasie rzeczywistym. Pozwala on na automatyczną aktualizację interfejsu kiedy tylko zmienia się dane w bazie Firebase. Klasa osiąga to poprzez zarządzanie obiektem typu „MutableStateFlow”. Kiedy menadżer jest tworzony po raz pierwszy, obiekt ten jest wypełniany odpowiednimi danymi i klasa dalej nasłuchuje zmian. Kolejne wywołania funkcji „stationsSummaryForUser” zwracają zawartość wspomnianego obiektu. Ten schemat jest obecny w wielu menadżerach danych.

```
@Singleton
class StationSummaryManager @Inject constructor(
    @param:ApplicationScope private val scope: CoroutineScope,
    authRepository: AuthRepository,

    private val repository: StationRepository
) {
    fun stationsSummaryForUser(): Flow<List<StationSummaryItemUiState>> =
        _stationsSummaryForUserCache

    init {
        authRepository.currentUser
            .map { it.id }
            .distinctUntilChanged()
            .onEach { userId ->
                observeStationsSummaryForUser(userId)
            }
            .launchIn(scope)
    }

    private val _stationsSummaryForUserCache =
        MutableStateFlow<List<StationSummaryItemUiState>>(value = emptyList())
    private var _stationsSummaryForUserJob: Job? = null

    private fun observeStationsSummaryForUser(userId: String) {
        _stationsSummaryForUserJob?.cancel()
        _stationsSummaryForUserJob = repository.getStationsByOwner( ownerId = userId)
            .map { stations ->
                stations.map { station ->
                    StationSummaryItemUiState(
                        stationId = station.id,
                        name = station.name,
                    )
                }
            }
            .onEach { _stationsSummaryForUserCache.value = it }
            .launchIn(scope)
    }
}
```

Listing 6.33 Menadżer podsumowania stacji

Listing 6.34 pokazuje, jak tworzona jest zawartość tego ekranu. Jak większość ekranów w tej aplikacji, interfejs oparty jest na elemencie „LazyColumn”. Jest to opakowanie na listę elementów o zmiennej wysokości. Automatycznie pozwala ono także na przewijanie ekranu.

```
@Composable
private fun HomeScreenContent(
    uiState: HomeUiState,
    onStationSummaryClicked: (String) -> Unit,
    onRecentAlarmOccurrenceClicked: (String) -> Unit,
) {
    LazyColumn (
        verticalArrangement = Arrangement.spacedBy( space = 4.dp),
    ) {
        item() {...}
        items( items = uiState.stations) {...}
        item() {...}
        items( items = uiState.recentAlarmOccurrences) {...}
    }
}
```

Listing 6.34 Tworzenie zawartości ekranu domowego

Elementy kolumny definiowane są za pomocą funkcji „item” lub „items”. Pierwsza z nich odpowiada głównie za nagłówki i inne pojedyncze elementy. Funkcja „items” z kolei, tworzy dany komponent dla każdego elementu listy przekazanej jako parametr. Przykładową zawartość takiego wywołania prezentuje Listing 6.35.

```

item() {
    Spacer(Modifier.height( height = 50.dp))
    Text(
        text = "Recent alarms:",
        Modifier.fillMaxWidth().padding( all = 10.dp),
        style = MaterialTheme.typography.headlineMedium,
        textAlign = TextAlign.Center
    )
    HorizontalDivider(Modifier.padding(horizontal = 10.dp))
}

items( items = uiState.recentAlarmOccurrences) { alarm ->
    Row (...) {
        val recentName =
            alarm.alarmName +
            " (" +
            alarm.date
                .format(
                    formatter = DateTimeFormatter.ofPattern( pattern = "MMM dd")
                ) +
            ")"
        Text( text = recentName,
            modifier = Modifier.weight( weight = 1f))
        Icon(
            imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
            contentDescription = "Show details")
    }
    HorizontalDivider(Modifier.padding(horizontal = 10.dp))
}

```

Listing 6.35 Tworzenie elementów kolumny ekranu domowego

Nietypowe zastosowanie nawiasów klamrowych za wywołaniem funkcji jest elementem języka Kotlin. Jest to równoznaczne z przekazaniem funkcji anonimowej zawartej w nawiasach klamrowych jako ostatni parametr funkcji poprzedzającej.

Rozdzielenie tworzenia ekranu i jego zawartości jest bardzo ważnym elementem kodu, znacznie ułatwiającym testowanie samej zawartości ekranu.

Funkcjonalnością strony domowej jest też wyświetlanie powiadomień. Stacje, które zgłosiły błąd lub dowolne inne powiadomienie wyświetlane są z odpowiednim symbolem, co przedstawia Rysunek 6.13. Konkretny symbol zależy od typu powiadomienia. Aktualnie obsługiwane są typy „warning” i „notification”. System przystosowany jest jednak do obsługi dowolnych powiadomień.

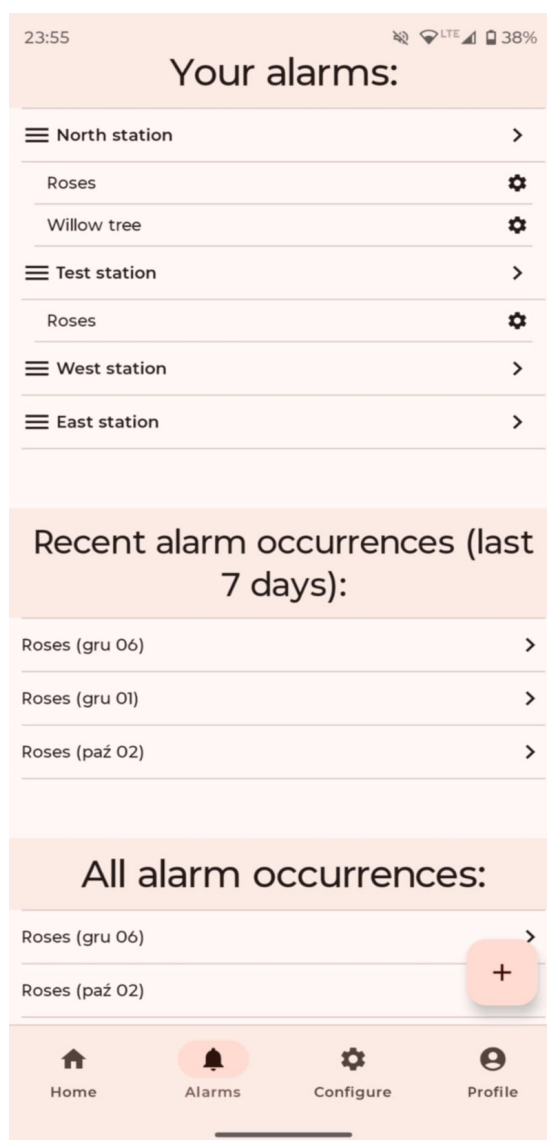
South station	>
North station	 >
East station	 >
West station	  >

Rysunek 6.13: Stacje z różną konfiguracją powiadomień

6.3.5 Alarmy

Drugim z głównych ekranów jest ekran alarmów. Jego główną częścią jest spis alarmów. Są one posegregowane po stacji. W przypadku, kiedy więcej niż jedna stacja obsługuje dany alarm, jest on wyświetlany wielokrotnie. Po kliknięciu w stację, użytkownik zostanie przekierowany do ekranu szczegółów stacji. Naciśnięcie alarmu spowoduje przejście do jego konfiguracji.

Poniżej znajdują się 2 sekcje zawierające wystąpienia alarmu. Pierwsza wyświetla dane z ostatnich 7 dni, natomiast druga wszystkie odczytane. Podział taki został zastosowany ze względu na dużo wyższy priorytet ostatnich wystąpień alarmu. Chcemy, aby użytkownik podświadomie oddzielał te sekcje. Drugim względem jest też możliwość przyszłej rozbudowy systemu o takie funkcjonalności jak stopnie alarmu czy oznaczanie ich jako przeczytanych.



Rysunek 6.14 Ekran alarmów

6.3.6 Pomiar

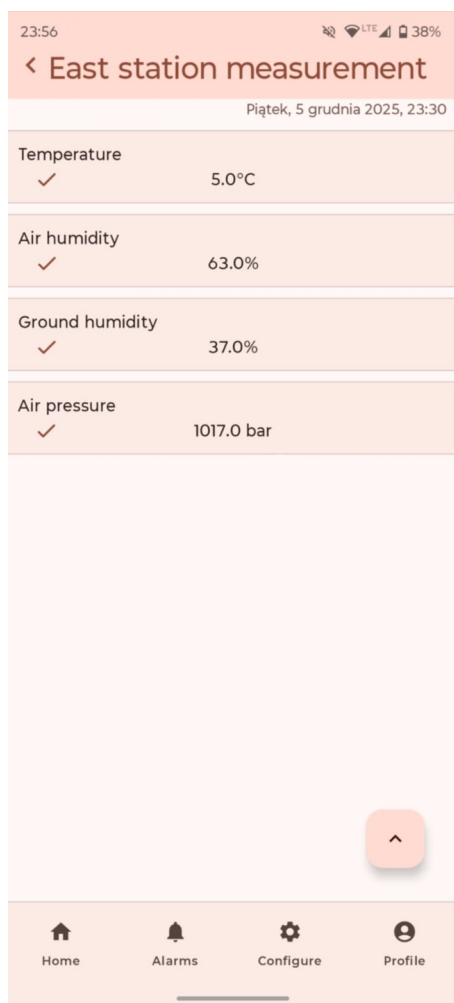
Aplikacja umożliwia sprawdzanie wartości każdego konkretnego pomiaru. Ekran jest identyfikowany przez nazwę stacji na górnym pasku ekranu, oraz dokładną datę wykonania tego pomiaru.

Wartości pomiaru podzielone są na segmenty. Każdy wyświetla nazwę parametru oraz wartość dokładną do 1 miejsca po przecinku, wraz z jednostką. W przypadku, kiedy dana wartość przekracza zakres jakiegoś (lub wielu) alarmów, pod nazwą zostaje wyświetlona nazwa tego alarmu, po której kliknięciu użytkownik zostaje przekierowany do ekranu konkretnego alarmu.

W przypadku brakującej wartości, aplikacja nie pokazuje danego segmentu. Takie zachowanie jest spowodowane pragnieniem przystosowania systemu do innych konfiguracji stacji, gdzie różne urządzenia mogą mieć różne czujniki.

W prawym dolnym rogu znajduje się menu, pozwalające usunąć pomiar z bazy.

a)



b)



Rysunek 6.15 Ekran szczegółów pomiaru

a) Pomiar normalny z brakującą wartością naświetlenia

b) Pomiar, który wywołał alarm z rozwiniętym menu

Ekran pomiaru jest jedną z części aplikacji, w której nie jest wykorzystywana aktualizacja w czasie rzeczywistym. Ułatwia to pracę z danymi i przyspiesza proces wytwarzania oprogramowania. W związku z tym, menadżer danych dla tego ekranu również działa inaczej. Pobiera on tylko jedną emisję z repozytoriów, po czym zamyka Flow. Pokazuje to Listing 6.36. Mapa wystąpień zawiera listę wystąpień alarmu dla każdego parametru.

```
suspend fun measurementDetailsSnapshot(measurementId: String): MeasurementScreenState {
    return try {
        val measurement = measurementRepository
            .getMeasurementById(measurementId)
            .firstOrNull()
        ?: return MeasurementScreenState.Error( message = "Measurement not found!")

        val occurrences = occurrencesRepository
            .getOccurrencesForMeasurement(measurementId)
            .firstOrNull()
            .orEmpty()

        val station = stationRepository
            .getStationById(measurement.stationId)
            .firstOrNull()
        ?: return MeasurementScreenState.Error( message = "Measurement without station!")

        val occurrenceMap = occurrences.mapNotNull { occurrence ->
            val alarm =
                alarmRepository
                    .getAlarmById(occurrence.alarmId)
                    .firstOrNull()
            val condition =
                conditionRepository
                    .getConditionById(occurrence.conditionId, occurrence.alarmId)
                    .firstOrNull()
            if (alarm != null && condition != null) {
                condition.parameter to TriggeredAlarm(
                    alarmId = alarm.id,
                    alarmName = alarm.name
                )
            } else null
        }.groupBy(
            keySelector = { it.first },
            valueTransform = { it.second }
        )
    }
}
```

Listing 6.36 Pierwsza część funkcji zestawiającej dane pomiaru

Druga część tej funkcji, mapuje dane na obiekt stanu interfejsu do wyświetlenia.

```
return MeasurementScreenState.MeasurementData(  
    stationName = station.name,  
    measurement.stationId,  
    measurement.date,  
    cards = listOf(  
        Parameter.TEMPERATURE to measurement.temperature,  
        Parameter.AIR_HUMIDITY to measurement.airHumidity,  
        Parameter.GROUND_HUMIDITY to measurement.groundHumidity,  
        Parameter.LIGHT to measurement.light,  
        Parameter.PRESSURE to measurement.pressure  
    ).map { (type, level) ->  
        MeasurementCardFactory.create(type, level, triggeredAlarms = occurrenceMap[type] ?: emptyList())  
    }  
)  
} catch (e: Exception) {  
    MeasurementScreenState.Error( message = e.message ?: "Unknown error")  
}  
}
```

Listing 6.37 Druga część funkcji pobierającej dane pomiaru

Klasa „MeasurementCardFactory” odpowiada za wypełnienie obiektu danymi pobranymi z obiektu typu wyliczeniowego „Parameter”. Implementuje ona wzorzec kreacyjny fabryki.

```
class MeasurementCardFactory {  
    companion object {  
        fun create(  
            type: Parameter,  
            level: Double?,  
            triggeredAlarms: List<TriggeredAlarm>  
        ): MeasurementCard {  
            return MeasurementCard(  
                title = type.title,  
                unit = type.unit,  
                value = level ?: 0.0,  
                triggeredAlarms = triggeredAlarms  
            )  
        }  
    }  
}
```

Listing 6.38 Fabryka obiektów interfejsu

Klasa wyliczeniowa „Parameter” jest pojedynczym źródłem prawdy na temat właściwości atmosferycznych o jakich wie aplikacja mobilna. Zawiera ona stałe wartości, które używane są w różnych miejscach interfejsu.

```
enum class Parameter(  
    val title: String,  
    val unit: String,  
    val dbName: String,  
    val descriptionText: String,  
    val minValue: Double,  
    val maxValue: Double,  
    val icon: IconType  
) {  
    10 Usages  
    TEMPERATURE( title = "Temperature", unit = "°C", dbName = "temperatureAir",  
        descriptionText = "temperatures", minValue = -30.0, maxValue = 50.0,  
        IconType.Dataset),  
    9 Usages  
    AIR_HUMIDITY( title = "Air humidity", unit = "%", dbName = "humidityAir",  
        descriptionText = "air humidity levels", minValue = 0.0, maxValue = 100.0,  
        IconType.Dataset),  
    7 Usages  
    GROUND_HUMIDITY( title = "Ground humidity", unit = "%", dbName = "humiditySoil",  
        descriptionText = "ground humidity levels", minValue = 0.0, maxValue = 100.0,  
        IconType.Dataset),  
    7 Usages  
    LIGHT( title = "Light level", unit = " lux", dbName = "sunlight",  
        descriptionText = "light levels", minValue = 1.0, maxValue = 350.0,  
        IconType.Dataset),  
    8 Usages  
    PRESSURE( title = "Air pressure", unit = " bar", dbName = "pressureAir",  
        descriptionText = "pressure levels", minValue = 900.0, maxValue = 1200.0,  
        IconType.Dataset)  
}
```

Listing 6.39 Klasa wyliczeniowa zawierająca informacje o parametrach

Usuwanie pomiaru odbywa się poprzez wywołanie prostej funkcji w repozytorium. Pokazuje to Listing 6.40.

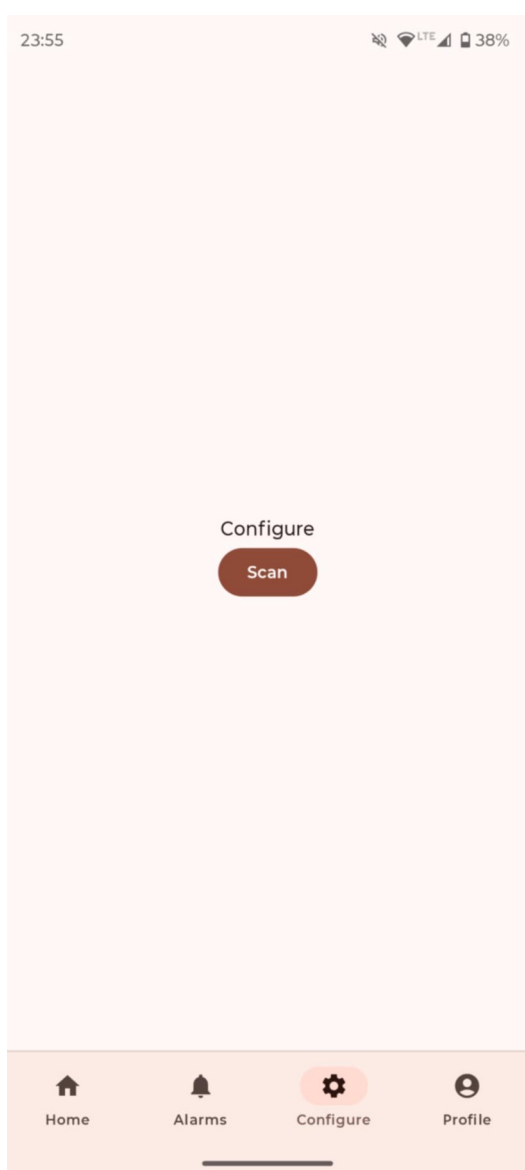
```
suspend fun deleteMeasurement(measurementId: String): MeasurementScreenState {  
    try {  
        measurementRepository.deleteMeasurement(measurementId).first()  
        return MeasurementScreenState.Deleted  
    } catch (e: Exception) {  
        return MeasurementScreenState.Error( message = e.message ?: "Unknown error")  
    }  
}
```

Listing 6.40 Wywołanie usunięcia pomiaru

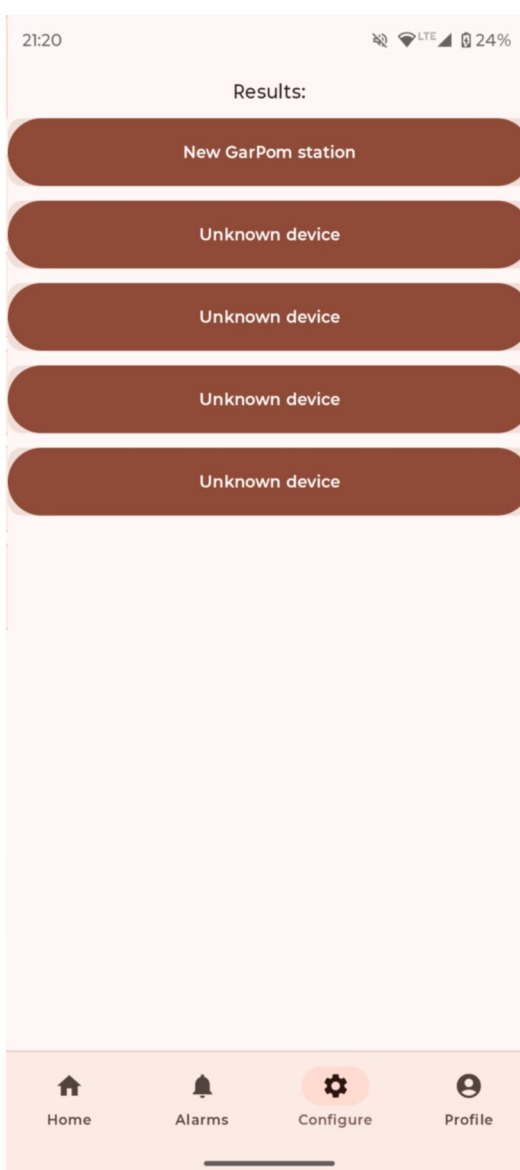
6.3.7 Konfiguracja stacji

Aby system był użyteczny, użytkownik musi przejść bardzo prostą procedurę konfiguracji stacji. Po kliknięciu w przycisk, urządzenie, po sprawdzeniu i ewentualnym zażądaniu uprawnień, skanuje urządzenia Bluetooth Low Energy (BLE) w pobliżu. Rozpoznane urządzenia z tego systemu prezentowane są po nazwie, a inne jako nierozpoznane. Decyzja pokazywania wszystkich urządzeń wynika z chęci zapewnienia możliwości ewentualnego stworzenia własnej stacji przez użytkownika zaawansowanego technicznie. Po kliknięciu w wynik, aplikacja przechodzi do ekranu konfiguracji stacji.

a)



b)

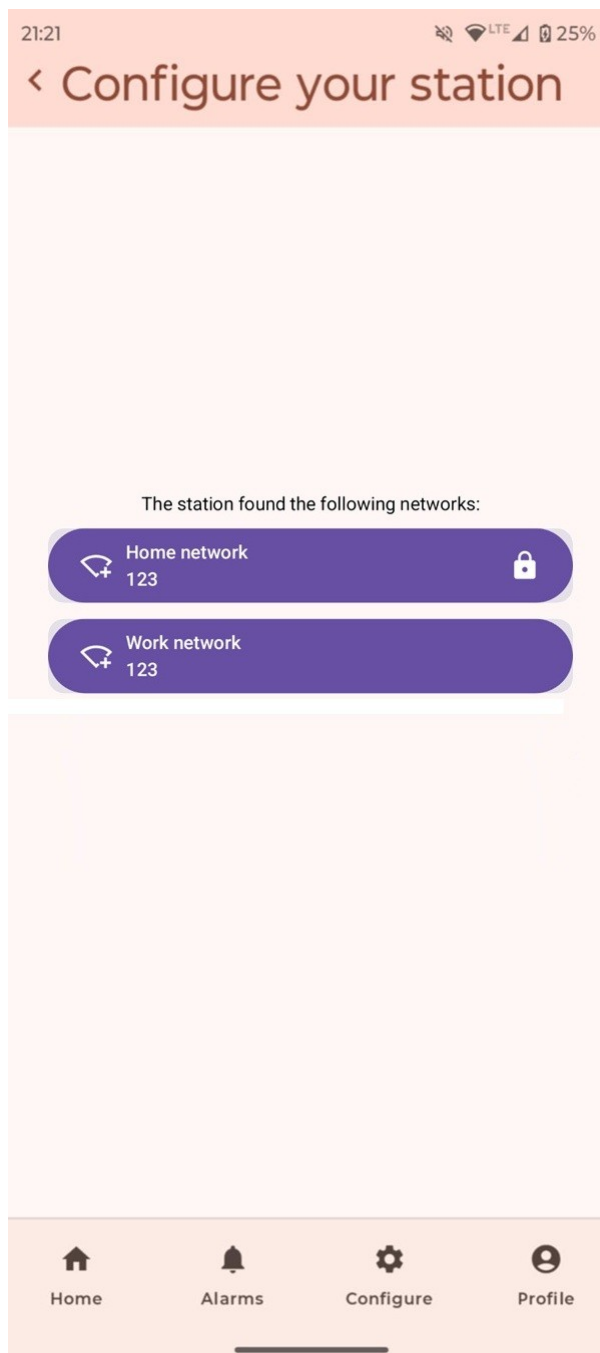


Rysunek 6.16 Ekran konfiguracji

a) Ekran proszący użytkownika o włączenie skanowania

b) Ekran wyboru stacji do połączenia

Po nawiązaniu połączenia ze stacją, aplikacja przechodzi do ekranu konfiguracji stacji. Można na nim skonfigurować połączenie stacji z siecią Wi-Fi. Lista sieci które wykrywa stacja znajduje się na środku ekranu. Są one odpowiednio oznaczone, w zależności od statusu połączenia i obecności hasła. Po kliknięciu sieci, użytkownik może wpisać hasło w dialogu (o ile jest wymagane). Stacja spróbuje wtedy nawiązać połączenie z daną siecią.



Rysunek 6.17 Ekran konfiguracji stacji

Cały proces konfiguracji mimo, że prosty z punktu widzenia użytkownika, składa się z wielu części. Pierwszym problemem jest uzyskanie pozwolenia od użytkownika na wykorzystanie Bluetooth. Wymaga to między innymi wyświetlania dialogów w dowolnym momencie procesu. Najważniejszą funkcją jest ta, sprawdzająca wymagane uprawnienia. Prezentuje ją Listing 6.41.

```
fun hasBtPermissions(): Boolean {  
    val scan = ContextCompat.checkSelfPermission(  
        context, permission = Manifest.permission.BLUETOOTH_SCAN  
    ) == PackageManager.PERMISSION_GRANTED  
  
    val connect = ContextCompat.checkSelfPermission(  
        context, permission = Manifest.permission.BLUETOOTH_CONNECT  
    ) == PackageManager.PERMISSION_GRANTED  
  
    val location = ContextCompat.checkSelfPermission(  
        context, permission = Manifest.permission.ACCESS_FINE_LOCATION  
    ) == PackageManager.PERMISSION_GRANTED  
  
    return scan && connect && location  
}
```

Listing 6.41 Funkcja sprawdzania uprawnień

Pokazywanie dialogów na różnych etapach konfiguracji zostało osiągnięte poprzez rozdzielenie stanu UI na dwa składowe interfejsy. Pokazuje je Listing 6.42.

```
4 Implementations  
interface ScreenState {  
    data object Initial : ScreenState  
    data object Scanning : ScreenState  
    data object ScanResults : ScreenState  
    data class PairingError(val message: String) : ScreenState  
}  
  
3 Implementations  
interface DialogState {  
    data object PermissionExplanationNeeded : DialogState  
    data object PermissionsDenied : DialogState  
    data object DeviceIncompatible : DialogState  
}  
  
data class ConfigureUiState(  
    val screenState: ScreenState = ScreenState.Initial,  
    val dialog: DialogState? = null  
)
```

Listing 6.42 Klasa przechowująca stan interfejsu konfiguracji

Jako jedna destynacja nawigacji obsługująca wiele ekranów, konieczne jest wyświetlenie odpowiedniej zawartości. Osiągnięte jest to poprzez sprawdzenie, która implementacja interfejsu została przekazana.

```
when (val s = configureState.value.screenState) {
    ScreenState.Initial -> {
        InitialScreen(
            onPair = { configureViewModel.initialConfiguration(activity!!) }
        )
    }
    ScreenState.Scanning -> {
        Column (
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Text( text = "Scanning...")
            LoadingIndicator()
        }
    }
    ScreenState.ScanResults -> {
        ScanResults(
            scanResultsUiState = scanResultsState,
            onRestart = { configureViewModel.initialConfiguration(activity!!) },
            onResultClicked = { onStationChosen(it) }
        )
    }
    is ScreenState.PairingError -> {
        Column (
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Text( text = "Pairing error: " + s.message)
        }
    }
}
```

Listing 6.43 Sprawdzanie klasy stanu interfejsu

Ciekawym jest sposób pokazywania tych powiadomień. W związku cyklem życia i implementacją elementów interfejsu na platformie Android, kod uruchamiający systemowe żądanie uprawnień nie może znajdować się daleko od interfejsu (w sensie abstrakcji). ViewModel musi więc w pewien sposób komunikować się z interfejsem, aby ten użył systemowych funkcji. Zostało to osiągnięte za pomocą funkcji zwrotnych (ang. *callback*). W ViewModel-u został zdefiniowany interfejs, którego używa on do żądania czynności od interfejsu. Prezentuje go Listing 6.44. Najważniejsze są tu dwie zdefiniowane zmienne.

```
// callback set by the Composable to trigger permission launcher
private var _requestPermissionsCallback: (() -> Unit)? = null
fun setRequestPermissionsCallback(callback: () -> Unit) {
    _requestPermissionsCallback = callback
}

private var _bluetoothEnableCallback: (() -> Unit)? = null
fun setBluetoothEnableCallback(callback: () -> Unit) {
    _bluetoothEnableCallback = callback
}

fun onExplanationAccepted() {
    _uiState.value = _uiState.value.copy(dialog = null)
    _requestPermissionsCallback?.invoke()
}

@RequiresPermission(value = Manifest.permission.BLUETOOTH_CONNECT)
fun onPermissionsGranted() {
    _uiState.value = _uiState.value.copy(dialog = null)
    initiatePairing()
}

@RequiresPermission(value = Manifest.permission.BLUETOOTH_CONNECT)
fun onBluetoothEnabled() {
    initiatePairing()
}

fun onPermissionsDenied() {
    _uiState.value = _uiState.value.copy(dialog = DialogState.PermissionsDenied)
}

fun clearDialog() {
    _uiState.value = _uiState.value.copy(dialog = null)
}
```

Listing 6.44 Funkcje służące do komunikacji z interfejsem

Tworzenie tych „launcherów” przebiega jak na Listingu 6.45.

```
val permissionLauncher = rememberLauncherForActivityResult(  
    contract = ActivityResultContracts.RequestMultiplePermissions()  
) { permissions ->  
    val granted = permissions.values.all { it }  
    if (granted) {  
        configureViewModel.onPermissionsGranted()  
    } else {  
        configureViewModel.onPermissionsDenied()  
    }  
}  
  
val btEnableLauncher = rememberLauncherForActivityResult(  
    contract = ActivityResultContracts.StartActivityForResult()  
) { result ->  
    when (result.resultCode) {  
        RESULT_CANCELED -> {  
            Log.d(TAG, msg = "Bluetooth enable canceled")  
        }  
        RESULT_OK -> {  
            configureViewModel.onBluetoothEnabled()  
        }  
    }  
}
```

Listing 6.45 Tworzenie "launcherów" akcji systemowych

Następnie utworzone obiekty są używane w funkcjach anonimowych, które z kolei przekazywane są do odpowiednich funkcji ViewModel-u. Funkcja „LaunchedEffect” zapewnia odpowiednie zachowanie kodu w cyklu życia ekranu.

```
LaunchedEffect( key1 = Unit) {  
    configureViewModel.setRequestPermissionsCallback {  
        permissionLauncher.launch(  
            input = arrayOf(  
                Manifest.permission.BLUETOOTH_SCAN,  
                Manifest.permission.BLUETOOTH_CONNECT,  
                Manifest.permission.ACCESS_FINE_LOCATION  
            )  
        )  
    }  
    configureViewModel.setBluetoothEnableCallback {  
        btEnableLauncher.launch( input = Intent(  
            action = BluetoothAdapter.ACTION_REQUEST_ENABLE))  
    }  
}
```

Listing 6.46 Przekazywanie funkcji anonimowych do ViewModel-u

Za skanowanie urządzeń BLE w pobliżu odpowiadają funkcje widoczne na Listingu 6.47.

```
@RequiresPermission( value = Manifest.permission.BLUETOOTH_CONNECT)
fun initiatePairing(resultCallback: (ScanResult) -> Unit, onFinishedCallback: () -> Unit) {
    if (bluetoothAdapter == null) {
        throw DeviceIncompatibleException()
    }
    if (!bluetoothAdapter.isEnabled) {
        throw BluetoothDisabled()
    }
    scanBluetooth(resultCallback, onFinishedCallback)
}

fun scanBluetooth(resultCallback: (ScanResult) -> Unit, onFinishedCallback: () -> Unit) {
    Log.d(TAG, msg = "Connecting")
    bluetoothLeScanner = bluetoothAdapter?.bluetoothLeScanner

    if (scanJob != null) return

    scanJob = scope.launch {
        withTimeoutOrNull( timeMillis = SCAN_TIMEOUT) {
            bluetoothLeScanner?.scanAsFlow()?.collect { result ->
                _scanResultsDevices.add(result) // for later connections
                resultCallback(result)
            }
        }
        onFinishedCallback()
        stopScan()
    }
}
```

Listing 6.47 Funkcje skanujące Bluetooth

Za połączenie odpowiada funkcja „connectToResultByAddress”, widoczna na Listingu 6.48

```
@RequiresPermission( value = Manifest.permission.BLUETOOTH_CONNECT)
fun connectToResultByAddress(address: String) {
    stopScan()
    val result = _scanResultsDevices.find { it.device.address == address }

    if (result == null) {
        Log.e(TAG, msg = "Invalid address selected")
        return
    }
    result.device!!.connectGatt(context, autoConnect = false, bluetoothGattCallback)
}
```

Listing 6.48 Łączenie z rezultatem skanowania Bluetooth

Na niższym poziomie zdefiniowana jest funkcja pomocnicza. Uruchamia ona skan, konwertuje wyniki na Flow, jak widoczne jest na Listingu 6.49.

```
fun BluetoothLeScanner.scanAsFlow(): Flow<ScanResult> = callbackFlow {
    val callback = object : ScanCallback() {
        override fun onScanResult(callbackType: Int, result: ScanResult) {
            super.onScanResult(callbackType, result)
            Log.d( tag = "btScanner", msg = "Sending result")
            trySend( element = result)
        }

        override fun onBatchScanResults(results: MutableList<ScanResult>) {
            results.forEach { trySend( element = it) }
        }

        override fun onScanFailed(errorCode: Int) {
            close( cause = RuntimeException( message = "Scan failed: $errorCode"))
        }
    }

    startScan(
        // in the future, maybe filter results by manufacturer
        // listOf(
        //     ScanFilter
        //         .Builder()
        //         .setManufacturerData(
        //             MANUFACTURER_ID,
        //             MANUFACTURER_DATA
        //         )
        //         .build()
        // ),
        filters = emptyList(),
        settings = ScanSettings.Builder().build(),
        callback)
    Log.d( tag = "btScanner", msg = "Started scan")

    awaitClose {
        stopScan(callback)
    }
}
```

Listing 6.49 Niskopoziomowe skanowanie BLE

6.3.8 Alarmy

Użytkownik może skonfigurować szereg właściwości alarmu. Może on całkowicie wyłączyć alarm, skonfigurować nazwę, opis, aktywować go na różnych stacjach, ustawić czas poza którym pomiary są ignorowane i co najważniejsze, ustawić zakresy dla wszystkich wartości.

Użytkownik może wprowadzić dowolne zmiany, które zostaną zapisane dopiero po naciśnięciu przycisku w rogu ekranu. Do wartości tekstowych i zakresów wartości zastosowano także przyciski resetujące, aby usprawnić korzystanie z systemu.

a)

23:55

< Roses

This alarm is enabled

Alarm name
Roses Reset

Alarm description
Roses beside the road Reset

Active only between:
6:35 : 19:10

Stations that use this alarm:

North station

Test station

West station

East station

+

Select desired ranges: Reset

Temperature ✓

Home Alarms Configure Profile

b)

23:55

< Roses

Select desired ranges: Reset

Temperature
This alarm will go off for temperatures above 18,8°C

Air humidity
This alarm will go off for air humidity levels below 31,3%

Ground humidity
This alarm will go off for ground humidity levels below 29,0%, or above 65,9%

Light level
This alarm will not check light levels ✓

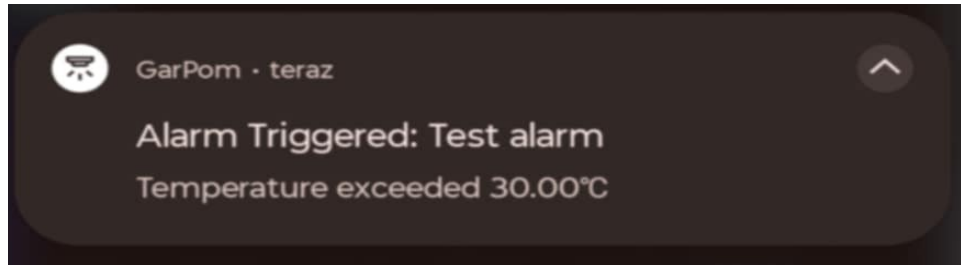
Home Alarms Configure Profile

Rysunek 6.18 Ekran konfiguracji alarmu

a) Konfiguracja włączenia, nazwy, opisu, czasu działania, wykorzystywanych stacji

b) Konfiguracja zakresów parametrów

Alarmy są kluczową funkcjonalnością aplikacji. Aby nadać im większą użyteczność, serwer wykorzystuje funkcję Firebase Cloud Messaging (FCM) [7] do przesyłania powiadomień Push na zarejestrowane urządzenie. Przykładowe powiadomienie prezentuje Rysunek 6.19.



Rysunek 6.19 Przykładowe powiadomienie

Przesyłanie powiadomień to odpowiedzialność kodu serwera. Aby było to jednak możliwe, w bazie danych muszą znajdować się tokeny FCM. W zapisywanym dokumencie znajduje się też dokładny czas wykonania operacji, jak pokazano na Listingu 6.50. Dzięki temu, możliwa jest kontrola i docelowo usuwanie nieużywanych tokenów.

```
fun sendRegistrationToServer(token: String?) {
    Log.d(TAG, msg = "sendRegistrationTokenToServer($token)")
    val deviceToken = hashMapOf(
        "token" to token,
        "timestamp" to FieldValue.serverTimestamp(),
    )
    if (firebaseAuth.currentUser == null) {
        Log.e(TAG, msg = "Send token called, but no user in auth")
        return
    }
    val userId = firebaseAuth.currentUser!!.uid
    Firebase.firestore
        .collection( collectionPath = "fcmTokens")
        .document( documentPath = userId)
        .set(deviceToken)
}
```

Listing 6.50 Wysyłanie tokenu do bazy danych

Aby obsługiwać przychodzące wiadomości, należy utworzyć i zarejestrować w aplikacji specjalny serwis. Implementację w tej aplikacji pokazuje Listing 6.51. Metoda „onCreate” odpowiada za utworzenie kanału dla powiadomień, zgodnie z wymaganiami nowych systemów Android [6]. Metoda „onNewToken” z kolei jest wywoływana kiedy środowisko Firebase zarządza aktualizacji tokenu. Co najważniejsze, ma to też miejsce przy pierwszym uruchomieniu aplikacji. Reasumując, ta metoda gwarantuje, że w bazie danych znajduje się aktualny token.

```
class MyFirebaseMessagingService : FirebaseMessagingService() {
    @Inject
    lateinit var serverInteractor: TokenUseCase
    @Inject
    lateinit var auth: FirebaseAuth

    override fun onCreate() {
        val channelId = "alarms"
        val channelName = "Alarms"

        val importance = NotificationManager.IMPORTANCE_DEFAULT
        val channel = NotificationChannel(channelId, channelName, importance).apply {
            description = "Notifications when your alarms go off"
            enableLights( lights = true)
            lightColor = Color.BLUE
            enableVibration( vibration = true)
        }

        val notificationManager =
            getSystemService( name = Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel)
        super.onCreate()
    }

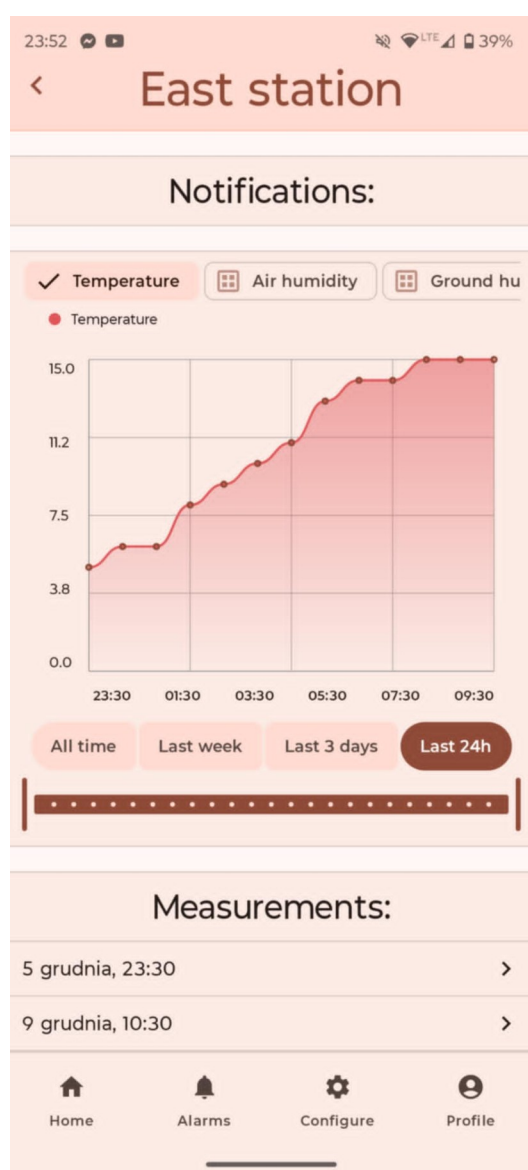
    override fun onNewToken(token: String) {
        Log.d(TAG, msg = "Refreshed token: $token")
        serverInteractor.sendRegistrationToServer(token)
    }
}
```

Listing 6.51 Serwis odpowiedzialny za przesyłanie powiadomień

6.3.9 Szczegóły stacji

Ekran szczegółów stacji jest tym, gdzie użytkownik może nabyć najwięcej danych statystycznych na temat pomiarów wykonywanych przez jego stacje. Kluczowym elementem tego interfejsu jest wykres. Prezentuje on wartości zmierzone w pomiarach. Aby był użyteczny, umożliwia on także filtrowanie prezentowanych danych. Pierwszym sposobem, jest zmiana wyświetlanych parametrów. Drugą opcją jest zaznaczenie żądanego przedziału czasowego, z możliwością dodatkowego ograniczenia tego zakresu suwakiem poniżej. Przykładowe wykresy prezentuje Rysunek 6.20 i Rysunek 6.21.

a)



b)

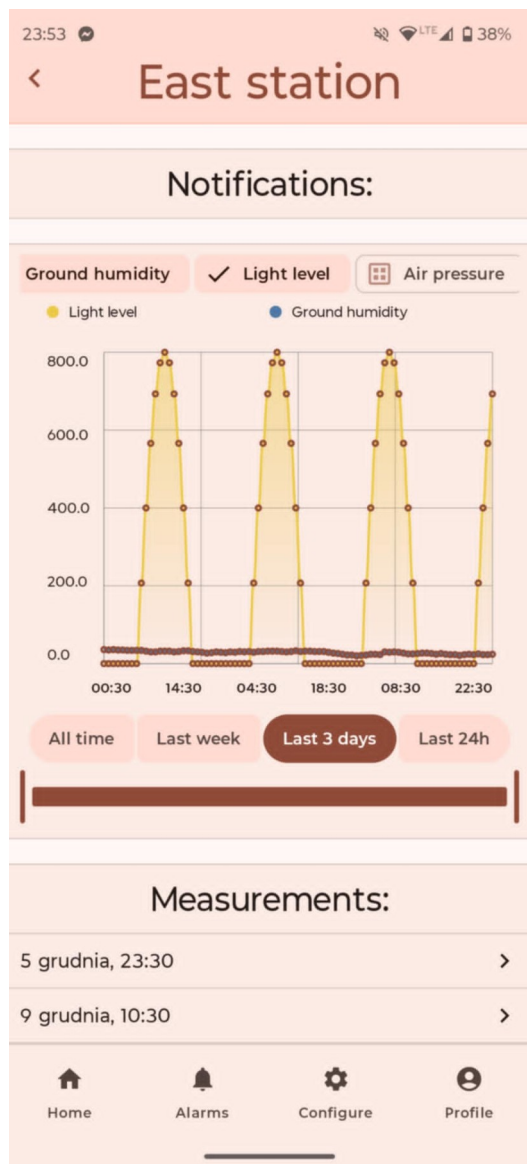


Rysunek 6.20 Ekran stacji

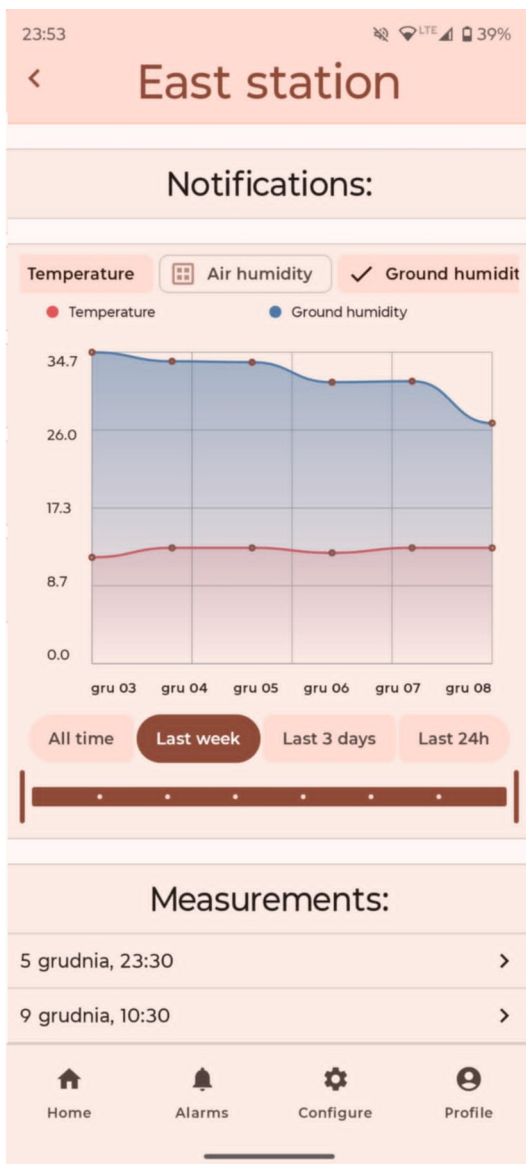
a) Z wykresem temperatury z ostatnich 24h

b) Z wykresem temperatury, wilgotności powietrza i gleby jednocześnie

a)



b)



Rysunek 6.21 Ekran stacji

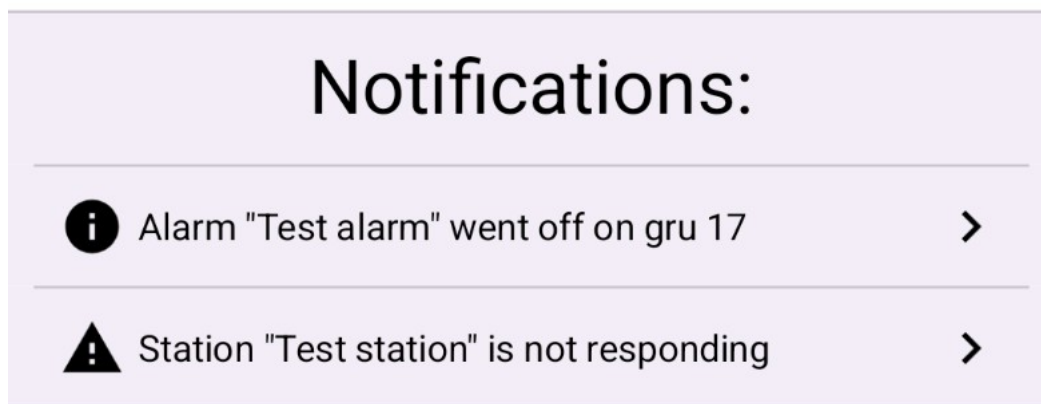
a) Z wykresem poziomu nasłonecznienia i wilgotności gleby na przestrzeni ostatnich 3 dni

b) Z wykresem temperatury i wilgotności gleby na przestrzeni ostatniego tygodnia

Ważnym jest, że przy zaznaczeniu opcji „Last week” i „All time”, wyświetlane są dane uśrednione. Są to odpowiednio wartości średnie dla każdego tygodnia i dnia. Pozwala to śledzić trendy występujące w danych na przestrzeni dłuższego okresu.

Do wyświetlania wykresów została użyta biblioteka „ComposeCharts” rozpowszechniana na licencji Apache-2.0 [5]. Tworzy ona dynamiczne wykresy, doskonałe wpisujące się w wymagania Material Design 3.

Na ekranie szczegółów stacji, znajdują się też dwie inne sekcje. Pierwszą z nich jest sekcja powiadomień. Wyświetlane są tutaj szczegóły dotyczące zgłoszonych błędów lub dowolnych innych wiadomości od stacji lub serwera. Na rysunku 6.22 przedstawione są przykładowe powiadomienia.



Rysunek 6.22 Przykładowe powiadomienia

Ostatnią sekcją na tym ekranie jest lista wszystkich pomiarów jakie wykonała stacja. Kolejne pomiary są widoczne na przewijalnej liście, oznaczone datą i czasem wykonania pomiaru.

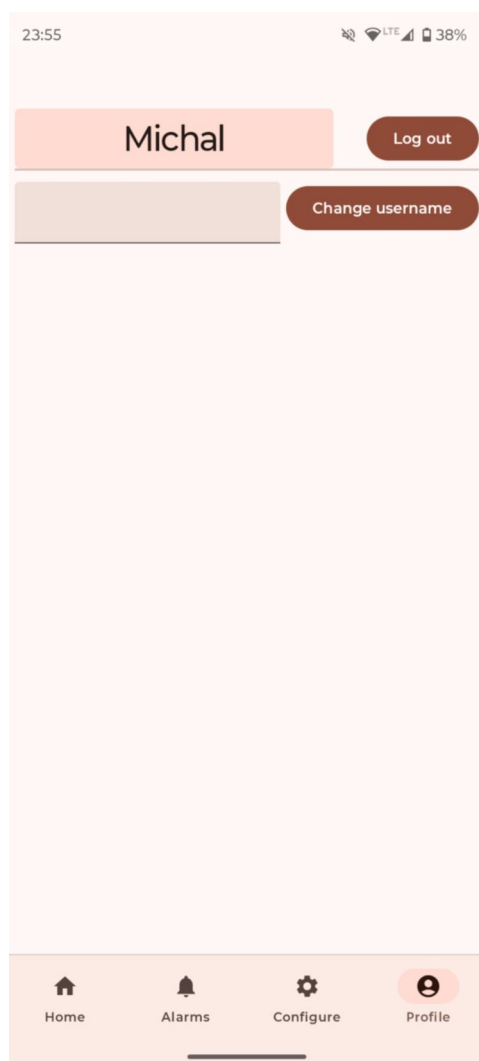
6.3.10 Logowanie

Logowanie jest niezbędne do korzystania z tej aplikacji. Aby przekazywać dane, stacje muszą być kojarzone z użytkownikiem. Dodatkowo, na to samo konto można zalogować się z wielu urządzeń, co jest przydatne na przykład rodzinom. Do konta przypisany jest modyfikowalny pseudonim wyświetlany na głównej stronie aplikacji.

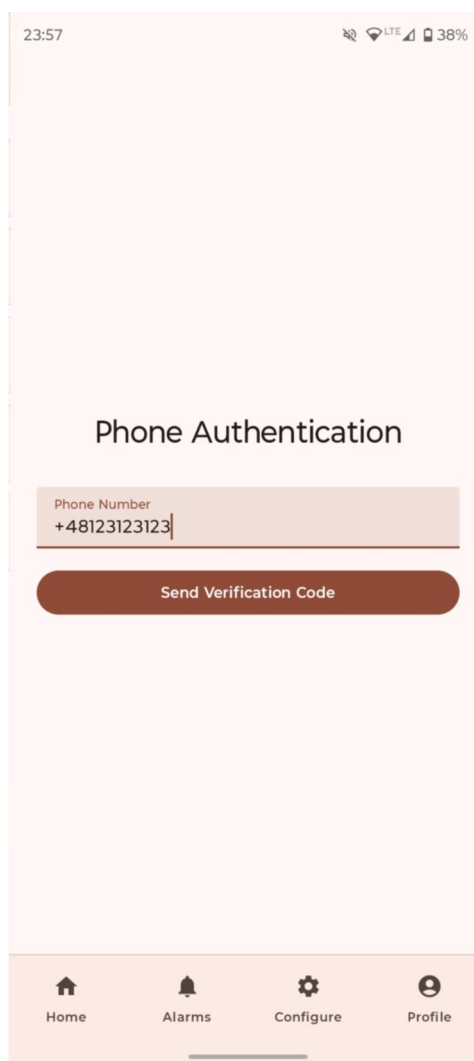
W związku z tym, że system jest skierowany do posiadaczy urządzeń typu smartfon, metodą uwierzytelniania użytkownika jest kod wysyłany na numer telefonu. Jest to w zupełności wystarczające do spełnienia wymagań rozwiązania.

Proces logowania prezentuje Błąd: nie znaleziono źródła odwołania i Rysunek 6.23. Wprowadzone dane są używane do testów.

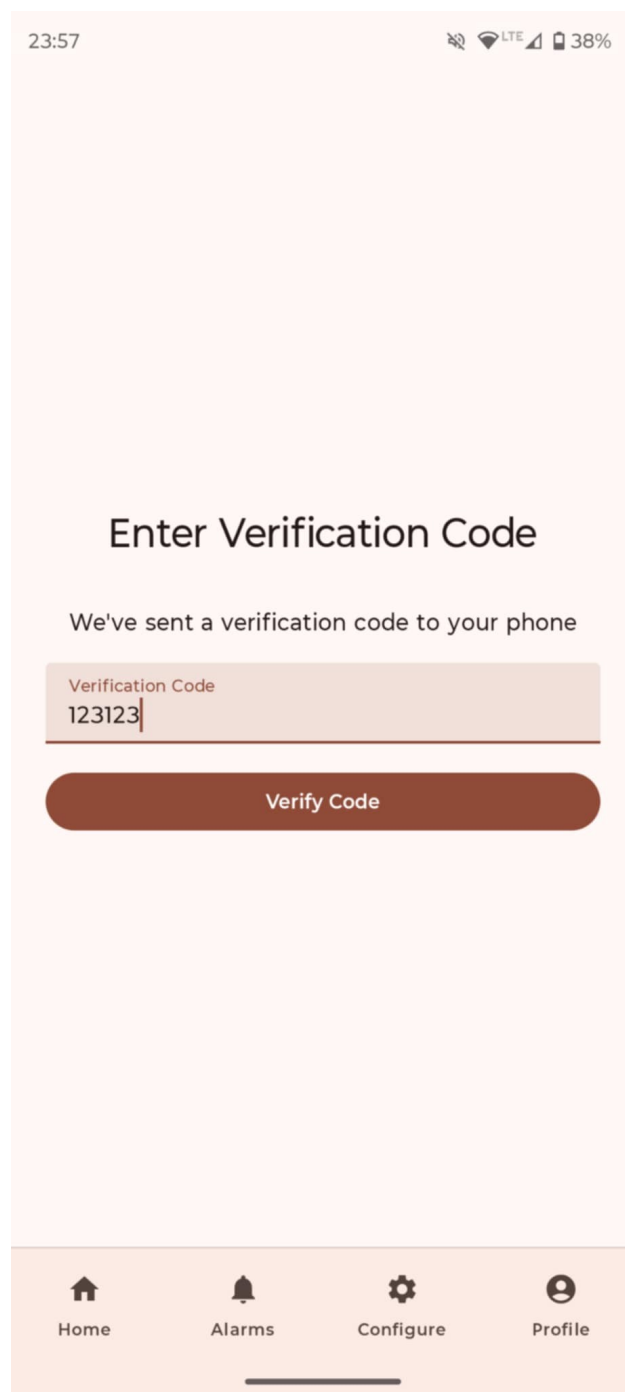
a)



b)



Rysunek



Rysunek 6.23 Ekran potwierdzenia kodu SMS

Wysyłanie kodu weryfikującego zrealizowane jest za pomocą Firebase Authentication [4]. Jest to kompleksowe rozwiązanie bardzo dobrze przystosowane do tworzenia aplikacji mobilnych.

Za udostępnianie obiektu użytkownika pozostałym częściom aplikacji odpowiada repozytorium. Najważniejszym jego elementem jest pole przechowujące informacje o użytkowniku. Jest ono zaprezentowane na Listingu 6.52. Kluczowe jest wywołanie funkcji „addAuthStateListener”, która zapewni aktualizacje na temat dowolnej zmiany w obiekcie użytkownika.

```
override val currentUser: StateFlow<AppUser> = callbackFlow {
    val listener = FirebaseAuth.AuthStateListener { auth ->
        val firebaseUser = auth.currentUser

        if (firebaseUser == null) {
            // User signed out
            trySend( element = AppUser())
            return@AuthStateListener
        }

        launch {
            appUserRepository
                .getUserById(firebaseUser.uid)
                .collect { user ->
                    trySend( element = user)
                }
        }
    }

    firebaseAuth.addAuthStateListener( p0 = listener)
    awaitClose { firebaseAuth.removeAuthStateListener( p0 = listener) }
}.stateIn(
    scope = CoroutineScope( context = SupervisorJob() + Dispatchers.Main.immediate),
    started = SharingStarted.Eagerly,
    initialValue = AppUser()
)
```

Listing 6.52 Pole przechowujące informacje o użytkowniku aplikacji

7. Testowanie

Poniższy rozdział pokazuje kod testujący części systemu oraz wyniki testów. Dla stacji pomiarowej są to głównie testy manualne, przeprowadzone poprzez weryfikację danych wysyłanych przez stację podczas zmiany warunków środowiska. Część serwerowa została przetestowana funkcjonalnie. Aplikacja mobilna została poddana testom jednostkowym, instrumentowanym, integracyjnym oraz testom interfejsu.

7.1 Testy serwera

Do testów aplikacji serwerowej zostały wykorzystane biblioteki Jest i Supertest, powszechnie używane do testowania aplikacji napisanych w JavaScript. Biblioteka Jest odpowiada za orkiestrację testów, a Supertest za symulację żądań.

Testy zostały napisane w stylu testowania funkcjonalnego. Kod zakłada więc minimalną znajomość kodu (tzw. testy czarnej skrzynki). Do warstwy danych zostały zastosowane fałszywe implementacje, widoczne na listingu 7.1.

```
const mockData = {};  
  
const mockCollection = (collectionName) => ({  
  doc: (docId) => ({  
    get: jest.fn(() => Promise.resolve({  
      exists: mockData[collectionName]?.[docId] ? true : false,  
      data: () => mockData[collectionName]?.[docId],  
    })),  
    set: jest.fn((data) => {  
      if (!mockData[collectionName]) mockData[collectionName] = {};  
      mockData[collectionName][docId] = data;  
      return Promise.resolve();  
    })),  
    delete: jest.fn(() => {  
      delete mockData[collectionName][docId];  
      return Promise.resolve();  
    })),  
  })),  
});  
  
jest.mock("firebase-admin", () => ({  
  initializeApp: jest.fn(),  
  credential: { cert: jest.fn() },  
  firestore: jest.fn(() => ({  
    collection: jest.fn((collectionName) => mockCollection(collectionName)),  
  })),  
  messaging: jest.fn(() => ({  
    send: jest.fn().mockResolvedValue("OK")  
  })),  
}));  
  
const resetMockData = () => {  
  for (const col in mockData) delete mockData[col];  
};
```

Listing 7.1 Przygotowanie testów serwera

Przykładowy kod testów serwera pokazuje listing 7.2. Testowany w nim jest punkt końcowy zapisywania pomiaru

```
beforeEach(() => resetMockData());

describe("READING ENDPOINT - functional tests", () => {
  beforeEach(async () => {
    await request(app)
      .post("/registerDevice")
      .send({
        station_id: "stationTest1124",
        password: "abc123",
        name: "Test Stations",
        ownerId: "user01",
      });
  });

  test("TC-03: Save measurement success", async () => {
    const res = await request(app)
      .post("/reading")
      .send({
        station_id: "stationTest1",
        password: "abc123",
        humidityAir: 40,
        humiditySoil: 15,
        temperatureAir: 21.5,
        pressureAir: 1012,
        sunlight: 300,
      });

    expect(res.status).toBe(200);
    expect(res.body.ok).toBe(true);
  });

  test("TC-04: Save measurement wrong password", async () => {
    const res = await request(app)
      .post("/reading")
      .send({
        station_id: "stationTest1",
        password: "wrongPass",
        temperatureAir: 21.5,
      });

    expect(res.status).toBe(401);
    expect(res.body.error).toBe("Invalid password");
  });

  test("TC-05: Save measurement missing fields", async () => {
    const res = await request(app).post("/reading").send({});

    expect(res.status).toBe(400);
  });

  test("TC-06: measurement triggers an alarm", async () => {
    const res = await request(app)
      .post("/reading")
      .send({
        station_id: "stationTest1",
        password: "abc123",
        temperatureAir: 99,
      });

    expect(res.status).toBe(200);
    expect(res.body.ok).toBe(true);
  });
});
```

Listing 7.2 Przykładowe testy serwera

Wszystkie testy przechodzą pomyślnie (listing 7.3). W związku z niewielkim rozmiarem aplikacji, jest ich bardzo mało, lecz wyczerpuje to przypadki testowe.

```
Test Suites: 3 passed, 3 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        2.16 s
Ran all test suites.
```

Listing 7.3 Wyniki testów serwera

7.2 Testy aplikacji mobilnej

Testy aplikacji mobilnej w środowisku Android przebiegają podobnie jak testy innych aplikacji zawierających interfejs użytkownika. Ważnym aspektem jest przystosowanie jej do obsługi różnych urządzeń i wersji systemu operacyjnego.

7.2.1 Testy interfejsu

Najważniejszym aspektem testów interfejsu jest tak zwany podgląd (ang. preview) interfejsu. Deklaracja podglądu została zaprezentowana na rysunku 7.1.

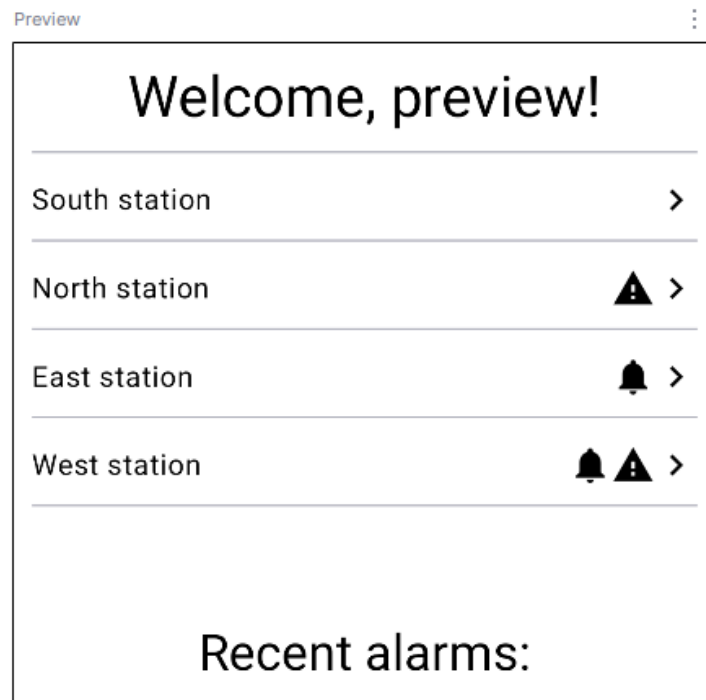
Tworzenie zmiennej „uiState” ma na celu symulację działania ViewModelu. Na tej podstawie można stwierdzić, że taki rodzaj testowania jest podobny do testów jednostkowych.

```
@Preview(showBackground = true)
@Composable
fun Preview() {
    val uiState = HomeUiState(
        isAnonymous = false,
        username = "preview",
        stations = listOf(
            StationSummaryItemUiState(stationId = "1", name = "South station"),
            StationSummaryItemUiState(stationId = "2", name = "North station", hasError = true),
            StationSummaryItemUiState(stationId = "3", name = "East station", hasNotification = true),
            StationSummaryItemUiState(
                stationId = "4",
                name = "West station",
                hasError = true,
                hasNotification = true
            ),
        ),
    )
}

GarPomTheme {
    HomeScreenContent(uiState, onStationSummaryClicked = {}, onRecentAlarmOccurrenceClicked = {})
}
```

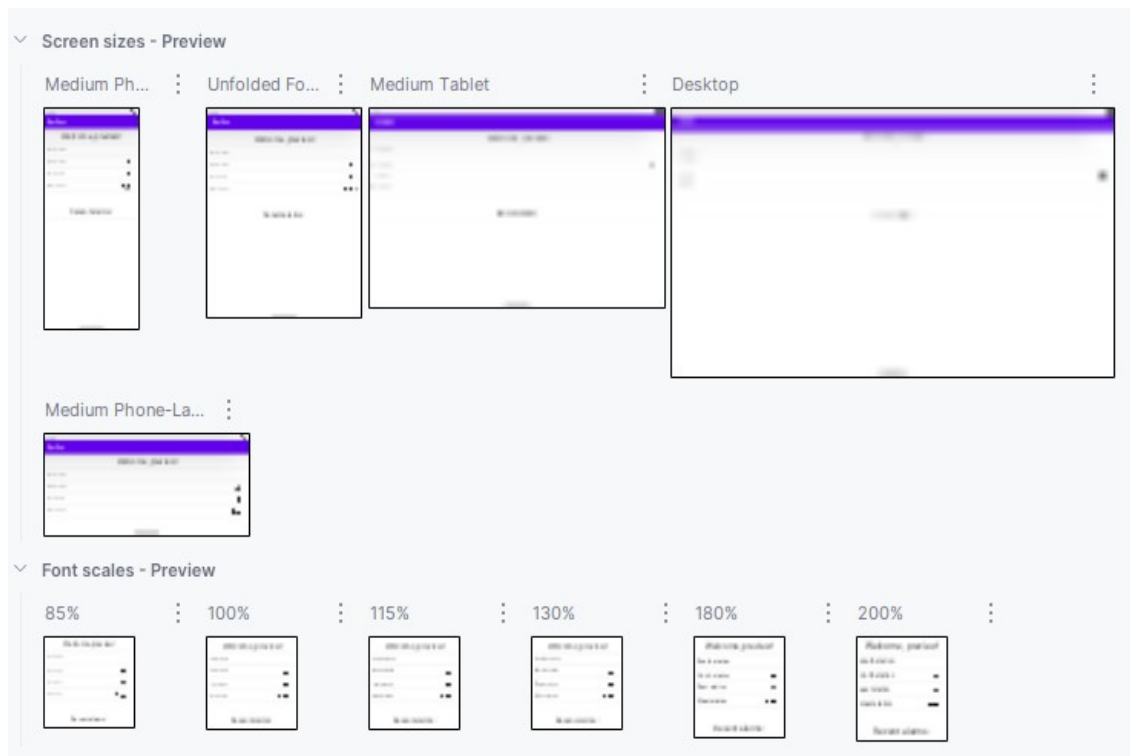
Rysunek 7.1 Tworzenie podglądu interfejsu użytkownika

Android Studio dla plików w których zadeklarowana jest taka funkcja generuje podgląd wywołanego ekranu, co widać na rysunku 7.2.



Rysunek 7.2 Przykładowy podgląd interfejsu

Podgląd w Android Studio oferuje też opcję „UI check”. Powoduje ona wygenerowanie podglądu dla różnych konfiguracji urządzenia. Najważniejsze są opcje zobaczenia interfejsu dla różnych wielkości ekranu i skali czcionki. Taki test pozwala wykryć niewłaściwe zachowanie i je skorygować. Przykładowe wygenerowane obrazy pokazuje Rysunek 7.3. Są one bardzo rozmazane, aby nie obciążać komputera na ekranie podsumowania.



Rysunek 7.3 Ekran funkcji "UI check"

Na te dwa sposoby zostały przetestowane wszystkie ekrany aplikacji. Zdefiniowanie podglądów we wczesnym etapie wytwarzania interfejsu pomogło w znacznym stopniu skrócić czas pracy i zminimalizować ilość błędów.

7.2.2 Testy jednostkowe

Testy jednostkowe zostały zastosowane przede wszystkim dla ViewModel-i aplikacji. Pozwoliło to ograniczyć manualne testowanie. Do tworzenia „zaślepek” (ang. mock) na funkcje i klasy została użyta biblioteka Mockk [3]. Same testy używają dobrze znanej biblioteki Junit.

```
@OptIn(markerClass = ExperimentalCoroutinesApi::class)
class HomeViewModelTest {

    private lateinit var authRepository: AuthRepository
    private lateinit var stationSummaryManager: StationSummaryManager
    private lateinit var alarmOccurrencesListManager: AlarmOccurrencesListManager

    @Before
    fun setup() {
        authRepository = mockk()
        stationSummaryManager = mockk()
        alarmOccurrencesListManager = mockk()
    }
}
```

Listing 7.4 Przykładowa konfiguracja testów

```

@Test
fun `uiState should combine values from repositories`() = runTest {
    // Given
    val user = AppUser(
        id = "123",
        username = "testUser",
        isAnonymous = false
    )

    val stations = listOf(
        StationSummaryItemUiState( stationId = "st1", name = "Station 1", hasNotification = true, hasError = true),
        StationSummaryItemUiState( stationId = "st2", name = "Station 2")
    )

    val alarms = listOf(
        RecentAlarmOccurrenceItemUiState( alarmName = "alarm1"),
        RecentAlarmOccurrenceItemUiState( alarmName = "alarm2")
    )

    every { authRepository.currentUser } returns MutableStateFlow( value = user)
    every { stationSummaryManager.stationsSummaryForUser() } returns flowOf( value = stations)
    every { alarmOccurrencesListManager.recentAlarmOccurrences() } returns MutableStateFlow(
        value = alarms
    )

    val viewModel = HomeViewModel(
        authRepository,
        stationSummaryManager,
        alarmOccurrencesListManager
    )

    // When
    val result = viewModel.uiState.first()

    // Then
    Assert.assertEquals( expected = false, actual = result.isAnonymous)
    Assert.assertEquals( expected = "testUser", actual = result.username)
    Assert.assertEquals( expected = stations, actual = result.stations)
    Assert.assertEquals( expected = alarms, actual = result.recentAlarmOccurrences)
}

```

Listing 7.5 Przykładowy test jednostkowy

Rysunek 7.4 prezentuje wyniki testów utworzonych dla ViewModel-i. Aplikacja pomyślnie spełnia wszystkie wymagania testowe.

✓ Test Results	12 sec 671 ms
✓ AlarmConfigScreenViewModelTest	3 sec 516 ms
✓ resetFromUiState_copies_correct_values	3 sec 357 ms
✓ resetSliders_restores_original_ranges	30 ms
✓ saveStates_calls_manager_and_sets_GoBack	115 ms
✓ init_loads_and_emits_ConfigData	14 ms
> ✓ AlarmsScreenViewModelTest	396 ms
✓ AuthViewModelTest	4 sec 176 ms
✓ signInWithPhoneCredential returns InvalidCredential	3 sec 621 ms
✓ startPhoneNumberVerification returns Error	129 ms
✓ signInWithPhoneCredential returns Error	55 ms
✓ signInWithPhoneCredential returns Success	77 ms
✓ startPhoneNumberVerification catches exception	64 ms
✓ verifyCode uses credential and signs in	86 ms
✓ startPhoneNumberVerification returns auto verification	94 ms
✓ backed resets state and cancels job	50 ms
✓ ConfigureScreenViewModelTest	4 sec 368 ms
✓ showScanResult adds unique devices	1 sec 700 ms
✓ initialConfiguration shows rationale dialog	521 ms
✓ clearDialog resets UI dialog	326 ms
✓ initialConfiguration requests permissions when missing	309 ms
✓ onPermissionsDenied updates dialog state	321 ms
✓ hasBtPermissions returns false when any denied	480 ms
✓ showScanResult ignores duplicates	352 ms
✓ hasBtPermissions returns true when all granted	359 ms
✓ HomeViewModelTest	215 ms
✓ uiState should combine values from repositories	215 ms

Rysunek 7.4 Wynik działania testów ViewModeli

7.2.3 Testy integracyjne

Testy integracyjne w zaprezentowanej aplikacji mobilnej wykorzystują także środowisko Android. Takie testy określane są też mianem testów instrumentowanych. Celem rzetelnej symulacji powiązań w kodzie, najważniejsze zależności zaimplementowane zostały za pomocą interfejsów. W sytuacji testowej, biblioteka Hilt wstrzykuje do kodu inną implementację wspomnianych interfejsów niż przy normalnym działaniu programu. Pozwala to na testowanie działania mechanizmu wstrzykiwania zależności oraz lepiej wspiera ponowne użycie kodu.

```

@Module
@TestInstallIn(
    components = [SingletonComponent::class],
    replaces = [AlarmRepositoryModule::class, AuthRepositoryModule::class, StationRepositoryModule::class]
)
abstract class TestRepositoryModule {
    @Binds
    abstract fun bindAuthRepository(fake: FakeAuthRepository): IAuthRepository

    @Binds
    abstract fun bindAlarmRepository(fake: FakeAlarmRepository): IAlarmRepository

    @Binds
    abstract fun bindStationRepository(fake: FakeStationRepository): IStationRepository
}

```

Listing 7.6 Moduł wstrzykujący zależności testowe

W ten sposób zostały zdefiniowane testowe klasy repozytoriów. Pozostałe elementy systemu uczestniczą w testach. Listing 7.7 przedstawia przykładową implementację.

```

@Singleton
class FakeAuthRepository @Inject constructor() : IAuthRepository {

    override val userExists: Boolean = true
    override val currentUser = MutableStateFlow<AppUser> { value = AppUser( id = "userId", username = "username") }

    override fun signInAnonymously() {}
    override fun signOut() {}
    override suspend fun changeUsername(username: String) = ChangeUsernameResult.Success

    override fun startPhoneNumberVerification(phoneNumber: String, timeout: Long) =
        MutableStateFlow<PhoneVerificationStatus> { value = PhoneVerificationStatus.CodeSent(
            verificationId = "fakeVerificationId"
        ) }

    override suspend fun getCredentialWithCode(verificationId: String, code: String) =
        throw NotImplementedError()

    override suspend fun linkWithCredential(credential: AuthCredential) = VerificationResult.Verified
    override suspend fun signInWithCredential(credential: AuthCredential) = VerificationResult.Verified
}

```

Listing 7.7 Implementacja testowego zamiennika klasy

Na Listingu 7.9 widać kod potrzebny do uruchomienia testu instrumentowanego (korzystającego z emulatora środowiska Android). Potrzebne są specjalne adnotacje i wywołania funkcji. Pokazany na tym listingu test sprawdza, czy w początkowym stanie interfejsu obecne są dane pobrane z testowego zamiennika repozytorium.

```
@HiltAndroidTest
@OptIn( ...markerClass = ExperimentalCoroutinesApi::class, ExperimentalMaterial3Api::class)
@RunWith( value = AndroidJUnit4::class)
class AlarmConfigScreenViewModelHiltTest {

    @get:Rule
    val hiltRule = HiltAndroidRule( testInstance = this)

    @Inject
    lateinit var alarmConfigManager: AlarmConfigManager

    @Before
    fun setup() {
        hiltRule.inject()
    }

    @Test
    fun testNewAlarmStateInitialization() = runTest {
        val viewModel = AlarmConfigScreenViewModel(alarmConfigManager, alarmId = "")
        viewModel.uiState.first { it is AlarmConfigUiState.ConfigData }
        val edit = viewModel.editState.first { it.stations.isNotEmpty() }
        assertEquals( expected = 2, actual = edit.stations.size)
    }
}
```

Listing 7.9 Przykładowy test integracyjny

Tak przygotowane testy mogą zostać uruchomione na dowolnym emulowanym lub fizycznym urządzeniu android. Przykładowe wyniki zaprezentowane są na listingu 7.8.

Status ██████████ 3 passed 3 tests, 2 m 56 s 375 ms		
Filter tests: ✓ ⊗ ↕ ✕ ↑ ↓ 🔍 📄 🔗		
Tests	Duration	Medium_Phone_API_36
✓ Test Results	253 ms	3/3
✓ AlarmConfigScreenViewModelHiltTest	253 ms	3/3
✓ testResetSliders	219 ms	✓
✓ testNewAlarmStateInitialization	2 ms	✓
✓ testSaveStatesCreatesAlarm	32 ms	✓

Listing 7.8 Wyniki testów integracyjnych konfiguracji alarmu

8. Wnioski

Praca inżynierska miała na celu zaprojektowanie i zaimplementowanie systemu pozwalającego na zdalne monitorowanie parametrów środowiskowych w ogrodzie. Prace programistyczne oparto na wyniku działań projektowych, diagramach, schematach i przede wszystkim wymaganiach funkcjonalnych i нефункциональных. Wzięto pod uwagę także dostępne na rynku komercyjne rozwiązania. Ważnym celem części technicznej projektu było osiągnięcie szerokiego zakresu funkcjonalności przy niskich kosztach.

Wszystkie początkowe założone cele zostały spełnione. Udało się stworzyć ergonomiczną aplikację mobilną i zintegrować ją z systemem pomiarowym. Rozwiązanie bardzo dobrze spełnia zadanie monitorowania warunków atmosferycznych i glebowych.

Trudnością podczas projektowania okazało się dobranie technologii, szczególnie serwera i aplikacji mobilnej. Rozwiązania oferowane przez Firebase są wygodne, ale wprowadzają wątpliwości do długoterminowego utrzymania systemu. Do implementacji aplikacji mobilnej wykorzystano najnowsze biblioteki, często będące w fazie eksperymentalnej. Pozwala to osiągnąć nowoczesny wygląd i responsywne działanie, ale naraża kod na brak wsparcia i konieczność zmiany dużych fragmentów.

Innym wnioskiem z implementacji projektu jest to, jak ważne jest definiowanie interfejsów integracji na wczesnym etapie projektowania. Znacznie ułatwia to przede wszystkim testowanie. Jest to istotne zarówno dla testów automatycznych, które mogą skupić się na konkretnych implementacjach, jak i dla testów manualnych. Jest to ważne w przypadku właśnie takich systemów jak ten autorski, gdzie ważnym zasobem dla programistów może być połączenie fizycznych urządzeń. Taki test pozwala między innymi uściślić założenia na temat sprzętu i jego funkcjonalności. Podczas prac nad tym systemem, w ten sposób zdefiniowane konkretne możliwości mikrokontrolera w zakresie takich parametrów Bluetooth jak zasięg i stabilność połączenia. Te właściwości mogą zmieniać swoje wartości w zależności na przykład od zasilania urządzenia.

Naturalnym kierunkiem rozwoju systemu jest wprowadzenie innych rodzajów stacji, mających inne możliwości pomiarowe. System może obsłużyć różne parametry środowiskowe takie jak pH gleby. Stacje mogą też przykładowo rejestrować ruch lub promieniowanie cieplne, celem analizy aktywności zwierząt. Możliwe jest też opracowanie funkcjonalności pozwalającej na automatyczne sterowanie innym urządzeniami, takie jak zraszacze czy automatyczne dachy.

Literatura

- [1] Jaskiewicz A., Inżynieria oprogramowania, Helion, 1997
- [2] Martin Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley Professional, 2002
- [3] Arduino IDE PLC, <https://docs.arduino.cc/software/plc-ide/>, (dostęp: 07.2025)
- [4] Most popular technologies, Web framework and technologies, <https://survey.stackoverflow.co/2025/technology>, (dostęp: 11.2025)
- [5] Android Studio & App tools, <https://developer.android.com/studio>, (dostęp: 06.2025)
- [6] MockK | Mocking library for Kotlin, <https://mockk.io/>, (dostęp: 11.2025)
- [7] Firebase Authentication, <https://firebase.google.com/docs/auth?hl=pl>, (dostęp: 7.2025)
- [8] Github - ehsannarmani/ComposeCharts, <https://github.com/ehsannarmani/ComposeCharts>, (dostęp: 8.2025)
- [9] Create and manage notification channels | Android developers, <https://developer.android.com/develop/ui/views/notifications/channels>, (dostęp: 9.2025)
- [10] Firebase Cloud Messaging, <https://firebase.google.com/docs/cloud-messaging?hl=pl>, (dostęp: 9.2025)
- [11] Thinking in Compose | Jetpack Compose | Android Developers, <https://developer.android.com/develop/ui/compose/mental-model#recomposition>, (dostęp: 9.2025)
- [12] Data Transfer Object, <https://martinfowler.com/eaCatalog/dataTransferObject.html>, (dostęp: 9.2025)
- [13] Guide to app architecture | Android Developers, <https://developer.android.com/topic/architecture>, (dostęp: 10.2025)
- [14] Bluetooth Low Energy, <https://web.archive.org/web/20170310111443/https://www.bluetooth.com/what-is-bluetooth-technology/how-it-works/low-energy>, (dostęp: 12.2016)
- [15] NimBLE-Arduino, <https://github.com/h2zero/NimBLE-Arduino>, (dostęp: 2.2023)
- [16] Adafruit Industries, <https://www.adafruit.com/>, (dostęp: 05.2025)
- [17] Embedded software development popularity, <https://www.jetbrains.com/lp/devecosystem-2021/embedded/>, (dostęp: 11.2025)

- [18] Top 20 Analytics SDK in Android apps, <https://42matters.com/sdk-analysis/top-analytics-sdks#firebase>, (dostęp: 11.2025)
- [19] Google Firebase SDK, <https://42matters.com/sdks/android/firebase>, (dostęp: 11.2025)
- [20] Hilt, <https://dagger.dev/hilt/>, (dostęp: 9.2025)
- [21] Jetpack Compose, <https://developer.android.com/compose>, (dostęp: 11.2025)
- [22] Biblioteki jetpack, <https://developer.android.com/jetpack/androidx/explorer?case=popular>, (dostęp: 11.2025)
- [23] Strona główna projektu Material Design, <https://m3.material.io/>, (dostęp: 11.2025)
- [25] Repozytorium Git języka Kotlin, <https://github.com/JetBrains/kotlin>, (dostęp: 11.2025)
- [24] Strona główna języka Kotlin, <https://kotlinlang.org/>, (dostęp: 11.2025)
- [26] Figma | Narzędzie do projektowania interfejsów online, <https://www.figma.com/pl-pl/>, (dostęp: 12.2025)
- [27] Bezprzewodowy czujnik wilgotności i temperatury gleby Spacetronik SP-ST01, <https://spacetronik.pl/pl/products/tester-wilgotnosci-i-temperatury-gleby-spacetronik-sp-st01-13570.html>, (dostęp: 11.2025)
- [28] ECOWITT GW1000 Operation Manual, <https://www.manualslib.com/manual/1521887/Ecowitt-Gw1000.html>, (dostęp: 11.2025)
- [29] Ambient Weather WS-2902 Home Wi-Fi Weather Station with Wi-Fi Remote Monitoring and Alerts, <https://ambientweather.com/ws-2902-smart-weather-station>, (dostęp: 11.2025)